

Elmir **BABOVIĆ**
Denis **MUŠIĆ**



OSNOVE WEB RAZVOJA I DIZAJNA

2021

Izdavači: Elmir Babović i Denis Mušić

Autori: Elmir Babović i Denis Mušić

Naslov djela: Osnove web razvoja i dizajna

Urednik: Elmir Babović

Recenzenti: prof.dr. Srđan Nogo i prof.dr. Zanin Vejzović

Lektor: Amina Balić, MA

Izdanje: prvo izdanje

Godina izdavanja: 2021

URL: <https://wrd.fit.ba/>

CIP - Katalogizacija u publikaciji

Nacionalna i univerzitetska biblioteka Bosne i Hercegovine, Sarajevo

004.438 HTML

BABOVIĆ, Elmir

Osnove web razvoja i dizajna [Elektronski izvor] / Elmir Babović, Denis Mušić. - El. knjiga. - Mostar : autori, 2021

Način pristupa (URL): <https://wrd.fit.ba/mat/OsnoveWRD.pdf>. - Nasl. sa nasl. ekrana. - Opis izvora dana 8. 12. 2021.

ISBN 978-9926-488-23-9

1. Mušić, Denis

COBISS.BH-ID 46535686

ELMIR BABOVIĆ

DENIS MUŠIĆ

OSNOVE WEB RAZVOJA I DIZAJNA

Mostar, 2021

Sadržaj

Sadržaj	iii
Popis slika	vi
Predgovor	1
Uvod	2
Historija Web-a.....	3
Upute za praćenje primjera.....	5
1. HTML	7
1.1 Uvod u HTML	7
1.2 Struktura HTML dokumenta – DOM.....	9
1.3 Tagovi	11
1.3.1 Atributi.....	12
1.3.2 Apsolutne i relativne putanje	15
1.4 Podjela tagova	17
1.4.1 Osnovni HTML tagovi	18
1.4.2 Tagovi za formatiranje.....	19
1.4.3 Tagovi za forme i <i>input</i> elemente.....	21
1.4.3.1 HTML 5 Validacija forme	27
1.4.4 Tagovi za liste	27
1.4.4.1 Uređena lista	28
1.4.4.2 Neuređena lista	29
1.4.4.3 Definijska lista	29
1.4.5 Tagovi za tabele.....	30
1.4.6 Tagovi za linkove	31
2. Osnove CSS-a	33
2.1 CSS selektori	33
2.2 Način uključivanja CSS stilova u HTML	36
2.3 Specifičnost selektora.....	39

2.4 Pozicioniranje elemenata	41
2.4.1 Position absolute	42
2.4.2 Position relative	44
2.4.3 Position fixed	47
2.4.4 Position sticky	48
2.4.5 Flex box	50
2.5 Pseudo-klase i pseudo-elementi	52
2.6 Relacije selektora ugnježđenih (Descendant) elemenata – CSS kombinatori	56
2.7 Display property i inline i block kao display obilježja	57
2.8 Box model	59
2.9 Responsivnost i Media query	61
2.10 Boje, fontovi i ostala obilježja	63
3. Osnove JavaScript jezika	69
3.1 Uvod u JS	69
3.2 Client Side i Server Side scripting	70
3.3 Način uključivanja JavaScript u HTML dokument	71
3.3 JS varijable	73
3.4 Operatori	77
3.5 Osnovne programske strukture	80
3.6 JS funkcije	81
3.7 JavaScript i DOM	85
3.8 Dijaloški prozori	90
3.9 Objekti	92
3.10 HTTP request	99
3.11 Greške - exception	101
3.12 Event listener	103
4. JavaScript razvojni okviri	107
4.1 Ember.JS	110
4.1.1 Instalacija potrebnih <i>Ember</i> komponenti	111

4.1.2 Razvoj Ember aplikacije	114
4.2 React.JS.....	119
4.2.1 Instalacija potrebnih React komponenti	120
4.2.2 Razvoj React aplikacija	122
4.3 Vue.JS	135
4.3.1 Razvoj Vue aplikacija	136
4.3.2 Instalacija potrebnih Vue komponenti	149
4.4 Osnovne karakteristike razvojnih okvira	151
5. Principi interakcijskog i vizualnog dizajna	154
5.1 Dizajn interakcije	154
5.2 Vizualni dizajn.....	164
Reference	169
Prilozi	170
CSS paleta boja s nazivima i pripadajućim vrijednostima	170

Popis slika

Ilustracija 1 - Prvi Web server.....	4
Ilustracija 2 - Pristup Inspect dijelu Google Chrome preglednika	6
Ilustracija 3 - Opcije Inspect prozora	6
Ilustracija 4 - Interpretacija koda na Web pregledniku	8
Ilustracija 5 - DOM - Document Object Model.....	9
Ilustracija 6 - Primjer <hr> taga	12
Ilustracija 7 - Primjer taga bez dodanih atributa	13
Ilustracija 8 - Prikaz slike u tagu.....	14
Ilustracija 9 - Primjer strukture direktorija Web site-a	16
Ilustracija 10 - Izgled <h1> - <h6> i <p> tagova	19
Ilustracija 11 - Izgled nekih osnovnih tagova za formatiranje	21
Ilustracija 12 - Primjer HTML5 forme	24
Ilustracija 13 - Pregled podataka poslanih sa HTML5 forme na Web server POST metodom	25
Ilustracija 14 - Pregled podataka poslanih sa HTML5 forme na Web server GET metodom	26
Ilustracija 15 - Izgled poruke upozorenja pri HTML5 validaciji.....	27
Ilustracija 16 - Izgled uređene liste.....	28
Ilustracija 17 - Izgled uređene liste sdodatnim atributima	29
Ilustracija 18 - Izgled neuređene liste	29
Ilustracija 19 - Izgled definicijske liste	30
Ilustracija 20 - Izgled HTML5 tabele	31
Ilustracija 21 - Korištenje nepostojećih elemenata - Neispravno stiliziranje	34
Ilustracija 22 - Prikaz implementacije CSS stila	36
Ilustracija 23 - Prikaz implementacije različitih metoda uključivanja CSS stilova	37
Ilustracija 24 - Primjer specifičnost selektora po imenu elementa	40
Ilustracija 25 - Redanje prioriteta	40
Ilustracija 26 - Implementacija prioriteta	41
Ilustracija 27 - Primjer neispravnog pozicioniranja	44
Ilustracija 28 - Izgled Flexbox elementa	52
Ilustracija 29 - Simulacija dinamičkog menija korištenjem pseudoklase	54
Ilustracija 30 - Primjer implementiranja pseudo-elementa	55
Ilustracija 31 - Izgled block elemenata	59
Ilustracija 32 - Izgled inline-block elemenata	59
Ilustracija 33 - Izgled inline elemenata	59
Ilustracija 34 - Box model HTML elementa	60

Ilustracija 35 - Realni izgled box modela u Google Chrome pregledniku	61
Ilustracija 36 - Outline elementa	61
Ilustracija 37 - Implementacija "float: right" obilježja.....	64
Ilustracija 38 - Implementacija "float: left" obilježja.....	64
Ilustracija 39 - Prikaz opcija bordera	67
Ilustracija 40 - Rezultat izvršenja internog JavaScript koda	72
Ilustracija 41 - Dinamička promjena tipa podatka dodjelom različitih vrjednosti	74
Ilustracija 42 - Izgled početne stranice Ember projekta sa generisanim postavkama	114
Ilustracija 43 - Redoslijed interakcije komponenti unutar Ember aplikacije.....	115
Ilustracija 44 Prikaz objekata iz modela koji predstavljaju listu predmeta	118
Ilustracija 45 - Izgled početne stranice React projekta sa generisanim postavkama	122
Ilustracija 46 - Prikaz komponente Predmeti	128
Ilustracija 47 - Prikaz ponuđenih i odabranih predmeta korištenjem komponenti u React-u	134
Ilustracija 48 - Komponente u kontekstu razvoja Vue aplikacija	135
Ilustracija 49 - Pristup osobinama Vue objekta.....	137
Ilustracija 50 - Modifikacija vrijednosti HTML elemenata putem Vue objekta	138
Ilustracija 51 - Pohrana podataka nastalih interakcijom sa korisnikom.....	140
Ilustracija 52 - Upravljanje stilovima elemenata	143
Ilustracija 53 - Odabir i ispis predmeta.....	149
Ilustracija 54- Izgled početne stranice Vue projekta sa generisanim postavkama ..	151
Ilustracija 55 - Dizajn interakcije stranice www.fit.ba	157
Ilustracija 56 Ikone za označavanje menija - dostupnih opcija	159
Ilustracija 57 Elementi Material Design aplikacijske trake	160
Ilustracija 58 Primjer navigacijske putanje	160
Ilustracija 59 Definisanje formata podataka	161
Ilustracija 60 Mapa web stranice Fakulteta informacijskih tehnologija.....	162
Ilustracija 61 Izgled predloška dizajna web stranice www.fit.ba	165
Ilustracija 62 Naglašavanje elemenata forme	165
Ilustracija 63 Hijerarhija na primjeru obavijesti	166
Ilustracija 64 Obavijest korisniku o bitnim događajima	167
Ilustracija 65 Serif i Sense Serif kategorije fontova	168

Predgovor

Moderno poslovanje u 21. vijeku je nezamislivo bez korištenja interneta i mogućnosti koje internet pruža. Od komunikacijskih mogućnosti, preko kupoprodaje, marketinga pa sve do društvenih mreža, internet je postao sveprisutna platforma preko koje funkcionira moderno društvo. Teško je pobrojati sve aspekte ljudskog života na koje je internet utjecao i promijenio ih u odnosu na period prije ekspanzije interneta. Opšti trend migriranja poslovnih aplikacija na web omogućava veliki broj prednosti u odnosu na klasične desktop bazirane klijent-server aplikacije koje su bile zastupljene 80-tih godina prošlog vijeka. Ubrzani razvoj i dostupnost mrežne i komunikacijske opreme, te dostupnost internet konekcije, omogućio je promjenu paradigme u oblasti softverskog inženjerstva. Poslovne aplikacije većine ozbiljnih privrednih subjekata su već odavno postale web bazirane. Navedeni trendovi su rezultirali uvećanjem potražnje za kvalificiranim kadrom iz oblasti web razvoja i dizajna.

Knjiga je prvenstveno namijenjena studentima dodiplomskog studija Fakulteta informacijskih tehnologija Univerziteta „Džemal Bijedić“ u Mostaru, studentima tehničkih fakulteta koji izučavaju područje web razvoja i dizajna, te ostalim čitaocima koji žele savladati osnove pomenutih oblasti.

Poštovani studenti, praćenjem trendova u svijetu, te pravovremenim prilagođavanjem promjenama i učenjem novih tehnologija i metoda, omogućavate sebi da učestvujete u tim procesima promjene, osigurate svoje mjesto u tržišnoj utrci poslije završetka studija i time kreirate svjetliju budućnost za sebe i za naše društvo u cjelini.

Uvod

Pojavom interneta kao globalne mreže, počeli su se uviđati potencijali virtualnog prostora kako za potrebe poslovnih procesa, prevashodno marketinga i kupoprodaje, tako i za ostale društvene aktivnosti. Početkom 21. vijeka, uvode se tehnologije koje omogućavaju ubrzan razvoj poslovanja u virtualnom okruženju. Sve više kompanija počinju migrirati svoje kupoprodajne kapacitete na internet. Potencijal interneta kao globalne mreže sve više mijenja tržište i uvodi jednu potpuno novu dimenziju poslovanja, ranije nepoznatu čovječanstvu. U toku prvih deset godina 21. vijeka, polako se povlače iz upotrebe klasična desktop rješenja standardnih klijent-server aplikacija i postepeno se zamjenjuju web-baziranim aplikacijama.

Tehnologije koje su omogućile ubrzani razvoj web-baziranih aplikacija, društvenih mreža te ostalih interaktivnih sadržaja na internetu se konstantno razvijaju. Ne bi bilo pogrešno tvrditi da su, pored vještačke inteligencije, robotike i mehatronike, internet i internet vezane tehnologije najbrže rastuća grana informacijskih tehnologija.

U pogledu studija softverskog inženjerstva, računarskih nauka i poslovne informatike, kompetencije iz oblasti web tehnologija i web razvoja i dizajna su postale nezaobilazan dio ukupnog znanja koje jedan inženjer mora posjedovati kako bi mogao biti konkurentan, kako na domaćem, tako i na međunarodnom tržištu rada.

U pogledu tehnologija, alata, framework-a, biblioteka i sl., situacija je vrlo dinamična i u prethodnom periodu se može vidjeti da se konstantno razvijaju novi front-end i back-end framework alati. U ovom djelu, glavni fokus se stavlja na front-end dio kompletnog procesa i obrađuju se front-end alati i jezici.

Knjiga je podijeljena na četiti glavna dijela u kojem se obrađuju tri jezika i jedna biblioteka. Na početku se čitalac uvodi u HTML jezik pomoću kojeg opisujemo sadržaj HTML stranice. U drugom dijelu čitalac se upoznaje sa CSS jezikom pomoću kojeg će se stilizirati i dizajnirati HTML stranica, s obzirom na to da osnovni HTML ne pruža prevelike mogućnosti dizajna sadržaja stranice. Dakle, u prva dva poglavlja će biti obrađena dva jezika i to:

- HTML – jezik kojim definiramo šta želimo prikazati na stranici

- CSS – jezik kojim definiramo kako treba da izgleda ono što smo prikazali pomoću HTML-a.

Bitno je napomenuti da su oba spomenuta jezika zapravo opisni jezici. Opisni jezici kao što je HTML razlikuju se od nekog programskog jezika kao što je C++, Java ili Python u tome što se programski jezici koriste kako bi se napisao kod nekog programa koji se prevodi, linkuje i izvršava, dok se opisni jezici ne prevode i njihov kod se ne izvršava. Kod opisnih jezika, u ovom slučaju HTML i CSS, se ne izvršavaju, već se jednostavno učitavaju u Browser (Internet preglednik) na računaru ili nekom drugom uređaju, te se njihov sadržaj tumači od strane Preglednika i prikazuje se sadržaj namijenjen nekom korisniku. Dakle, Browser mora „znati“ kako će prikazati sadržaj i kako će „protumačiti“ ili interpretirati ono što mu programer želi reći svojim HTML i CSS kodom.

Neki autori se neće složiti da je CSS uopšte jezik. CSS znači *Cascade Styles Sheet* i zapravo označava kaskadne stilove. Umjesto da ulazimo u diskusiju vezanu za terminologiju, mi ćemo CSS jednostavno koristiti u sklopu naših stranica, a raspravu o terminologiji ćemo prepustiti nekim drugim autorima.

Treće poglavlje ove knjige obrađuje JavaScript jezik. JavaScript je **objektno-orijentirani** i **interpretirani** programski jezik. JavaScript ima još mnogo atributa. Svi ovi pojmovi, kao i još neki, će biti objašnjeni u trećem poglavlju.

U četvrtom poglavlju će biti napravljen pregled JavaScript baziranih razvojnih okvira. Postoji niz prednosti korištenja razvojnih okvira ili *framework* i JavaScript baziranih biblioteka u odnosu na JavaScript. Međutim, postoje i određena ograničenja učenja takvih razvojnih okvira i biblioteka.

I na kraju, u petom poglavlju, čitalac će biti upoznat s principima dobrog web dizajna.

Iskreno se nadamo da će ova knjiga biti korisna čitaocima koji tek počinju s web dizajnom i razvojem, kao i iskusnijim programerima koji žele proširiti svoje znanje u nekim od pomenutih oblasti.

Historija Web-a

Prve ideje za razvoj interneta se pojavljuju još 1960-ih godina. Ono što se tada razvijalo u okviru ARPA (Advanced Research Projects Agency) nije internet u onom obliku u kojem ga danas poznajemo. Grupa naučnika se bavila razvojem ideje sve do kraja 60-ih godina kada ta ideja zaživljava u obliku ARPANET-a koji se nastavio razvijati i godinama kasnije. Koristio se za umrežavanje samo nekoliko institucija.

Već u nekoliko prvih godina poslije razvoja ARPANET-a, početkom 70-ih godina počinju se razvijati i prvi protokoli za umrežavanje. Tih godina su u okviru različitih institucija i organizacija nastale još neke velike ideje koje su nas dovele do interneta bez kojeg je danas život gotovo nezamisliv. Jedna od ideja nastalih baš tad jeste i Ethernet koji je prvobitno nazvan Alto Aloha. Nešto kasnije, razvijena je i ideja prenosa podataka koristeći telefonske linije.

Zahvaljujući dotadašnjim otkrićima, 80-ih godina se već povezuje nekoliko mreža. To je omogućilo pojavljivanje komunikacije korištenjem e-maila. U tom periodu, već se razvija i prva LAN mreža. Kreiran je DNS, te je 1985. godine kreirana i prva domena symbolics.com. Tim Berners-Lee je 1989. u CERN-u razvio WWW. Njegov prvi prijedlog od 1998. i drugi 1999.¹ godine smatraju se prvim koracima u razvoju onoga što danas imamo gao globalnu mrežu – internet.

Također, tu je početkom 90-ih godina napravljen i prvi server te se još uvijek nalazi u CERN-u.



Ilustracija 1 - Prvi Web server

Prvi server u USA se počeo koristiti u decembru 1991. godine u laboratoriju za fiziku na Stanford univerzitetu. Već 1993. godine u okviru NSCA (National Center for Supercomputing Applications) kreirana je prva verzija Mosaic preglednika. U tom periodu pokazala se potreba da se protokoli i koncepti na kojima je zasnovana nova

¹ <http://cds.cern.ch/record/369245/files/dd-89-001.pdf>

mreža i standardiziraju. Koraci u tom pravcu su napravljeni 1994. godine kada je Tim Berners-Lee prešao iz CERN-a na MIT i formirao W3C (*Međunarodni WWW konzorcium*).

Krajem 20. i početkom 21. vijeka dešava se ekspanzija interneta, veliki broj kompanija migrira svoje poslovanje na internet, tako da internet postaje nezaobilazan dio poslovnog svijeta, ne samo zbog komunikacijskih mogućnosti, nego i zbog potencijala pristupa većem broju klijenata u dijelovima planete gdje firme nisu mogle otvoriti svoje poslovnice. Početkom 21. vijeka dolazi do fenomena ekspanzije tzv. *Dot-com* kompanija, odnosno kompanija koje svoje poslovanje većinom baziraju na globalnoj mreži internet. Neke od tih kompanija su, naravno, propale, ali neke od njih još uvijek uspješno posluju i predstavljaju svjetske poslovne gigante kao što je eBay ili Amazon.

Također, pojavio se i dodatni fenomen koji je zauvijek izmijenio, ne samo poslovni svijet, nego društvo u cjelini. Taj fenomen danas poznajemo kao društvene mreže. Veliki broj stručnjaka, ne samo iz oblasti IT-a, već i iz drugih oblasti kao što su sociologija, psihologija i sl., i danas raspravljaju o pozitivnim i negativnim uticajima. Naravno, pozitivni utjecaji interneta su daleko veći od nekog malog broja negativnih utjecaja.

Razvojem mrežne i komunikacione opreme, te razvojem mobilnih tehnologija, internet je omogućio i razvoj potpuno novih disciplina kao što su IoT (*Internet of Things*). Postoje mnoge definicije IoT oblasti, ali pošto se radi o relativno novoj disciplini, proces definisanja i klasifikacije u formalno akademskom smislu, još uvijek nije završen. Po nekim autorima, IoT predstavlja globalnu mrežu računara i drugih uređaja s procesorom ili mikrokontrolerom, embedded uređaja, mašina i objekata, te ljudi i ostalih živih bića koji su povezani u smislu da mogu razmijenjivati podatke preko globalne mreže. IoT se bazira na M2M ili *machine-to-machine* komunikaciji koja se može odvijati bez ljudske intervencije.

Dot-com kompanije, društvene mreže i IoT su samo neki od trendova koji su omogućeni razvojem interneta i sve većom dostupnošću internet mreže putem fiksnih i mobilnih konekcija.

Upute za praćenje primjera

Preglednik koji je korišten u izradi ovog materijala je Google Chrome. Svi primjeri koji su navedeni testirani su u tom pregledniku. Savjetujemo čitaoca da se upozna s mogućnostima preglednika. Prilikom testiranja nekih primjera bit će korištena opcija

„Inspect“ kojoj se može pristupiti tako što će se uraditi klik desnom tipkom miša na prostor predviđen za učitavanje sadržaja web-stranice te izabrati „Inspect“ opciju. Alternativna prečica za ovu opciju je kombinacija tipki Ctrl+Shift+I na tastaturi.

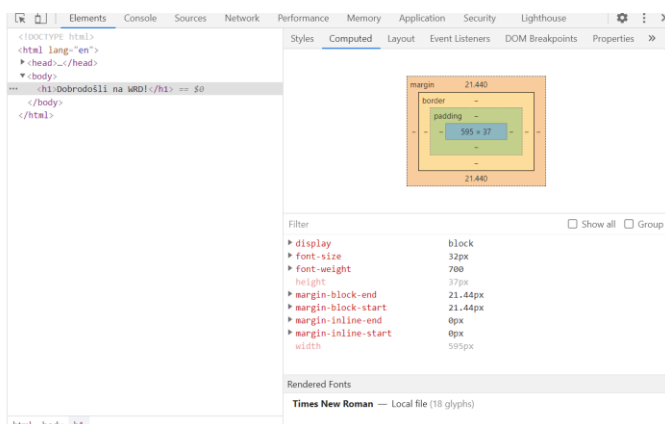
Dobrodošli na WRD!

Back	Alt+Left arrow
Forward	Alt+Right Arrow
Reload	Ctrl+R
Save as...	Ctrl+S
Print...	Ctrl+P
Cast...	
Translate to bosanski	
View page source	Ctrl+U
Inspect	Ctrl+Shift+I

Ilustracija 2 - Pristup Inspect dijelu Google Chrome preglednika

To će rezultirati otvaranjem dodatnih opcija u kojima se može mijenjati sadržaj učitanе stranice, modificirati stilove, debugirati JavaScript kod, pratiti zahtjeve koji su poslani te njihov odgovor, dobiti informacije vezane za sigurnost, pregledati izgled web-stranice na mobilnim uređajima i još mnogo drugih opcija.

Dobrodošli na WRD!



Ilustracija 3 - Opcije Inspect prozora

U ovom poglavlju koje je posvećeno HTML jeziku, posebno će biti interesantni „Elements“ dio i „Computed“ dio na kojem se može vidjeti tzv. Box model, o kojem će biti više informacija u nastavku.

1

HTML

1.1 Uvod u HTML

Html je jezik. Glavna osobina HTML-a je da se radi o tzv. opisnom jeziku ili „markup“ jeziku. HTML opisuje sadržaj i strukturu web-stranice. To podrazumijeva da se kod napisan u HTML jeziku neće prevesti niti izvršiti na procesoru nekog uređaja.

Na početku svakog HTML dokumenta se nalazi definicija HTML jezika, odnosno njegove verzije. Posljednja verzija HTML jezika koji se koristi u ovom materijalu je HTML5. Prethodne verzije HTML dokumenta su, u ovom tagu, trebale sadržavati informacije o verziji HTML jezika. Za standardiziranje HTML jezika zadužen je W3 ili WWW Konzorcijum². W3 Konzorcijum je međunarodna organizacija, osnovana 1994. godine, čiji je zadatak standardiziranje World Wide Web-a. Posljednja preporuka W3 Konzorcijuma je korištenje HTML verzije 5. U prethodnim verzijama HTML jezika, na početku HTML datoteke (engl. file), bilo je potrebno navesti o kojoj verziji HTML

² <https://www.w3.org/>

jezika se radi. Ukoliko se radilo o nekoj starijoj verziji HTML 4, na početku HTML dokumenta bilo je potrebno napisati npr. sljedeći tag:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
```

Međutim, za verziju HTML 5, dovoljno je na početku HTML dokumenta navesti samo sljedeći tag bez posebnog navođenja verzije jezika kako je prikazano na sljedećoj liniji:

```
<!DOCTYPE html>
```

Svi preglednici će prilikom učitavanja HTML dokumenta, već na ovoj prvoj liniji, prepoznati da se radi o HTML dokumentu koji je pisan u HTML jeziku verzije 5.

HTML kod će se interpretirati na pregledniku, odnosno internet pregledniku računara ili nekog drugog uređaja, te će se njegov sadržaj protumačiti na osnovu čega će preglednik prikazati neki sadržaj.

Npr. ako napišemo sljedeći kod kao dio naše HTML stranice:

```
<h1>Dobrodošli na WRD</h1>
```

to predstavlja jednu liniju HTML koda. Ovu liniju će Browser protumačiti na sljedeći način:

- Korisnik želi da se prikaže tekst uvećanim slovima tipa „H1“. Osnovna funkcionalnost svakog Web preglednika/Browser-a je tumačenje HTML koda. Dakle, Browser već „zna“ koji je to font i koja veličina.
- Korisnik želi da se tim fontom napiše tekst: „Dobrodošli na WRD“.

Rezultat tog intepretiranja HTML koda od strane Web preglednika će biti sljedeći:

Dobrodošli na WRD!

Ilustracija 4 - Interpretacija koda na Web pregledniku

Dakle, Preglednik je interpretirao/protumačio HTML kod i prikazao sadržaj. Rezultat interpretacije sadržaja je prikazan na slici. Veličina fonta, oblik slova podebljanja i sl. moraju biti poznati pregledniku. Dakle, preglednik ima ugrađene informacije kako će protumačiti <h1> tag.

Detalji specifikacije HTML jezika verzije 5.2 nalaze se na linku:

<https://www.w3.org/TR/html52/index.html>

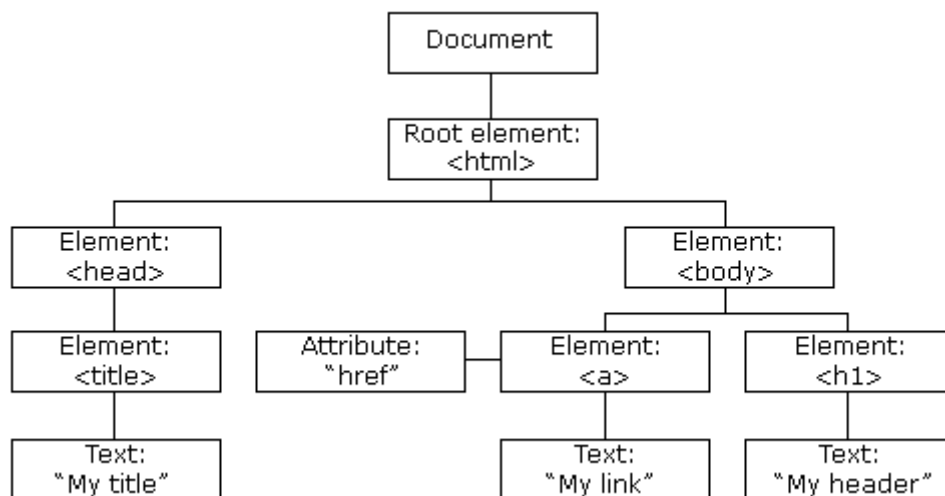
Lista HTML elemenata, atributa i događaja grupisanih po kategorijama, specifikacijama i referencama nalazi se na linku:

<https://www.w3.org/TR/html52/fullindex.html>

1.2 Struktura HTML dokumenta – DOM

HTML dokument je strukturiran po modelu koji se naziva DOM – *Document Object Model*. Struktura HTML dokumenta se sastoji od tag-ova koji počinju i završavaju znacima < i >. Svaki HTML tag je zaokružen navedenim znacima. Internet preglednik prepoznaje tag na osnovu navedenih znakova.

Kompletan dokument je strukturiran u formi drveta što odgovara DOM modelu.



Ilustracija 5 - DOM - Document Object Model³

³ https://www.w3schools.com/whatis/whatis_html5dom.asp

HTML DOM dakle predstavlja model i strukturu HTML dokumenta. U korijenu HTML dokumenta nalazi se *root* element koji zapravo predstavlja kompletnu HTML stranicu. Prva dva „child“ elementa su `<head>` i `<body>`.

Sadržaj `<head>` tag-a su obično `<title>` odnosno tag s naslovom stranice, kao i neki meta tagovi čija je uloga da definiraju osobine dokumenta kao što je character set, meta podatke za *search engine*, te ostale informacije. Head tag može sadržavati i druge tagove kao što je `<style>` ili `<script>` što predstavlja tagove u kojima se pišu CSS stilovi i JavaScript kod.

Sadržaj `<body>` taga zapravo predstavlja sadržaj stranice koja će se prikazati na pregledniku. To predstavlja sve vidljive elemente koji će biti prikazani na pregledniku, te tagovi, odnosno HTML elementi, koji će služiti za formatiranje stranice, raspored elemenata i sl. Osim navedenog i u `<body>` tagu mogu biti elementi stila, i JavaScript dijelovi koda, međutim to se ne upotrebljava često u odnosu na `<head>` tag. Svi elementi na HTML stranici pa samim tim i u `<body>` tagu moraju biti napisani tako da se ispoštuje struktura DOM. Jedan element mora u cijelosti biti unutar drugog elementa ili njegov susjed. Struktura DOM elementa razlikuje dva odnosa među elementima i to *parent-child* odnos i *siblings* odnos. U smislu roditelj-dijete (engl. parent-child) odnosa, jedan HTML element je u potpunosti sadržan ili ugnježđen unutar drugog HTML elementa. U slučaju braća-sestre (engl. sibling) elemenata, odnos između dva elementa je takav da imaju isti parent element, a ne nalaze se jedan u drugom. Osnovna struktura HTML dokumenta se, dakle, sastoji od definicije jezika, HTML taga, čiji su child elementi `<head>` i `<body>` tagovi. Head i body tagovi su u međusobnom odnosu *siblings*-a. U sljedećim linijama koda, dat je prikaz osnovnog HTML dokumenta.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
```

```
</html>
```

Osim rasporeda i odnosa elemenata u strukturi DOM-a, DOM definira sve HTML elemente kao objekte. U okviru DOM-a, HTML elementi, odnosno objekti imaju definirane i svoje događaje (engl. events), metode, odnosno funkcije (engl. methods) i obilježja (engl. properties). Events, methods i properties će biti intenzivno korišteni u CSS-u i JavaScript-u.

1.3 Tagovi

Kao što je navedeno, kompletan HTML dokument se sastoji od određenog broja tagova. Tag predstavlja opisni izraz koji pregledniku služi kako bi identificirao početak i kraj individualnog HTML elementa. Kod većine HTML elemenata postoji tzv. „otvarajući“ i „zatvarajući“ tag, što predstavlja početak, odnosno kraj elementa. Tako npr. ukoliko želimo prikazati jedan HTML element tipa „h1“, odnosno heading stil teksta, tu liniju HTML koda ćemo početi otvarajućim tagom <h1>, nakon čega ćemo napisati onaj tekst koji želimo korisniku prikazati u formatu heading 1, te ćemo na kraju te linije HTML koda napisati zatvarajući tag </h1>. Time ćemo pregledniku omogućiti ispravno interpretiranje dijela teksta koji želimo prikazati u određenom formatu.

Uraditi! Kreirati osnovni HTML dokument s jednim h1 elementom u kojem ćete napisati tekst: „Dobrodošli na WRD“. Te isprobati rješenje u pregledniku Google Chrome.

Rješenje:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
</head>
<body>
  <h1>Dobrodošli na WRD</h1>
</body>
</html>
```

Za lakšu izradu ovih zadataka, koristiti Visual Studio Code editor.

Većina HTML elemenata imaju otvarajući i zatvarajući tag. Međutim, određeni HTML elementi nemaju zatvarajući tag te takve elemente nazivamo samozatvarajućim elementima. Primjer takvog taga su `<br/>`, `<hr/>`, `<img>` itd. Navedeni `<hr/>` tag, poznatiji kao horizontal rule ili horizontal line, služi za odvajanje HTML sadržaja. Tako će sljedeći HTML kod:

```
<body>
  <h1>Dobrodošli na WRD</h1>
  <hr/>
  <h1>Ovo je primjer odvojenog sadržaja</h1>
</body>
```

rezultirati sljedećom interpretacijom na pregledniku:

Dobrodošli na WRD

Ovo je primjer odvojenog sadržaja

Ilustracija 6 - Primjer <hr> taga

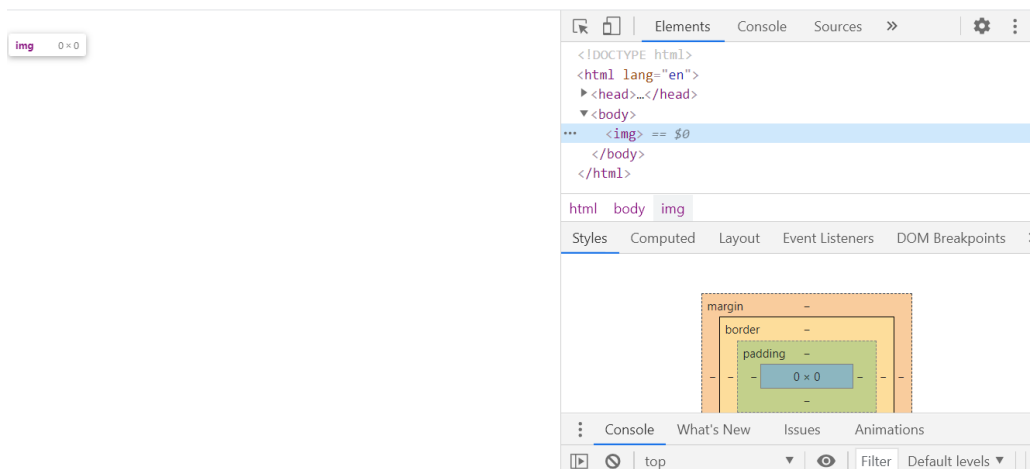
Na primjeru se može primjetiti da H1 element ima otvarajući i zatvarajući tag, dok HR element ima jedan samozatvarajući tag.

1.3.1 Atributi

Osim osnovnih tagova, bitno je napomenuti da tagovi imaju i svoje atribute. Atribut taga predstavlja dodatnu informaciju o tagu. Određen broj tagova ne bi imao smisla bez pripadajući atributa dok neki tagovi mogu biti korišteni i bez ikakvih pripadajućih atributa. Npr. `<h1>` tag ima smisla i bez dodatnih atributa jer će sam tag biti ispravno interpretiran na pregledniku i imat će svoj smisao i značenje, kao što je prikazano na prethodnom primjeru. Međutim, tagovi kao što su `<img>` (tag za element slika) ili `<a>` (tag za element link) nemaju smisla ukoliko im se ne dodaju dodatne informacije u atributima.

Npr. `<img>` tag mora imati minimalno atribut „src“ da bi bilo kakav sadržaj mogao biti prikazan.

Na sljedećoj slici prikazan je `img` tag bez atributa.



Ilustracija 7 - Primjer `<img>` taga bez dodanih atributa

Kao što se vidi na prethodnoj slici, `<img>` tag je ispravno učitao i nalazi se na stranici, ali korisniku stranice nema nikakvu vrijednost jer ništa nije prikazano.

Da bi slika bila prikazana, `<img>` tag mora imati minimalno jedan dodatni atribut „src“ u kojem se navodi link do slike koja treba biti prikazana na stranici. Naravno, „src“ atribut mora sadržavati ispravan link do datoteke sa slikom. U folder u kojem se nalazi html dokument potrebno je kopirati datoteku sa slikom koji u ovom primjeru ima naziv „picture1.jpg“. Zatim je potrebno dodati „src“ atribut u `img` tag kao na sljedećem primjeru, kako bi preglednik mogao ispravno protumačiti taj html element i prikazati ga u prozoru preglednika.

```
<body>
  
</body>
```

U ovom slučaju preglednik će ispravno prikazati željenu sliku.



Ilustracija 8 - Prikaz slike u tagu

Osim atributa „src“, tag može sadržavati i dodatne attribute koji opisuju način prikazivanja linkovane slike. Ti atributi su „alt“ odnosno atribut koji sadrži alternativni tekst koji će se prikazati ukoliko sliku nije moguće učitati, te „width“ i „height“ čije vrijednosti će preglednik koristiti kako bi prikazao sliku u željenim dimenzijama širine i visine.

```
<body>  
    
</body>
```

U ovom slučaju preglednik će prikazati navedenu sliku u dimenzijama 300px x 200px. U slučaju definiranja tačnih dimenzija, potrebno je voditi računa da se ne desi deformacija slike, odnosno da se omjer širine i visine slike prilagodi originalnom omjeru u kojem je slika formatirana. Ukoliko se desi razlika između navedenih omjera, slika može izgledati suženo ili rašireno u odnosu na originalnu sliku. Naravno, ovo su samo neki od mogućih atributa taga.

<a> tag, odnosno „anchor“ je još jedan primjer taga koji nema vrijednost na stranici bez dodatnih atributa. <a> tag je jedan od napoznatijih tagova i predstavlja tag u kojem se nalazi link/poveznica. Linkovi/poveznice mogu biti za eksterne html ili druge dokumente, za Web lokacije, za slanje e-mail poruke kroz defaultni e-mail program na računaru, ali i na sekcije unutar istog html dokumenta.

Primjeri linkova/poveznica s objašnjenjima dati su u sljedećem kodu.

```

<body>
  <a href="http://www.fit.ba">Klikni ovdje da otvoriš web
stranicu</a>
  <br/>
  <a href="mailto:testmailadresa@test.com">Klikni ovdje da
pošalješ e-mail</a>
  <br/>
  <a href="#sekcija2">Klikni ovdje da navigiraš na sljedeću
sekciju unutar ovog html dokumenta</a>
  <br/>
  <a href="dokument3.html">Klikni ovdje da otvoriš drugi html
dokument na ovom site-u</a>
  <br/>
  <h2>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
do eiusmod tempor incididunt ut labore et dolore magna aliqua...
  </h2>
  <h3 id="sekcija2">
    <h1>Sekcija 2</h1>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit,
sed do eiusmod tempor incididunt ut labore et dolore magna
aliqua...</p>
  </h3>
</body>

```

Osim navedenih linkova, moguće je postaviti i link s brojem telefona, kao što je prikazano u sljedećoj liniji koda:

```

<a href="tel:+38761123456">+387-61-123-456</a>

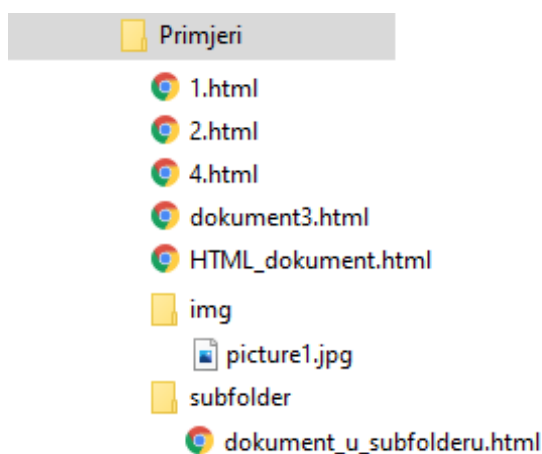
```

što će omogućiti funkcije telefonskog poziva na uređajima na kojima je to moguće izvršiti.

1.3.2 Apsolutne i relativne putanje

Posebnu pažnju potrebno je obratiti na apsolutne i relativne linkove/poveznice u atributima html tagova kao što su „src“ ili „href“ atributi. Pojam apsolutni link podrazumijeva punu putanju do neke lokacije kao što je npr. lokacija i ime datoteke sa slikom u slučaju „src“ atributa. Ukoliko se na računaru, na kojem se razvija web-stranica, dokument nalazi na nekoj lokaciji kao na npr. D:\ disku, potrebno je voditi računa da se ta slika neće nalaziti na istoj putanji ukoliko uradimo deploy web

stranice na neki server. Stoga je potrebno voditi računa da se kompletan web site koji se kreira nalazi u jednom folderu koji može biti postavljen na web server, te da sve putanje do lokacija unutar tog foldera budu relativne, odnosno da se navode putanje u odnosu na trenutno otvoreni html dokument. U prethodnim primjerima putanja do html dokumenta `<a href="dokument3.html">` kako je navedeno u primjeru `<a>` taga ili putanji do slike `` kako je navedeno u primjeru `<img>` taga, radi se o slici, odnosno html dokumentu koji se nalazi u istom folderu u kojem je i otvoreni html dokument u kojem se nalazi navedeni link, odnosno referenca. Best practice, odnosno nabolja praksa je organizirati folder tako da se sve slike smjeste u subfolder koji je imenovan kao „img“, svi CSS dokumenti u subfolder koji je imenovan kao „css“, JavaScript file-ovi u subfolder koji je imenovan kao „js“ itd. U tom slučaju, relativne putanje će sadržavati putanju od otvorenog dokumenta do ciljnog dokumenta putujući strukturom foldera korištenjem znakova „/“ i „..“. Znak „/“ označava trenutni folder te omogućava definiranje putanje do subfoldera te može označavati i root folder trenutnog site-a, dok se dvije tačke „..“ koriste za navigiranje jedan nivo foldera iznad trenutnog foldera, te se odatle navodi putanja do destinacijskog dokumenta. Ukoliko pretpostavimo primjer strukture foldera kao na sljedećoj slici.



Ilustracija 9 - Primjer strukture direktorija Web site-a

Relativni link u html u dokumentu nazvanom dokument3.html do slike koja se nalazi u subfolderu nazvanom „img“ bio bi kao u sljedećem kodu:

```
<body>
```

```

</body>
```

Dakle, potrebno je navesti naziv subfoldera koji se nalazi na istoj lokaciji kao i navedeni dokument, te koristiti znak „/“ do naziva dokumenta navodeći sve ostale eventualne subfoldere do željene slike.

Ukoliko se link nalazi u html dokumentu koji je smješten u subfolder, potrebno je prvo navigirati unazad koristeći „..“ kako bi se došlo do prvog zajedničkog čvorišta od kojeg se putanja ponovo navodi na isti način kao i u prethodnom slučaju. Tako bi, za dokument na putanju „subfolder/dokument_u_subfolderu.html“ putanja do iste slike bila kao u sljedećem kodu.

```
<body>
  
</body>
```

Važno je napomenuti da ne postoje neka ograničenja u navigiranju, tako da se korištenje „..“ i „/“ za navigiranje uz strukturu direktorija i korištenje „/“ za navigiranje niz navedenu strukturu može koristiti onoliko puta koliko je potrebno da se ispravno mapira putanja do destinacijskog dokumenta.

Znak „/“ na početku putanje označava root folder ili korijenski direktorij web site-a.

U slučaju linkova na web-stranice van site-a, podrazumijeva se korištenje apsolutnih web putanja kao npr.

1.4 Podjela tagova

HTML sadrži određen broj tagova koji su ugrađeni u sam jezik. U različitoj literaturi tagovi su grupisani na različite načine. Važno je napomenuti da je lista tagova promjenjiva i zavisi od verzije HTML jezika. U HTML5 neki tagovi nisu podržani, jer su zamijenjeni novijim verzijama tagova koji omogućavaju istu ili sličnu funkcionalnost, ili su u potpunosti uklonjeni jer se te funkcionalnosti u praksi više ne koriste.

Osnovna podjela tagova je na:

- osnovni HTML tagovi

- tagovi za formatiranje
- tagovi za forme i input elemente
- frame odnosno okviri u kojima je većina tagova zastarjela i ne koristi se u HTML5
- tagovi za slike
- tagovi za audio i video sadržaje
- tagovi za liste
- tagovi za tabele
- tagovi za stilove i semantiku
- tagovi za linkove
- tagovi za programske kodove i skripte
- meta tagovi koji se ne prikazuju korisniku, ali imaju svoju vrijednost i značenje na samoj stranici.

1.4.1 Osnovni HTML tagovi

U osnovne HTML tagove ubrajaju se tagovi za osnovnu strukturu stranice kao što je `<!DOCTYPE>` tag u kojem se definira verzija HTML jezika, zatim `<html></html>` koji definira početak i kraj html dokumenta te `<head></head>` i `<body></body>` tag, koji definiraju prve dvije sekcije stranice. Navedeni tagovi se također ne primjećuju u prozoru preglednika. Ostali tagovi koji se ubrajaju u osnovne HTML tagove su `<title></title>` tag koji se nalazi u `<head></head>` tagu i definira naslov stranice, odnosno tekst koji će biti prikazan na tab-u preglednika, zatim `<h1></h1>` - `<h6></h6>` tagovi pomoću kojih se prikazuju naslovi različitih veličina. U osnovne tagove ubraja se i `<p></p>` tag odnosno paragraf teksta, zatim `<br/>` i `<hr/>` tagovi koji su već ranije spomenuti.

Primjer navedenih tagova dat je u sljedećem kodu:

```
<body>
  <h1> 1. nivo naslova - H1 </h1>
  <h2> 2. nivo naslova - H2 </h2>
  <h3> 3. nivo naslova - H3 </h3>
  <h4> 4. nivo naslova - H4</h4>
  <h5> 5. nivo naslova - H5 </h5>
  <h6> 6. nivo naslova - H6 </h6>
  <p> Paragraf tekst </p>
</body>
```

rezultat interpretacije navedenog koda dat je na sljedećoj slici.

1. nivo naslova - H1

2. nivo naslova - H2

3. nivo naslova - H3

4. nivo naslova - H4

5. nivo naslova - H5

6. nivo naslova - H6

Paragraf tekst

Ilustracija 10 - Izgled <h1> - <h6> i <p> tagova

Osim navedenih, HTML sadrži i tag koji označava komentar. Slično kao i komentiranje linije koda u programskim jezicima, što rezultira isključenjem određenog broja linija koda i procesa kompajliranja, u slučaju HTML opisnog jezika, tag za komentar može služiti da se određen broj linija HTML koda isključi, te da ih preglednik ignoriše prilikom učitavanja stranice, ili se mogu koristiti kao i ostali komentari za dodatna pojašnjenja koda.

Sintaksa za html tag komentara je `<!-- -->` te se koristi na sljedeći način: `<!-- neki sadržaj koji isključujemo -->`

1.4.2 Tagovi za formatiranje

Tagovi za formatiranje služe za prikazivanje sadržaja u specifičnim formatima, kao što su `<strong>`, `<small>`, `<i>` itd. Postoji veći broj tagova za formatiranje teksta. Neki od njih su objašnjeni u nastavku.

Tag	Opis
<code><i></code>	označava tekst koji će biti prikazan <i>italic</i> slovima
<code><sub></code>	označava tekst koji će biti prikazan u subscript formatu
<code><sup></code>	označava tekst koji će biti prikazan u superscript formatu
<code></code>	označava tekst koji će biti prikazan bold slovima
<code></code>	označava tekst koji će biti prikazan prekrižen
<code><mark></code>	označava tekst koji će biti prikazan markiranim slovima

U navedenim tagovima, kao i u tagovima koji su navedeni kao osnovni HTML tagovi, uzastopni razmaci u tekstu ili prelasci u novi red će biti ignorisani. Ukoliko želimo prikazati dodatne razmake i prelasci u novi red teksta, koristi se <pre> tag koji će prikazati preformatirani tekst, odnosno tekst onakav kakav je napisan u kodu.

<pre> tag označava tekst koji će biti prikazan sa svim dodatnim razmacima i prelascima u novi red.

Primjer koda s navedenim tagovima dat je u sljedećem prikazu.

```
<body>
  <p>Napon V<sub>cc</sub> jednak je U<sup>2</sup>*2</p>
  <p>Lorem ipsum dolor sit amet, <b>consectetur adipiscing elit,</b> sed do eiusmod tempor...</p>
  <p>Lorem ipsum dolor sit amet, <i>consectetur adipiscing elit,</i> sed do eiusmod tempor...</p>
  <p>Lorem ipsum dolor sit amet, <cite>consectetur adipiscing elit,</cite> sed do eiusmod tempor...</p>
  <p>Lorem ipsum dolor sit amet, <del>consectetur adipiscing elit,</del> sed do eiusmod tempor...</p>
  <p>Lorem ipsum dolor sit amet, <mark>consectetur adipiscing elit,</mark> sed do eiusmod tempor...</p>
  <p>Može se vidjeti da dodatni "space" razmaci
    ne utiču na prikaz teksta osim
    u slučaju "pre" taga.
    <pre>"pre" tag
    omogućava da se tekst sa dodatnim razmacima
    i prelaskom u novi red
    prikaže <strong>preformatirano (pre)</strong> odnosno tačno
    onako kako je i zapisan u HTML kodu.
  </pre></p>
</body>
```

Navedeni kod će biti interpretiran na sljedeći način.

Napon V_{cc} jednak je $U^2 \cdot 2$

Lorem ipsum dolor sit amet, **consectetur adipiscing elit**, sed do eiusmod tempor...

Lorem ipsum dolor sit amet, *consectetur adipiscing elit*, sed do eiusmod tempor...

Lorem ipsum dolor sit amet, *consectetur adipiscing elit*, sed do eiusmod tempor...

Lorem ipsum dolor sit amet, ~~consectetur adipiscing elit~~, sed do eiusmod tempor...

Lorem ipsum dolor sit amet, **consectetur adipiscing elit**, sed do eiusmod tempor...

Može se vidjeti da dodatni "space" razmaci ne utiču na prikaz teksta osim u slučaju "pre" taga.

```
"pre" tag
omogućava da se tekst sa dodatnim razmacima
i prelaskom u novi red
prikaže preformatirano (pre) odnosno tačno
onako kako je i zapisan u HTML code-u.
```

Ilustracija 11 - Izgled nekih osnovnih tagova za formatiranje

1.4.3 Tagovi za forme i *input* elemente

Forme predstavljaju specifičnu funkcionalnost HTML jezika. One su komponente HTML stranice koje služe za prikupljanje i slanje podataka k serveru ili nekoj drugoj aktivnoj komponenti, te za prikaz podataka u slučajevima web baziranih aplikacija. Za prikaz podataka na formi koriste se kontrole, koje predstavljaju polja za unos ili prikaz podataka.

Najčešće korištene kontrole su:

- Polje za unos teksta `<input type="text">` -
- Polje za unos dužeg teksta `<textarea name="komentar" maxlength=1000`

`cols="80" rows="6">` -

- Polje za unos e-mail adrese `<input type="email">` -

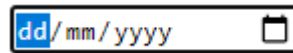
- Polje za unos šifre `<input type="password">` -

- Polja za unos brojne vrijednosti `<input type="number">` -

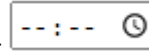
- Polje za unos vrijednosti u rangu <input type="range"> -



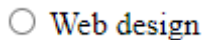
- Polje za unos datuma <input type="date"> -



- Polje za unos vremena <input type="time"> -



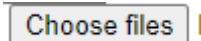
- Polje za izbor jedne opcije uz više ponuđenih opcija <input type="radio"> -



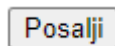
- Polje za izbor višestrukih mogućih opcija <input type="checkbox"> -



- Polje za upload file-ova <input type="file" name="dokumenti" multiple> -



- Dugme za submit, reset i ostale akcije <input type="submit" value="Posalji"> -



- itd.

Na sljedećem primjeru dat je prikaz tipičnog koda za formu za prikupljanje podataka i slanje na server.

```
<body>
  <form method="post" enctype="application/x-www-form-urlencoded"
action="https://neka_akcija_na_server_side.aspx">
    <p><label>Ime: <input name="ime" required
autocomplete="ime"></label></p>
    <p><label>Telefon: <input type=tel name="telefom"
autocomplete="tel"></label></p>
    <p><label>Poštanski broj: <input name="posta"
inputmode="numeric"></label></p>
    <p><label>Email addressa: <input type=email name="email"
autocomplete="email"></label></p>
    <fieldset>
      <legend> Omiljena oblast </legend>
      <p><label> <input type=radio name=oblast required
value="robotika"> Robotika </label></p>
      <p><label> <input type=radio name=oblast required
value="programiranje"> Programiranje </label></p>
      <p><label> <input type=radio name=oblast required
value="webdesign"> Web design </label></p>
    </fieldset>
```

```

        <fieldset>
            <legend> Prijavljujem predmete: </legend>
            <p><label> <input type=checkbox name="predmet"
value="fit-it-15"> PR I </label></p>
            <p><label> <input type=checkbox name="predmet"
value="fit-it-18"> PR II </label></p>
            <p><label> <input type=checkbox name="predmet"
value="fit-it-32"> IoT </label></p>
            <p><label> <input type=checkbox name="predmet"
value="fit-it-47"> Embedded i RT sistemi </label></p>
        </fieldset>
        <p><label>Željeni datum prisustva nastavi: <input type=date
name="nastavadan" required></label></p>
        <p><label>Željeno vrijeme prisustva nastavi: <input
type=time min="09:00" max="18:00" name="nastavasat"
required></label></p>
        <p><label>Instrukcije za predavača - posebne želje:
<textarea name="komentar" maxlength=1000></textarea></label></p>
        <p><button>Submit</button></p>
    </form>
</body>

```

Forma počinje i završava <form></form> tagom. U otvarajućem form tagu, potrebno je navesti attribute za akciju koja će se izvršiti po submitiranju forme, te opcionalno metod submitiranja forme, tip enkodiranja podataka, target tab koji će se otvoriti po submitiranju forme i eventualno još neke dodatne informacije. Podaci s kontrola forme će biti zapakovani u request Form Data i poslati na server na destinaciju navedenu u atributu „action“.

Ukoliko pretpostavimo da je korisnik web aplikacije unio podatke kao na slici:

Ime:

Telefon:

Poštanski broj:

Email address:

Omiljena oblast

☒ Robotika

☐ Programiranje

☐ Web design

Prijavljujem predmete:

☐ PR I

☐ PR II

☒ IoT

☒ Embedded i RT sistemi

Željeni datum prisustva nastavi: 

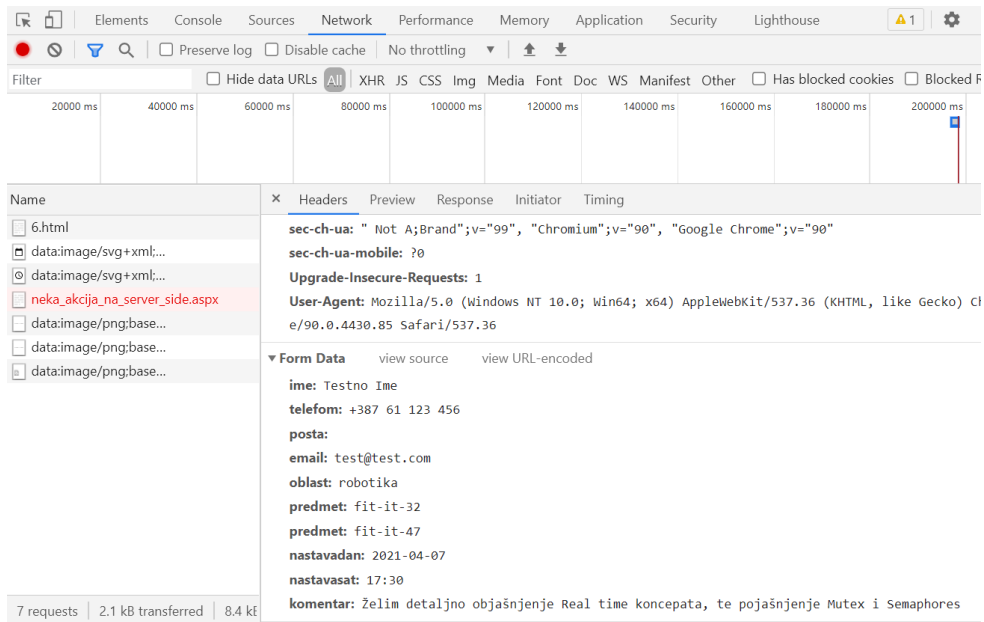
Željeno vrijeme prisustva nastavi: 

Instrukcije za predavača - posebne želje:

Želim detaljno objašnjenje Real time konceptata, te pojašnjenje Mutex i Semaphores

Ilustracija 12 - Primjer HTML5 forme

poslane podatke je moguće vidjeti u Inspect sekciji preglednika pod tabom Network.

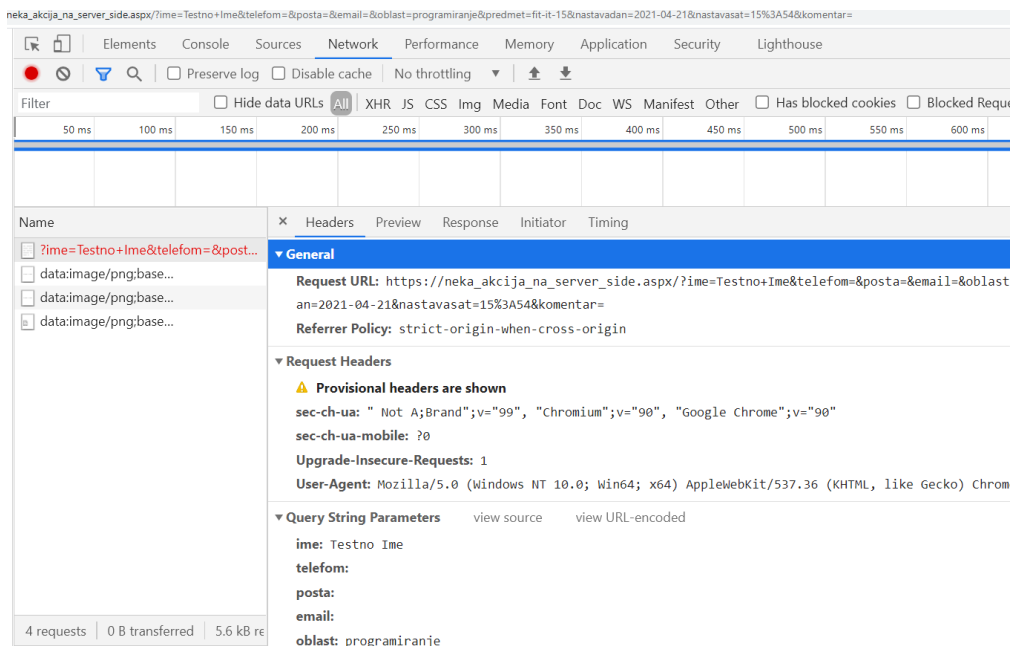


Ilustracija 13 - Pregled podataka poslanih sa HTML5 forme na Web server POST metodom

Na network tabu se može primijetiti da su podaci poslani serveru POST zahtjevom i da su zapakovani u Form Data. Ukoliko se promijeni metod forme u GET

```
<form method="get"
action="https://neka_akcija_na_server_side.aspx"></form>
```

prilikom submitiranja forme može se primijetiti da je slanje podataka također promijenjeno.



Ilustracija 14 - Pregled podataka poslanih sa HTML5 forme na Web server GET metodom

U ovoj fazi potrebno je obratiti pažnju na dvije osnovne razlike. Svi parametri su proslijeđeni serveru u formi tzv. URL parametara i vidljivi su korisniku. Također, u network tabu može se primijetiti da je zahtjev poslan u obliku tzv. *query string* parametara. Prednost ove vrste slanja je u činjenici da korisnik može označiti (engl. bookmark) ovakav zahtjev, te da u samom URL djelu preglednika može ručno promijeniti vrijednosti pojedinih parametara. Nedostatak ove metode slanja podataka na server je u tome što podaci nisu zaštićeni, posebno ako se radi o formi u koju unosimo povjerljive podatke kao npr. matični broj, broj kreditne kartice i sl. Također, za razliku od slanja POST metodom, GET metoda ima ograničenu veličinu URL parametara pa je time programer limitiran koliko podataka može poslati jednim zahtjevom.

Osim navedenog, moguće je koristiti i ostale atribute forme. Jedan od često korištenih atributa je i „target“ kojim se definira ponašanje taba na pregledniku poslije submitiranja forme. Na sljedećoj liniji date su opcije target atributa. Potrebno je izabrati samo jedan od ponuđenih.

target="_blank _self _parent _top frameName"
--

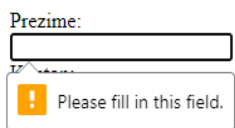
Npr. ukoliko se navede vrijednost target atributa „_blank“, submitirana forma će biti otvorena u novom tabu, dok će „_self“ značiti da će submitirana forma biti otvorena u trenutnom tabu preglednika/browsera.

*Napomena: Target atribut je primjenjiv i na **link element** i ima isti efekat.*

1.4.3.1 HTML 5 Validacija forme

U kodu forme navedene na početku ovog poglavlja, mogu se primijetiti i neki dodatni atributi kao što je atribut bez navedene vrijednosti „required“ koji označava da je to polje obavezno za unos. Osim toga mogući su atributi „pattern“ koji označava u kojem formatu mora biti unesena vrijednost u kontrolu. Vrijednost pattern="[A-Za-z]{5}" predstavlja format unosa 5 velikih ili malih slova. Zatim npr. atributi min="100" i max="1000" služe za limitiranje mogućeg unosa numeričke vrijednosti ili rang vrijednosti u odgovarajuće kontrole.

Ukoliko korisnik web aplikacije ne ispoštuje tražene opcije unosa, HTML formu neće biti moguće submitirati, te će korisniku biti prikazana odgovarajuća poruka kao na sljedećoj slici.



Ilustracija 15 - Izgled poruke upozorenja pri HTML5 validaciji

Ovo se smatra prvim stepenom zaštite aplikacije od neželjenog unosa. Međutim, kao što je opisano ranije, preglednik omogućava korisniku modificiranje svih elemenata na HTML stranici, pa samim tim i kontrola na formi, tako da je vrlo lako ukloniti attribute kontrola koji predstavljaju kriterij validacije. Zbog navedenog, ali i drugih razloga koji neće biti analizirani u ovom materijalu, HTML5 validacija nije dovoljna za ozbiljnu web aplikaciju. Sljedeći stepen validacije je JavaScript ili neka druga front-end validacija. Ipak, najbolji stepen zaštite od neželjenog unosa je server-side validacija koja se obavlja na back-end aplikacije.

1.4.4 Tagovi za liste

Liste su nezaobilazan dio web-stranica. Koriste se za potrebe naslova, menija pa sve do galerija fotografija. Postoje tri osnovne vrste lista:

- uređena lista
- neuređena lista

- definicijska lista.

1.4.4.1 Uređena lista

Uređena lista je tip liste koja ima numeraciju. Osnovni tag za uređenu listu je `<ol>` (Organised List). U sljedećem kodu prikazan je primjer uređene liste.

```
<ol>
  <li>PR I</li>
  <li>PR II</li>
  <li>WRD</li>
  <li>Engleski</li>
  <li>Matematika</li>
</ol>
```

Iz navedenog primjera možemo vidjeti da je sadržaj uređene liste tag `<li>` (List Item), što će biti slučaj i s ostalim listama. Navedeni kod će rezultirati sljedećim prikazom u pregledniku.

1. PR I
2. PR II
3. WRD
4. Engleski
5. Matematika

Ilustracija 16 - Izgled uređene liste

Kao i u slučaju ostalih HTML elemenata, uređena lista može imati svoje atribute. Atributom „start“ se definiše početna vrijednost numeracije, atributom „type“ definira se tip markera u listi. Moguće vrijednosti su 1-numeracija arapskim brojevima, „I“ i „i“ – numeracija rimskim brojevima (uključujući i mogućnost prikaza rimskih brojeva malim slovima), te „A“ i „a“ odnosno slovne oznake markera. Atribut „reversed“ omogućava prikaz liste u obrnutom redoslijedu numeracije. Sljedeći kod

```
<ol start="20" type="I" reversed>
  <li>PR I</li>
  <li>PR II</li>
  <li>WRD</li>
  <li>Engleski</li>
  <li>Matematika</li>
</ol>
```

rezultat će sljedećim prikazom u pregledniku.

XX. PR I
XIX. PR II
XVIII. WRD
XVII. Engleski
XVI. Matematika

Ilustracija 17 - Izgled uređene liste sdodatnim atributima

1.4.4.2 Neuređena lista

Neuređena lista je vrlo slična uređenoj, osim što se ne koristi numeracija za označavanje nego *bullet*. Sljedeći kod

```
<ul>  
  <li>PR I</li>  
  <li>PR II</li>  
  <li>WRD</li>  
  <li>Engleski</li>  
  <li>Matematika</li>  
</ul>
```

će biti interpretiran na pregledniku kao

- PR I
- PR II
- WRD
- Engleski
- Matematika

Ilustracija 18 - Izgled neuređene liste

1.4.4.3 Definijska lista

Kao što i samo ime kaže, definijska lista služi na navođenje pojmova i njihovih opisa. Ključni tagovi definijske liste su <dl> - tag za listu, te <dt> i <dd> tagovi za pojam i opis. Sljedeći kod

```
<dl>  
  <dt>PR I</dt>  
  <dd>Jako interesantan predmet</dd>  
  <dt>PR II</dt>  
  <dd>Još interesantniji predmet</dd>  
</dl>
```

će bit interpretiran na pregledniku kao

```

PR I      Jako interesantan predmet
PR II     Još interesantniji predmet

```

Ilustracija 19 - Izgled definicijske liste

1.4.5 Tagovi za tabele

Tabele su neizostavan dio svake web aplikacije koja prikazuje podatke iz baze podataka. Neke JavaScript biblioteke, koje imaju komponente za prikaz tabelarnih podataka, svoje tabele baziraju na HTML tabelama.

Tagovi specifični za tabele su:

Tag	Opis
<code><table></table></code>	Tag za tabelu
<code><tr></tr></code>	Tag za red tabele
<code><th></th></code>	Tag za naslove kolona tabele – osnovni prikaz je podebljanim slovima centrirano
<code><td></td></code>	Tag za ćeliju tabele – osnovni prikaz je izravnavo lijevo

Sljedeći kod tabele

```

<table border="1">
  <tr>
    <th>Ime</th>
    <th>Prezime</th>
    <th colspan="2">Predmeti</th>
  </tr>
  <tr>
    <td>Denis</td>
    <td>Mostarac</td>
    <td>Programiranje</td>
    <td>Razvoj software-a I</td>
  </tr>
  <tr>
    <td>Rijad</td>
    <td>Marić</td>
    <td>Web Razvoj i Dizajn</td>
    <td>Razvoj software-a II</td>
  </tr>
  <tr>

```

```

        <td>Mateo</td>
        <td>Marić</td>
        <td>Matematika</td>
        <td>IoT</td>
    </tr>
</table>

```

će biti interpretiran na pregledniku kao na slijedećoj ilustraciji.

Ime	Prezime	Predmeti	
Denis	Mostarac	Programiranje	Razvoj software-a I
Rijad	Marić	Web Razvoj i Dizajn	Razvoj software-a II
Mateo	Marić	Matematika	IoT

Ilustracija 20 - Izgled HTML5 tabele

Kao što se vidi u prethodnom primjeru, tabele i tagovi sadržaja tabele mogu imati dodatne attribute. Atribut „border“ <table> taga definira debljinu linija. Atribut „colspan“ <th> i <td> taga definira na koliko kolona će se prostirati dato polje. Većina atributa tabele se odnosi na stilove pa će oni biti opisani u sljedećem poglavlju.

1.4.6 Tagovi za linkove

Kao što je već pomenuto u uvodu, jedan od najpoznatijih tagova je hyperlink ili <a> tag. Link je zapravo omogućio prvim web-stranicama navigaciju kroz sadržaje, tako što su različite web-stranice, putem poveznica/linkova, omogućile kretanje i promjenu sadržaja učitavanjem drugih HTML dokumenata. HTML definicija sadrži tri taga vezana za linkove:

Tab	Opis
<a>	hyperlink
<link>	tag za povezivanje eksternih resursa s HTML stranicom, kao što su CSS stilovi
<nav>	tag koji po sebi nije link, ali služi za navođenje uzastopnih linkova

Način korištenja <a> taga je objašnjen i demonstriran u prethodnim poglavljima. Osim navedenog u poglavlju *Atributi*, <a> tag može se koristiti i u kombinaciji sa slikom, tako da slika predstavlja hyperlink kao što je demonstrirano u sljedećem kodu.

```
<a href="https://fit.ba/">  
    
</a>
```

Ukoliko se pokazivač (engl. pointer) pozicionira na sliku, može se primijetiti da se oblik pokazivača promijenio iz standardne strelice u oblik šake s ispruženim prstom, što je karakteristično za HTML hiperlink.

Tag <link> se najčešće koristi za uvezivanje eksternog CSSdokumenta sa stila, definicijama stilova s HTML dokumentom. Sintaksa za link je <link rel="stylesheet" href="moj_stil.css">

<link> tag se najčešće nalazi u <head> sekciji HTML dokumenta.

2

Osnove CSS-a

CSS (engl. Cascading Style Sheets) je jezik za stiliziranje HTML sadržaja, Također je, iopisni jezik. U nekim izvorima CSS se ne definira kao jezik, nego jednostavno kao opis stilova.

Za HTML jezik se može reći da se njime definira sadržaj koji će preglednik (engl. browser) prikazati na web- stranici, a CSS stilovima se definira kako će taj sadržaj biti prikazan.

2.1 CSS selektori

CSS slektori služe za identificiranje na koji HTML element će stil biti primijenjen.

Sintaksa CSS stilova je sljedeća:

selektor {**obilježje1**: **vrijednost1**; **obilježje2**:**vrijednost2**;}

Selektor može biti:

- HTML element, odnosno bilo koji HTML tag koji preglednik može prikazati
- klasa
- ID

- atribut
- kombinirani selektori.

Također, moguće je imenovati element koji ne postoji u HTML definiciji, a u isto vrijeme da se ne navodi „.“ ili „#“, odnosno da se ne definira stil po klasi ili ID. U tom slučaju, element se može koristiti kao novi HTML element i zatvara se u tag u kojem navodimo ime tog novog elementa. Sljedeći primjer prikazuje takav element.

Dat je sljedeći CSS kod:

```
mojelement{  
    background-color: lightblue;  
    font-size: large;  
}
```

koji je implementiran sljedećim HTML kodom:

```
<mojelement>Neki tekst mog elementa</mojelement>
```

što će rezultirati sljedećim prikazom.

Neki tekst mog elementa

Ilustracija 21 - Korištenje nepostojećih elemenata - Neispravno stiliziranje

Međutim, navedeni način stiliziranja nije preporučan i nikada se ne koristi. Selektor uvijek treba biti jedan od navedenih u prethodnoj listi.

Primjer implementacije stila po imenu elementa dat je u sljedećem kodu.

```
h2 {color:brown;}
```

Navedeni stil će biti implementiran na h1 elemente na stranici.

Da bi se stil implementirao po klasi potrebno je definirati stil klase. Naziv klase počinje tačkom kao u sljedećem primjeru.

```
.klasaElementa{  
    color: darkblue;  
}
```

Nakon toga moguće je primijeniti navedeni stil na svaki element sa spomenutom klasom kao npr.

```
<p class="klasaElementa">Primjer implementacije stila po klasi.</p>
```

Implementacija stila po klasi se koristi u slučajevima potrebe da se određeni stil implementira na više HTML elemenata istog tipa ili više različitih tipova elemenata.

Za implementaciju stila po ID elementa, potrebno je definirati ID selektor koji počinje znakom „#“ kao u sljedećem primjeru.

```
#idElementa{  
    color: darkred;  
}
```

Navedeni stil će biti implementiran na elementu sa spomenutim ID kao npr.

```
<p id="idElementa">Primjer implementacije stila po ID elementa.</p>
```

Savjetuje se da na jednoj stranici ID elementa bude jedinstven.

Implementacija stila po atributu elementa podrazumijeva da će se stil implementirati na sve HTML elemente na stranici koji sadrže specifičan atribut ili specifičnu vrijednost određenog atributa. Tako npr. sljedeći kod

```
a[target="_self"]{  
    color: green;  
}
```

definira stil boje slova svakog <a> elementa koji ima atribut „target“ sa specifičnom vrijednošću „_self“. Navedeni stil će biti implementiran na element koji je napisan na sljedeći način.

```
<a href="https://fit.ba/" target="_self">Primjer implementacije  
stila po atributu elementa. U ovom slučaju radi se o hiperlinku sa  
atributom "target"</p>
```

Moguće je implementirati stil na elemente s određenim atributom bez obzira na vrijednost datog atributa. U tom slučaju u definiciji stila se navodi samo naziv atributa te se ne navodi vrijednost atributa, kao npr.

```
a[target]{  
    color: rgb(116, 5, 160);  
}
```

Kombinirani selektori, pseudoklase i pseudoselektori će biti objašnjeni u narednim poglavljima.

2.2 Način uključivanja CSS stilova u HTML

Nijedna ozbiljna web-stranica ne može biti prikazana u zadovoljavajućem obliku i dizajnu bez upotrebe CSS stilova. CSS stilovi se implementiraju na HTML elemente uz određena pravila uključivanja i prioriteta implementiranja stilova.

CSS stilovi se mogu uključiti u HTML na tri načina:

- eksterni CSS stilovi
- interni CSS stilovi
- inline stil.

Inline stil se implementira u obliku atributa HTML elementa. Npr. ukoliko napišemo sljedeću liniju koda

```
<h1 style="color: blue;">Dobrodošli na WRD!</h1>
```

to će rezultirati sljedećim prikazom na pregledniku.

Dobrodošli na WRD!

Ilustracija 22 - Prikaz implementacije CSS stila

Može se primjetiti da je boja slova promijenjena kroz atribut HTML elementa. Korištenjem atributa „style“, CSS stil se primjenjuje na HTML element i to se naziva *inline* stil.

Osim navedenog „inline“ stila, ukoliko se isti stil primjenjuje na više HTML elemenata, koristi se interni ili eksterni CSS stil.

Interni CSS stil se najčešće navodi u <head> sekciji HTML dokumenta i smješta se u <style></style> tag.

Eksterni CSS dokument se veže sa HTML dokumentom putem <link> taga.

U sljedećem kodu dati su primjeri tri spomenuta načina uključivanja CSS u HTML.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Primjeri načina uključivanja CSS stilova</title>
  <style>
    h2{
      color:brown;
    }
  </style>
  <link rel="stylesheet" href="css/moj_stil.css">
</head>
<body>
  <h1 style="color: blue;">Dobrodošli na WRD!</h1>
  <h2>Ovo su primjeri internog... </h2>
  <h3>... i eksternog CSS stila </h3>
</body>
</html>
```

Te ako je sadržaj dokumenta moj_stil.css sljedeći

```
h3{
  color: blueviolet;
}
```

Navedeni kod će rezultirati sljedećim prikazom na pregledniku.

Dobrodošli na WRD!

Ovo su primjeri internog...

... i eksternog CSS stila

Ilustracija 23 - Prikaz implementacije različitih metoda uključivanja CSS stilova

Interni i eksterni CSS stil mijenjaju izgled elemenata u cijelom HTML dokumentu, dok se inline stil primijenjuje samo na HTML element u čijem je atributu naveden, te ne utiče na prikaz ostalih elemenata iste vrste.

Ukoliko se internim ili eksternim CSS stilom mijenja ista osobina istog HTML elementa, za prikaz na stranici se koristi **poslijednji učitani stil**. Nako npr. ukoliko se u internom i u eksternom CSS stilu navede boja teksta h2 elementa kao npr.:

U HTML dokumentu:

```
<style>
  h2{
    color:brown;
  }
</style>
<link rel="stylesheet" href="css/moj_stil.css">
```

U eksternom CSS dokumentu:

```
h2{
  color: blueviolet;
}
```

tekst <h2> elementa će biti prikazan u „blueviolet“ boji isključivo zato jer je eksterni stil učitao nakon definicije internog stila.

Somenuti redoslijed se uklapa u pojam prioriteta stilova. Prioriteti stilova se implementiraju na sljedeći način:

- 1 – Ključna riječ „!important“
- 2 – Inline stil
- 3 – Media query
- 4 – Eksterni CSS ili interni CSS stil napisan u <style> tagu
- 5 – Specifičnost selektora
- 6 – Zadnji učitani stil
- 7 – Naslijeđeni stil

8 – Bilo koji CSS stil

9 – Prikaz elementa po HTML definiciji bez stila – defaultni prikaz preglednika

2.3 Specifičnost selektora

Specifičnost selektora je pojam koji je uveden zbog lakšeg određivanja i kalkulacije koji stil će biti implementiran u slučaju da postoji više od jednog definiranog stila za isti HTML element. Ukoliko se više CSS stilova primjenjuje na isti element, potreban je mehanizam rezrješavanja konflikta u implementaciji kako bi se omogućilo pregledniku (engl. browseru) da identificira jedan stil koji će biti implementiran.

Specifičnost selektora se može predstaviti kao četverocifreni broj u kojem svaka cifra ima svoju težinsku vrijednost identičnu kao u decimalnom brojnom sistemu. Pozicija i vrijednost svake cifre ovise o broju i tipu selektora koji je implementiran na određenom HTML elementu. Četverocifreni broj se predstavlja ciframa a,b,c,d. Cifra d predstavlja jedinice, cifra c desetice, cifra b stotine i cifra a hiljade. Cifre odgovaraju tipu selektora, odnosno pripadajućeg stila, tako da važi:

a – stil kao atribut (inline stil) - 1000

b – stil po ID atributu - 100

c – ostali atributi - 10

d – po nazivu elementa – 1

Ukoliko implementiramo stil po nekoliko različitih pripadajućih cifara, cifre će se zbrajati uz poštivanje pravila njihove težinske vrijednosti. Tako npr. ukoliko imamo sljedeći definirani stil:

```
div h2 p{
  color: darkblue;
}
```

te odgovarajući HTML kod kojim se implementira navedeni stil:

```
<div>
  <h2>
    <p>Ovo redanje prioriteta "div h2 p" predstavlja
0,0,0,3</p>
  </h2>
```

```
</div>
```

budući da se radi o tri selektora po nazivu HTML elementa, sve tri cifre će imati težinsku vrijednost jedinica u decimalnom sistemu, te će se računati $1+1+1=3$, odnosno navedeni stil će imati specifičnost izraženu brojem 3, te će navedeni kod rezultirati sljedećim prikazom u pregledniku (engl.browser).

Ovo redanje prioriteta "div h2 p" predstavlja 0,0,0,3

Ilustracija 24 - Primjer specifičnosti selektora po imenu elementa

Međutim, ukoliko postoji stil implementiran po ID kao u sljedećem kodu:

```
#identifikator{  
    color: green;  
}
```

koji je implementiran sljedećim HTML kodom:

```
<div>  
    <h2>  
        <p id="identifikator">Ovo je redanje prioriteta "ID" predstavlja  
0,1,0,0 te se on implementira</p>  
    </h2>  
</div>
```

navedeni kod rezultirat će sljedećim prikazom upregledniku:

Ovo je redanje prioriteta "ID" predstavlja 0,1,0,0 pa se on implementira

Ilustracija 25 - Redanje prioriteta

Očito je da je stil po ID imao veći prioritet od stila po tri element selektora jer je $100>3$.

Pojednostavljen primjer implementiranja stilova po specifičnosti selektora za sva četiri nivoa težinske vrijednosti sa po jednim selektorom može se demonstrirati sljedećim kodom. Ukoliko imamo CSS:

```
h1{
```

```

        color: blue;
    }
    .klasaZaH{
        color: red;
    }
    #identifikator{
        color: green;
    }

```

koji je implementiran kroz sljedeći HTML kod:

```

<h1>Ovo je tekst samo H1</h1>
<h1 class="klasaZaH">Ovo je tekst H1 sa klasom. Ocito je da stil po
klasi ima veci prioritet od element selektora.</h1>
<h1 class="klasaZaH" ID="identifikator">Ovo je tekst H1 sa klasom i
ID. Ocito je da stil po ID ima veci prioritet od stila po klasi i po
element selektoru.</h1>
<h1 class="klasaZaH" ID="identifikator" style="color: magenta">A
opet na kraju inline stil "pobjedi"...</h1>

```

to rezultat sijedećim prikazom na pregledniku:

Ovo je tekst samo H1

Ovo je tekst H1 sa klasom. Ocito je da stil po klasi ima veci prioritet od element selektora.

Ovo je tekst H1 sa klasom i ID. Ocito je da stil po ID ima veci prioritet od stila po klasi i po element selektoru.

A opet na kraju inline stil "pobjedi"...

Ilustracija 26 - Implementacija prioriteta

2.4 Pozicioniranje elemenata

Početak izrade svake web-stranice svodi se na kreiranje *layout* ili osnovnog „kostura“ ili rasporeda glavnih elemenata. Taj raspored glavnih elemenata određuje dijelove stranice u kojima će se smjestiti određeni sadržaji. *Container* ili okvir u koji se smještaju sadržaji, najčešće se implementira upotrebom *div* tagova. Div element je opisan u prethodnom poglavlju.

Izbor osnovnog izgleda stranice ovisi o namjeni stranice i ne predstavlja tehničko pitanje. Izbor izgleda stranice se radi na nivou planiranja projekta, što uključuje želju klijenta za kojeg se stranica radi, estetski i stilski izbor grafičkih dizajnera, te limiti

koje nameće aplikacijski dio web-stranice koji implementira podršku poslovnim procesima ukoliko se radi web aplikacija.

Pozicioniranje div elemenata na stranci implementira se sljedećim obilježjima:

- static
- relative
- absolute
- fixed
- sticky,

te se dodatno koristi i flex, odnosno flexbox koji se obično ne svrstava u pozicioniranje, ali se također koristi za layout stranice, kao i za mnoge druge potrebe kao što je raspored sadržaja unutar elemenata layouta i sl.

Position static predstavlja default-no odnoso osnovno pozicioniranje elemenata onako kako je to predefinirano.

Position relative predstavlja relativnu poziciju uz predefinirno pomjeranje u odnosu na default-nu poziciju elementa.

Position absolute predstavlja relativno pozicioniranje u odnosu na parent element.

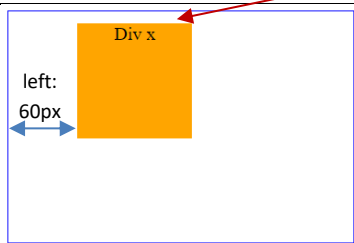
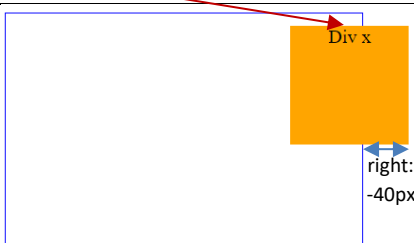
Position sticky omogućava definisanje minimalne udaljenosti od ivica parent elementa. Pozicioniranje elementa *sticky* pozicijom rezultira ponašanjem sličnom *absolute* pozicioniranju u slučaju da je element udaljeniji od predefiniranog minimuma, a ponašanje mu se promijeni u slučaju da se scroll-om pomjeri sadržaj parenta preko predefiniranog limita. U tom slučaju element se ponaša slično elementu čije je pozicioniranje definirano kao *absolute*.

2.4.1 Position absolute

Kako je navedeno, *position absolute* predstavlja relativno pozicioniranje elementa u odnosu na parent element. To u praksi znači da će konačna pozicija elementa ovisiti o poziciji parent elementa uz pomjeranje koje je definisano ostatkom CSS obilježja, kao što su *top*, *left* itd.

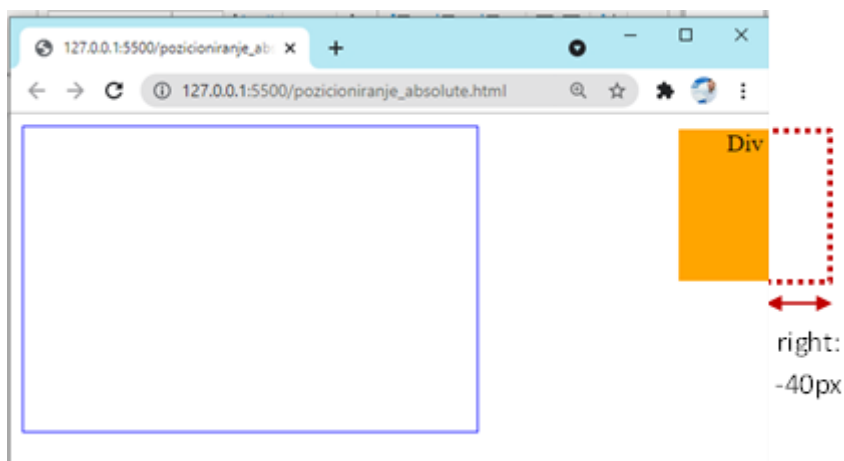
U sljedećim primjerima može se vidjeti rezultat implementacije *position absolute* na div. Parent div koji je oivičen plavom linijom ima definisan *position relative*. Njegov child div je obojen narančastom (engl. orange) bojom. Kao što se na primjeru vidi, postavljanje obilježja child div-a kao što su *top*, *left* i *right*, rezultirat će pomakom

child div-a u odnosu na parent div. U primjeru prikazanom na lijevoj slici, child div je pomjeren za 60 pixela od lijeve strane parenta dok je u desnom primjeru child div pomjeren za -40 pixela u odnosu na desnu marginu parent div-a. U desnom primjeru, korisniku će biti prikazana stranica na kojoj će izgledati da je child div pozicioniran djelomično van okvira parent div-a.

top: 10px	
	
<pre> <div-x{ 100px;="" 10px;="" 1px="" 200px;="" 300px;="" 60px;="" <="" <div-parent{="" absolute;="" background-color:="" blue="" border:="" center;="" height:="" left:="" orange;="" position:="" pre="" relative;="" solid;="" text-align:="" top:="" width:="" }=""> </div-x{></pre>	<pre> <div-x{ -40px;="" 100px;="" 10px;="" 1px="" 200px;="" 300px;="" <="" <div-parent{="" absolute;="" background-color:="" blue="" border:="" center;="" height:="" orange;="" position:="" pre="" relative;="" right:="" solid;="" text-align:="" top:="" width:="" }=""> </div-x{></pre>
<pre> <div class="div-parent"> <div class="div-x">Div x</div> </div> </pre>	

Da bi navedeno pozicioniranje bilo ispravno, potrebno je da obilježje position parent div-a bude relative. U suprotnom, neće biti moguće ispravno postići navedeno pozicioniranje. Primjer neispravnog korištenja position absolute dat je na sljedećoj slici i pripadajućem kodu.

Position parent elementa nije postavljena kao relative, što će rezultirati neispravnim pozicioniranjem child elementa.



Ilustracija 27 - Primjer neispravnog pozicioniranja

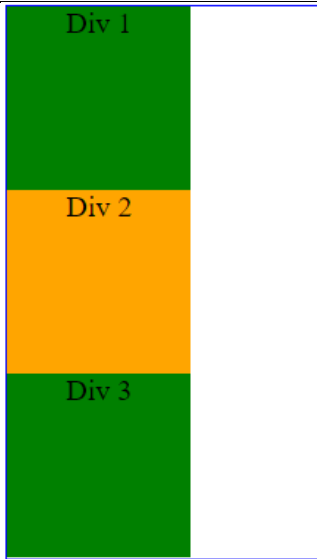
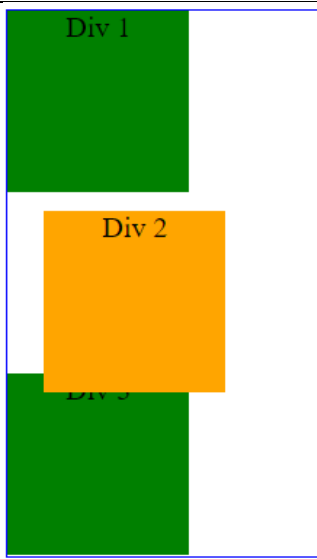
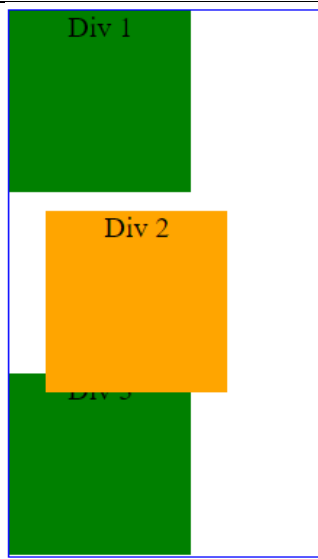
Isti rezultat u sva 3 html fajla	
<pre><div class="div-parent"> <div class="div-x">Div x</div> </div></pre>	<pre>.div-x{ position: absolute; background-color: orange; width: 100px; height: 100px; text-align: center; top: 10px; right: -40px } .div-parent{ width: 300px; height: 200px; border: 1px blue solid; }</pre>
<pre><div class="div-parent"></div> <div class="div-x">Div x</div></pre>	
<pre><div class="div-x">Div x</div> <div class="div-parent"></div></pre>	

2.4.2 Position relative

Position relative, za razliku od prethodnog pozicioniranja, rezultira relativnim pozicioniranjem elementa na stranici u odnosu na njegovu default poziciju. To znači da će obilježja *top*, *left* itd., pozicionirati elemente u odnosu na poziciju gdje bi se inače nalazili bez navedenih obilježja.

U sljedećem primjeru dat je prikaz korištenja *position relative* u odnosu na *position static*. Kao što se može primijetiti, CSS kod nije optimiziran. Umjesto optimizacije, dat je detaljniji popis zbog lakšeg razumijevanja primjera. U lijevom primjeru prikazana je pozicija elementa s *div-x* klasom čiji je sadržaj tekst „Div 2“. U lijevom

primjeru nisu postavljena obilježja left i top, te to rezultira pozicioniranjem elementa na default poziciji. Na srednjoj slici prikazan je isti primjer s razlikom u postavljanju obilježja left i top, što rezultira pomjeranjem pozicije elementa za zadati broj piksela u odnosu na njegovu default poziciju. U desnom primjeru prikazan je isti rezultat koji je dobijen korištenjem translateX i translateY opcija.

<pre> <body> <div class="div-parent"> <div class="div-s">Div 1</div> <div class="div-x">Div 2</div> <div class="div-s">Div 3</div> </div> </body> </pre>		
		
<pre> .div-x{ position: relative; background- color: orange; width: 100px; height: 100px; text-align: center; left: 0px; top: 0px; } .div-s{ position: static; } </pre>	<pre> .div-x{ position: relative; background- color: orange; width: 100px; height: 100px; text-align: center; left: 20px; top: 10px; } .div-s{ position: static; } </pre>	<pre> .div-x{ position: static; background- color: orange; width: 100px; height: 100px; text-align: center; transform: translateX (20px) translateY(10px); } .div-s{ position: static; } </pre>

<pre> background- color: green; width: 100px; height: 100px; text-align: center; } .div-parent{ width: 200px; border: 1px blue soli d; } </pre>	<pre> background- color: green; width: 100px; height: 100px; text-align: center; } .div-parent{ width: 200px; border: 1px blue soli d; } </pre>	<pre> position: static; background- color: green; width: 100px; height: 100px; text-align: center; } .div-parent{ width: 200px; border: 1px blue soli d; } </pre>
---	---	---

U nastavku je dat primjer cijelog koda HTML stranice koja rezultira prikazom kao na primjeru na desnoj slici.

```

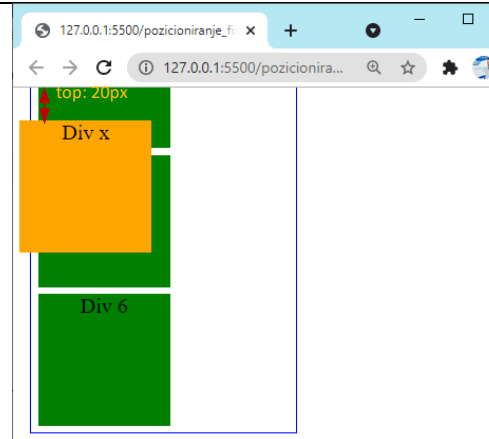
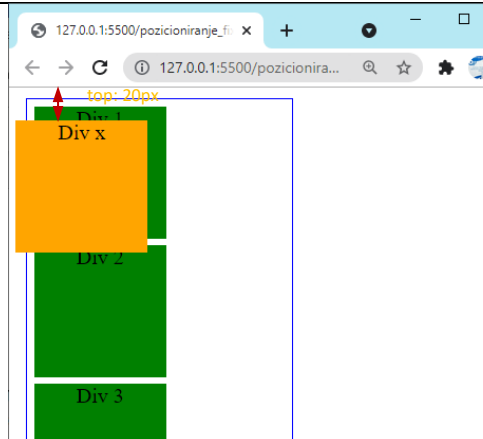
<!DOCTYPE html>
<html lang="en">
<head>
  <style>
    .div-x{
      position: static;
      background-color: orange;
      width: 100px;
      height: 100px;
      text-align: center;
      transform: translateX(20px) translateY(10px);
    }
    .div-s{
      position: static;
      background-color: green;
      width: 100px;
      height: 100px;
      text-align: center;
    }
    .div-parent{
      width: 200px;
      border: 1px blue solid;
    }
  </style>
</head>
<body>
  <div class="div-parent">
    <div class="div-s">Div 1</div>
    <div class="div-x">Div 2</div>
    <div class="div-s">Div 3</div>
  </div>
</body>

```

```
</html>
```

2.4.3 Position fixed

Za razliku od prethodna dva načina pozicioniranja, koja su vezana za ostatak sadržaja HTML stranice, position fixed rezultira pozicioniranjem elementa u odnosu na prozor web preglednika. Pomoću position fixed, moguće je pozicionirati element na stranici potpuno neovisno o ostalim elementima. Na sljedećim primjerima može se primijetiti da je pozicija diva klase div-x uvijek na istom mjestu, čak i u slučaju pomjeranja ostatka stranice *scroll* akcijom. U primjeru je definisana pozicija navedenog diva 0 piksela u odnosu na lijevu granicu prozora preglednika i 20 piksela u odnosu na gornju granicu prozora preglednika. Prostor za prikaz sadržaja stranice na pregledniku, što ne uključuje gredice s izbornicima, navigacijske komande i sl., uopćeno se nazva „viewport“. Dakle, može se reći da *position fixed* rezultira prikazom elementa relativno pozicioniranog u odnosu na *viewport*.

	
<pre>div{ margin: 5px; } .div-x{ position: fixed; background-color: orange; width: 100px; height: 100px; text-align: center; left: 0px; top: 20px; } .div-s{ position: static; background-color: green; width: 100px; height: 100px; text-align: center; }</pre>	<pre><div class="div-parent"> <div class="div-s">Div 1</div> <div class="div-s">Div 2</div> <div class="div-s">Div 3</div> <div class="div-s">Div 4</div> <div class="div-s">Div 5</div> <div class="div-s">Div 6</div> </div> <div class="div-x">Div x</div></pre>

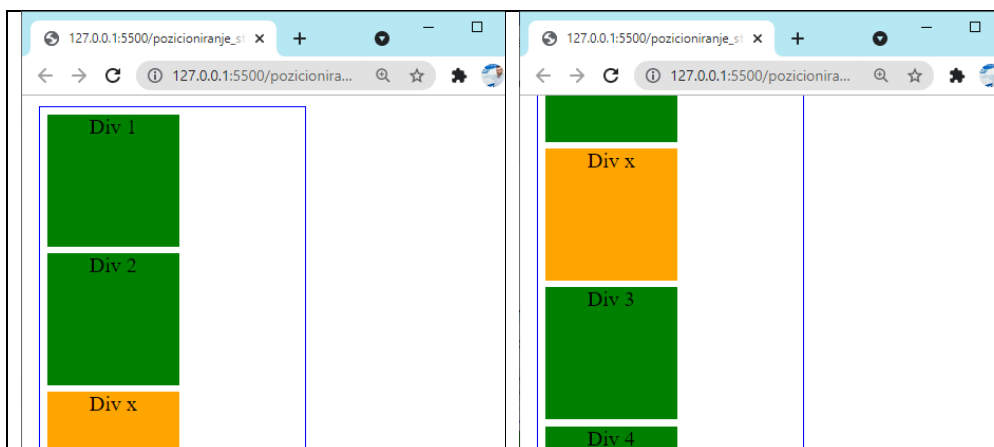
<pre> } <div-parent{ 1px="" 200px;="" <="" blue="" border:="" pre="" solid;="" width:="" }=""> </div-parent{></pre>	
---	--

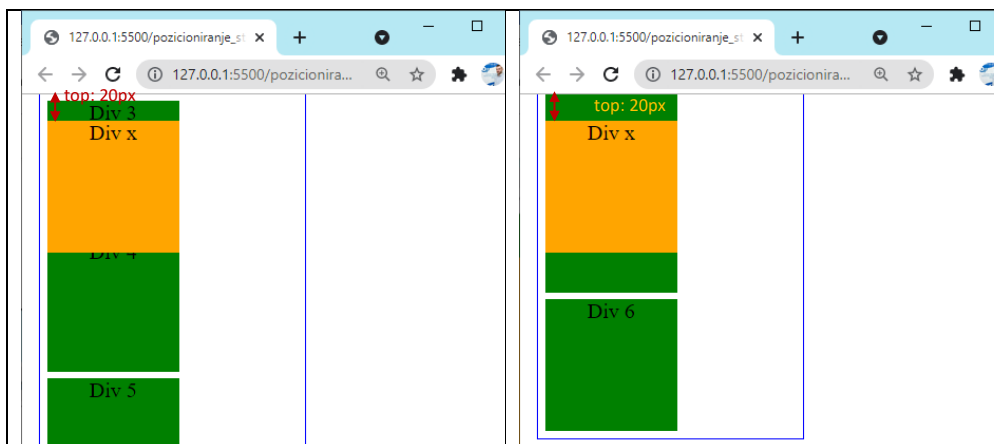
2.4.4 Position sticky

Position sticky rezultira pozicioniranjem elementa na *viewport* preglednika, ovisno o trenutnom prikazu sadržaja. Ukoliko je npr. postavljeno obilježje *top* u vrijednosti *X* piksela, a trenutni prikaz na *viewport*-u preglednika prikazuje sadržaj stranice tako da je gornja ivica *viewport*-a udaljena više od *X* piksela u odnosu na gornju marginu elementa, element će biti pozicioniran na način da okupira svoju default poziciju. Dakle, element će biti pozicioniran kao i u slučaju da je obilježje *position* postavljeno na vrijednost *relative*. Međutim, ukoliko korisnik *scroll* akcijom pomjeri sadržaj tako da je ivica *viewport*-a na udaljenosti od *X* piksela u odnosu na gornju marginu elementa, te nastavi sa *scroll* akcijom kako bi sadržaj pomjerao dalje, element će ostati pozicioniran na udaljenosti od *X* piksela od gornje ivice *viewport*-a, bez obzira na ostatak sadržaja stranice.

Zbog ovog ponašanja, ova vrijednost obilježja *position* se i naziva *sticky* (engl. *Sticky* = ljepljiv) jer korisniku izgleda kao da se element zalijepi na određenu poziciju.

U sljedećem primjeru dat je prikaz korištenja *sticky* pozicioniranja, te izgled *viewport*-a preglednika prilikom *scroll* akcije u slučaju da je *position* obilježje elementa *div-x* postavljeno na vrijednost *sticky*, a obilježje *top* postavljeno na vrijednost *20px*.





```
div{
    margin: 5px;
}
.div-x{
    position: sticky;
    background-color: orange;
    width: 100px;
    height: 100px;
    text-align: center;
    left: 0px;
    top: 20px;
}
.div-s{
    position: static;
    background-color: green;
    width: 100px;
    height: 100px;
    text-align: center;
}
.div-parent{
    width: 200px;
    border: 1px blue solid;
    position: relative;
}
```

```
<div class="div-parent">
  <div class="div-s">Div 1</div>
  <div class="div-s">Div 2</div>
  <div class="div-x">Div x</div>
  <div class="div-s">Div 3</div>
  <div class="div-s">Div 4</div>
  <div class="div-s">Div 5</div>
  <div class="div-s">Div 6</div>
</div>
```

2.4.5 Flex box

Kao što je već pomenuto, flex box se obično ne svrstava u komande za pozicioniranje elemenata. Iako se Flexbox se smatra layout modulom, najčešće se spominje u kontekstu pozicioniranja elemenata i često se koristi za potrebe kreiranja osnovne postavke i rasporeda stranice.

Da bi se implementirao flexbox, potrebno je kreirati parent i nekoliko child div elemenata. Implementacija flexboxa se vrši na parent elementu dodavanjem obilježja „display: flex;“. Child elementi nemaju posebno obilježje vezano za flexbox. Dakle, flexbox osobina se definira samo na parent, odnosno „wrapper“ elementu.

Flexbox ima šest dodatnih obilježja kojima se određuje način prikaza sadržaja:

- flex-direction
- justify-content
- align-items
- align-content
- flex-wrap
- flex-flow.

Flex direction obilježje služi za određivanje pravca i smjera redanja child elemenata. Dakle direction određuje vertikalnu ili horizontalnu orijentaciju te redanje elemenata od prvog navedenog ka zadnjem i obratno.

Sljedeći CSS kod

```
<style>
    .spoljni {
        display: flex;

/*redoslijed*/
        /*flex-direction: column-reverse; */
        /*flex-direction: column;*/
        flex-direction: row;
        /*flex-direction: row-reverse;*/

/*ponasanje unutrasnjih*/
        /* flex-wrap: wrap;*/ /*u ovom slucaju izbacite width u
procentima iz unutrasnjih elementa*/
        flex-wrap: nowrap;
```

```

/*justify*/
    justify-content: center;
/*justify-content: flex-start;
    justify-content: flex-end;
    justify-content: space-around;
    justify-content: space-between;*/

    background-color: lightblue;
}

.spoljni > .unutrasnji {
    background-color: lightcyan;
    width: 15%;
    margin: 10px;
    padding: 10px;
}
</style>

```

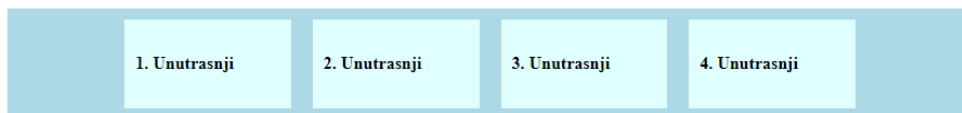
implementiran sljedećim HTML kodom

```

<div class="spoljni">
  <div class="unutrasnji">
    <h4>1. Unutrasnji</h4>
  </div>
  <div class="unutrasnji">
    <h4>2. Unutrasnji</h4>
  </div>
  <div class="unutrasnji">
    <h4>3. Unutrasnji</h4>
  </div>
  <div class="unutrasnji">
    <h4>4. Unutrasnji</h4>
  </div>
</div>

```

rezultat će prikazom kao na sljedećoj slici.



Ilustracija 28 - Izgled Flexbox elementa

Kao što je već spomenuto, flexbox se ne svrstava u pozicioniranje elemenata. Međutim, flexbox kao layout modul se često koristi kako za raspored elemenata na stranici, tako i za raspored sadržaja elemenata zbog svoje osobine da se lako i jednostavno raspoređuju sadržaji.

2.5 Pseudo-klase i pseudo-elementi

Pseudoklase i pseudoelementi predstavljaju vrste CSS formatiranja u kojima se formatira dio elementa ili element u specifičnom stanju.

Pseudoklasa formatira element u specifičnom stanju. Pod specifičnim stanjem se najčešće podrazumijeva neka od korisničkih akcija na tom elementu kao npr. zadržavanje pokazivača miša na elementu ili postavljanje tzv. fokusa na element, što znači u sučaju text box elementa, klik unutar polja za upisivanje teksta. Ova vrsta stiliziranja se takođe često koristi za simuliranje dinamičnosti stranice kao u slučajevima menija u kojima se klikom ili prelaskom pokazivača miša preko parent elementa, aktivira vidljivost child elementa. Selektor se koristi za bilo koji element za koji je, po definiciji jezika, predviđeno da može imati iskoristivo stanje za potrebe formatiranja.

Pseudo-klasa se koristi tako što se navede element koji može imati određeno stanje, zatim dvotačka, te naziv pseudo-klase.

```
selektor:pseudoklasa{}
```

Način korištenja se može vidjeti na sljedećem primjeru:

```
.izborLinija:hover {  
    background-color: lightblue;  
}
```

U datom kodu se može vidjeti da je izabran selektor “.izborLinija” što predstavlja CSS klasu. Svaki element na stranici na koji je implementirana data klasa će dobiti navedeno obilježje pseudoklase. Pseudoklasa korištena u ovom primjeru je “hover” odnosno pozicioniranje pokazivača miša na datom elementu. Dakle, ukoliko na bilo

koji element, koji ima klasu ".izborLinija" pozicioniramo pokazivač, taj element će dobiti svijetloplavu boju pozadine.

Programiranje I

Programiranje I

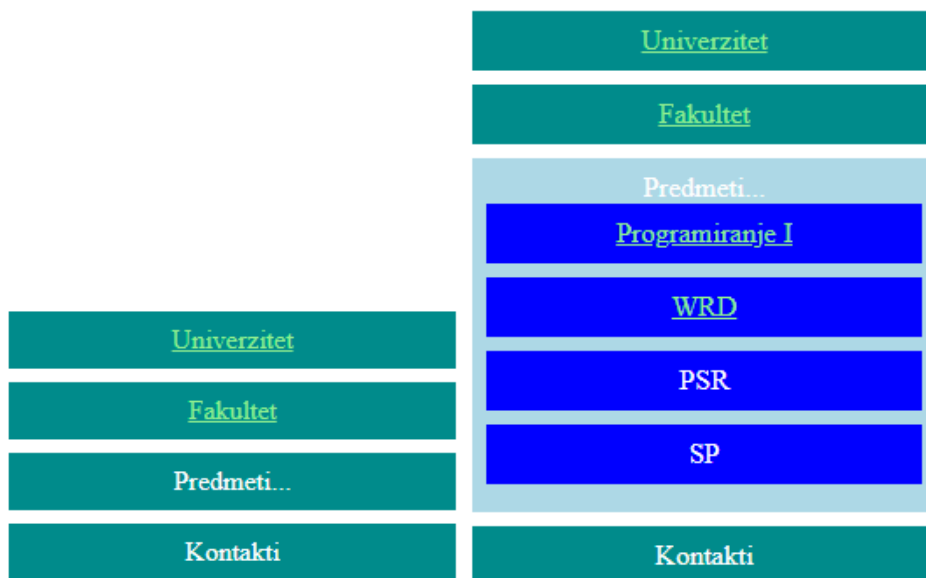
Osim toga, kako je navedeno, pseudoklase se često koriste da simuliraju pojavljivanje elemenata na stranici. U praksi se takva vrsta dinamičnosti ostvaruje koristeći programski jezik JavaScript, ali se za jednostavnije stranice može koristiti i ova osobina elemenata. Način implementiranja ove vrste simulacije dinamičkog menija se ostvaruje implementiranjem "display" obilježja, odnosno njegove vrijednosti "none" i "block". U sljedećem primjeru dat je prikaz dinamičkog menija ostvarenog korištenjem pseudoklase. Zbog jednostavnijeg prikaza, u primjeru je dat samo dio koda koji se odnosi na navedenu funkcionalnost.

```
.pod_meni{
    display:none;
    width: 100%;
    padding: 0px;
}
.stavkaMenija:hover>.pod_meni{
    display:block;
}
```

Navedeni CSS kod je implementiran preko sljedećeg HTML koda:

```
<li class = "stavkaMenija"> Predmeti...
    <ul class = "pod_meni">
        <li class = "pod_stavkaMenija">
```

to će rezultirati sljedećim prikazom na stranici.

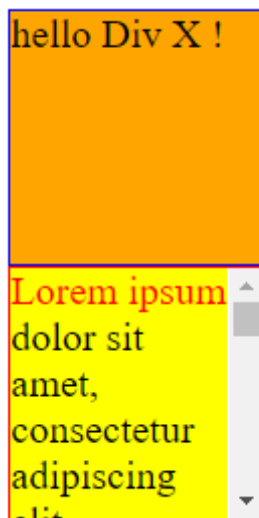


Ilustracija 29 - Simulacija dinamičkog menija korištenjem pseudoklase

U datom primjeru koristi se child element i na njega se primjenjuje promjena u trenutku kada se na parent elementu otkrijeo kidanje pseudoklase. U ovom slučaju se na elementu klase “.stavkaMenija” koristi pseudo-klasa “hover”, a promjena se primjenjuje na child element klase “.pod_meni”. Osim ovog načina, isto se može koristiti i u slučaju ostalih relacija između elemenata koje se mogu adresirati koristeći osobinu DOM-a koja je spomenuta u prethodnim poglavljima. Deteljniji opis navedenih relacija je dat u narednom poglavlju.

Pseudo-elementi, za razliku od pseudo-klasa ne koriste stanje elementa nego njegov dio. U praksi se najčešće koristi adresiranje prvog slova ili prvog reda nekog teksta kako bi se taj dio elementa naglasio ili jednostavno stilizirao na drugačiji način od ostatka tog elementa. Pseudo-element se koristi tako što se navede element kojem je moguće stilizirati određen dio, zatim se navode dvije dvotačke, te naziv pseudo-elementa, odnosno dijela kojeg se želi stilizirati.

Na slijedećem primjeru može se vidjeti korištenje pseudo-elementa za dodavanje sadržaja obilježjem “content” te prve linije elementa klase “.div-y”.



Ilustracija 30 - Primjer implementiranja pseudo-elementa

```
.div-x {
  border: 1px solid blue;
  width: 100px;
  height: 100px;
  background-color: orange;
}
.div-x::before {
  content: "hello ";
}
.div-x::after {
  content: "!";
}

.div-y {
  border: 1px solid red;
  width: 100px;
  height: 100px;
  background-color: yellow;
  overflow: auto;
}
.div-y::first-line {
  color: red;
}

<div class="div-x">
  Div X
</div>
<div class="div-y">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse vitae
  est augue. Etiam velit nisl, pretium ut molestie a, placerat in neque. Int
  eger tincidunt mi sit amet ex fermentum, ac .
  Div Y
</div>
```

2.6 Relacije selektora ugnježđenih (Descendant) elemenata – CSS kombinatori

CSS omogućava adresiranje ili dohvaćanje elemenata ovisno o njihovom međusobnom odnosu u DOM-u. Te relacije ili međusobni odnosi definirani su HTML jezikom, a DOM je ukratko opisan u poglavlju 1.2.

Sa stanovišta CSS stiliziranja, ove relacije su važne zbog mogućnosti adresiranja elemenata na stranici u prema njihovom međusobnom odnosu. Npr. ukoliko je potrebno stilizirati samo one „span“ elemente koji se nalaze unutar nekog div elementa na kojem je implementirana određena klasa, to se može uraditi na način da se elementi dohvate putem CSS kombinatora.

CSS kombinatori su:

Kombinator	Primjer sa opisom
Descendant Selector ili selektor relacije roditelj-dijete	div span{} – stilizirat će sve span elemente koji se nalaze unutar nekog div elementa neovisno koliko duboko se taj span element nalazi unutar DOM strukture. Bitno je samo da je neki od parent nivoa tog span elementa div element. Sintaksa relacije je space između selektora.
Child Selector ili selektor relacije roditelj-dijete koji se odnosi na prvi child određenog parent elementa	div > span{} – stilizirat će sve span elemente čiji je direktni parent div element. Sintaksa relacije je znak > između selektora.
Adjacent Selector ili selektor relacije slijedbenika/susjeda	div + span{} – stilizirat će sve span elemente čiji je prvi prethodnik div element. Sintaksa relacije je znak + između selektora.
General Sibling Selector ili opšti selektor relacije slijedbenika	div ~ span{} – stilizirat će sve span elemente čiji je prethodnik div element neovisno o tome ima li između njih još nekih elemenata. Sintaksa relacije je znak ~ između selektora.

Spomenute relacije CSS kombinatora se mogu koristiti s bilo kojim selektorima, što uključuje i kombiniranje elemenata s implementiranim klasama ili ID i sl.

Ukoliko je potrebno iste stilove implementirati na više selektora, nazivi selektora se odvajaju zarezom. Npr. **h1, h2, p {obilježje1: vrijednost1;**

obilježje2:vrijednost2;}}, implementirat će navedena obilježja i pripadajuće vrijednosti na elemente tipa h1, h2 i p neovisno o njihovom međusobnom položaju. Zbog toga se odvajanje selektora zarezom ne svrstava u CSS kombinatore.

2.7 Display property i inline i block kao display obilježja

Mada je *default* ponašanje svakog vidljivog HTML elementa definisan kao inline ili block, ova obilježja se posebno navode u CSS dijelu zbog čestog mijenjanja default ponašanja HTML elementa pomoću CSS stilova.

Default ponašanje blok elemenata je takvo da zauzimaju cijeli prostor prozora preglednika horizontalno neovisno o njihovoj stvarnoj veličini. To znači da se na pregledniku ne može prikazati nikakav dodatni sadržaj pored blok elemenata. Ostatak sadržaja je moguće prikazati ili iznad ili ispod blok elementa.

Za razliku od blok elementa, inline element zauzima samo onoliko prostora koliko je potrebno zbog sadržaja ili zbog obilježja koja se odnose na veličinu, a koja mogu forsirati veličinu prikaza.

Osim navedenih *default* vrijednosti display obilježja, može se koristiti i inline-block kao vrijednost. Inline-block forsira inline prikaz elemenata, implementirajući totalnu visinu zauzetog prostora prema elementu a najvećom visinom.

Svi HTML elementi koji se prikazuju na stranici imaju svoje *default* ponašanje u smislu inline ili block.

Neki od elemenata čije je *default* ponašanje inline su: <a>, , <input>, , <button> itd.

Neki od elemenata čije je *default* ponašanje blok su: <div>, <h1>-<h6>, <p>, <form>, , , <table> itd.

U praksi se često mijenja *default* ponašanje elemenata. Na sljedećem primjeru može se vidjeti promjena u ponašanju elemenata, te razlike između inline, inline-blok i blok vrijednosti obilježja display. Dat je sljedeći CSS kod.

```
<style>
  div{
    display: block;
    background-color: antiquewhite;
    margin-top: 10px;
```

```

        margin-bottom: 10px;
        padding-top: 10px;
        padding-bottom: 10px;
    }
    .prvi{
        min-height: 50px;
    }
    .drugi{
        min-height: 30px;
    }
    .treci{
        min-height: 40px;
    }
    .wrapper{
        background-color: lightblue;
        height: auto;
    }
    span{
        border: red solid 1px;
    }
}
</style>

```

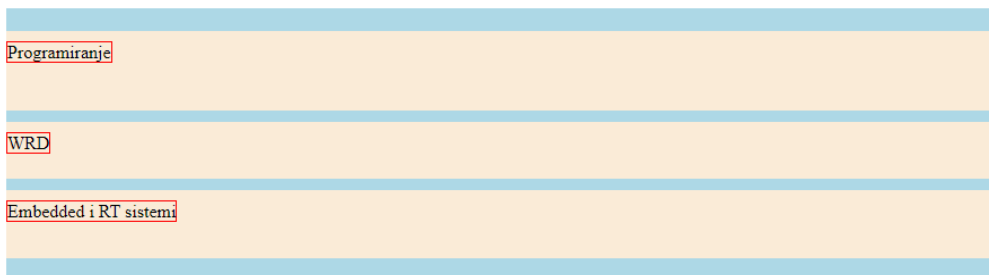
koji je implementiran sljedećim HTML kodom.

```

<div class="wrapper">
  <div class="prvi">
    <span>Programiranje</span>
  </div>
  <div class="drugi">
    <span>WRD</span>
  </div>
  <div class="treci">
    <span>Embedded i RT sistemi</span>
  </div>
</div>

```

što će rezultirati sljedećim prikazom na stranici.

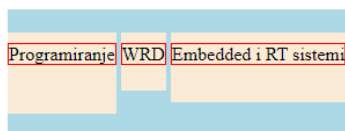


Ilustracija 31 - Izgled block elemenata

Međutim, ako se promijeni display obilježje div elementa te se postavi vrijednost inline-blok kao na sljedećem primjeru.

```
div{
  display: inline-block;
```

to će rezultirati sljedećim prikazom na pregledniku.

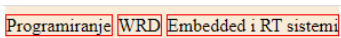


Ilustracija 32 - Izgled inline-block elemenata

U trećem primjeru, obilježje div elementa se postavlja na vrijednost inline

```
div{
  display: inline;
```

što će rezultirati sljedećim prikazom na pregledniku.



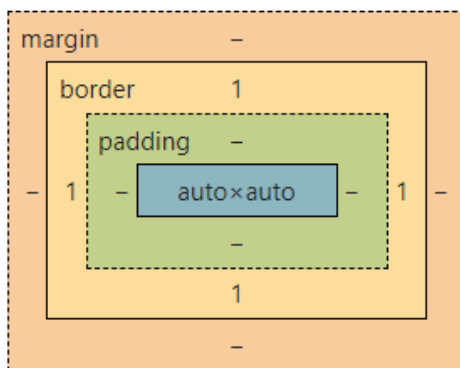
Ilustracija 33 - Izgled inline elemenata

2.8 Box model

Svi vidljivi HTML elementi na stranici se prikazuju u okviru tzv. Box modela. Box model predstavlja kompletan prostor na stranici, koji element može zauzimati i koji se može stilizirati. Box model uključuje elemente:

- Margine – vanjski okvir elementa izvan bordera

- Borders – granice elementa koje se mogu prikazati linijama
- Padding – prostor između granice elementa i sadržaja
- Prostor za sadržaj koji uključuje visinu i širinu elementa.



Ilustracija 34 - Box model HTML elementa

Većina preglednika omogućava programerima uvid u stvarni izgled box modela, te selekciju i uvid u pojedine elemente box modela. U Google Chrome pregledniku, kako je objašnjeno u uvodnom djelu, moguće je izabrati opciju Inspect->Elements gdje je moguće imati uvid u Box model svakog elementa. Ova opcija je vrlo korisna programerima posebno u slučaju kreiranja osnovnog layout-a stranice.

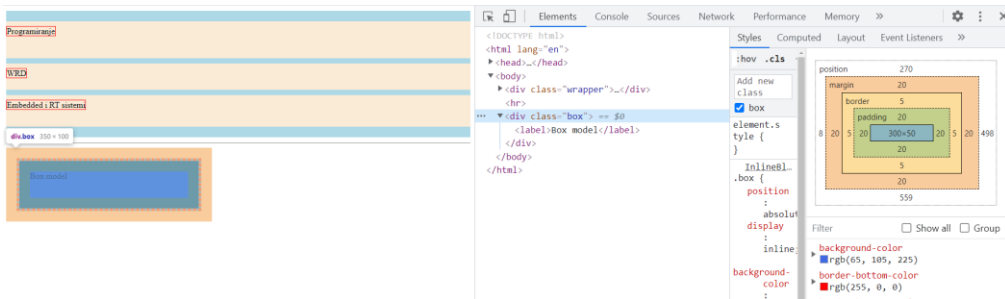
Mada Position ne spada u Box model, Google Chrome će prikazati i position elementa u sastavu istog prikaza. Sljedeći primjer demonstrira prikaz elementa u okviru Box modela. Dat je sljedeći CSS kod

```
.box{
    position: absolute;
    display: inline;
    background-color: royalblue;
    margin: 20px;
    border: 5px dotted red;
    padding: 20px;
    height: 50px;
    width: 300px;
}
```

koji je implementiran sljedećim HTML kodom

```
<div class="box">
  <label>Box model</label>
</div>
```

što će rezultirati sljedećim prikazom u pregledniku.

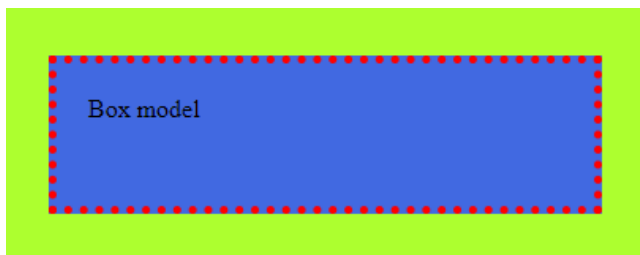


Ilustracija 35 - Realni izgled box modela u Google Chrome pregledniku

Outline opcija ne spada u elemente box modela. Outline služi za naglašavanje određenih elemenata. Ukoliko se npr. u prethodnom primjeru u CSS kod doda linija

```
outline: greenyellow solid 30px;
```

te se pristupi Ispect->Elements opciji u pregledniku Chrome, može se primijetiti da je područje margine s okolnim područjem naglašeno navedenom bojom outline-a.



Ilustracija 36 - Outline elementa

2.9 Responsivnost i Media query

Već od početaka uključivanja internet konekcije u mobilne uređaje, pokazala se potreba da se web- stranice, a posebno poslovne web aplikacije prilagode mobilnim uređajima. To je jedan od načina da se, osim kreiranja aplikacija za mobilne uređaje, omogući korisnicima lakše korištenje webstranca. Čak i u slučajevima da kompanija koja je vlasnik web-stranice ima namjeru kreirati mobilnu aplikaciju za Android, iOS i

ostale operativne sisteme mobilnih uređaja, i dalje postoji potreba da se i web-stranica prilagodi mobilnim uređajima. Takve stranice se nazivaju responsivnim stranicama ili stranicama koje imaju prilagodbu mobilnim uređajima ili uređajima s manjim ekranima za prikaz sučelja ili viewport-a. Npr. eBay stranica za kupoprodaju ima mobilnu aplikaciju koja se može preuzeti s Google Playstore, ali posjeduje i odgovarajuću verziju web-stranice prilagođene manjim viewport-ovima.

CSS pruža mogućnost *media query*, što omogućava programerima da na istom HTML dokumentu stiliziraju sve ili samo neke elemente ukoliko se stranica prikazuje na uređaju na kojem preglednik ima manje ili više od određene predefinirane vrijednosti broja piksela po horizontali.

Sintaksa *media query* je:

```
@media only screen and (max-width: 500px) {  
.....  
}
```

Ukoliko se određeni CSS stilovi napišu između vitičastih zagrada, ti stilovi će se implementirati na uređajima i u slučajevima kada je viewport manji ili jednak 500px. Tako npr. sljedeći CSS kod

```
<style>  
  img{  
    width: 200px;  
    height: 150px;  
    float:left;  
  
  }  
  @media only screen and (max-width: 500px) {  
    img{  
      width: 100px;  
      height: 50px;  
      float:right;  
    }  
  }  
</style>
```

će rezultirati veličinom slike 200x150 px koja je pozicionirana lijevo od teksta na viewportu horizontalne veličine veće od 500 px, dok će na mobilnim uređajima,

ukoliko se veličina preglednika smanji tako da je širina viewporta manja ili jednaka od 500 px, slika biti prikazana u veličini 100x50 px pozicionirana s desne strane teksta.

Najčešće se reponsivnost koristi i za potpunu promjenu reaporeda elemenata na stranici, posebno ako se radi o slikama, formama za unos podataka, a neki od elemenata koji nisu esencijalni za korištenje web aplikacije se obično potpuno uklone sa stranice, tako što im se postavi obilježje *display* na vrijednost *none*.

2.10 Boje, fontovi i ostala obilježja

Budući da nema potrebe da se u jednom materijalu navode sve specifikacije jednog jezika, u nastavku su data samo neka obilježja koja će omogućiti da se lakše savladaju osnove CSS dizajna.

Obilježje **float** je jedno od bitnijih obilježja sa stanovišta pozicioniranja i rasporeda elemenata na stranici. Također se koristi za pomjeranje elementa uz lijevu ili desnu ivicu prostora sadržaja unutar kojeg se taj element nalazi. Kao primjer dat je sljedeći CSS kod

```
<style>
  img{
    width: 200px;
    height: 150px;
    float:right;
  }
</style>
```

koji je implementiran sljedećim HTML kodom

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
  do eiusmod tempor incididunt ut labore et dolore magna
  aliqua....</p>
```

što će rezultirati sljedećim prikazom na pregledniku.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Ilustracija 37 - Implementacija "float: right" obilježja

Ukoliko bi se vrijednost obilježja float postavila na vrijednost „left“, to bi rezultiralo sljedećim prikazom na pregledniku.



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Ilustracija 38 - Implementacija "float: left" obilježja

Osim navedenih osobina, svaki vidljivi HTML element se može stilizirati bojama:

- **pozadine**
- **teksta (ako se radi o tekstualnom sadržaju)**
- **bordera**
- **outlinea.**

Boje u CSS jeziku mogu biti definirane na tri načina:

- nazivom boje
- heksadecimalnom vrijednošću
- heksadecimalnom vrijednošću s indeksom prozirnosti
- RGB paletom
- RGBA paletom koja uključuje i alfa index opacity
- HSL paletom
- HSLA paletom koja uključuje i alfa index opacity.

Načelno se nazivi boja grupišu u dvije grupe, grupa standardnih boja koja sadrži 16 naziva boja, te proširena grupa koja sadrži sve boje definisane CSS jezikom. Lista boja s nazivima i pripadajućim numeričkim vrijednostima se nalazi u prilogu.⁴

⁴ <https://www.w3.org/TR/css-color-3/>

Ukoliko se boja definira nazivom, dovoljno je navesti ime boje kao vrijednost određenog obilježja npr.:

```
background-color: aliceblue;
```

Ukoliko se željena boja ne nalazi na listi imenovanih boja, moguće je izabrati boju elementa na jedan od navedenih načina. Tako npr. boja može biti definirana heksadecimalnom vrijednošću ili heksadecimalnom vrijednošću s indeksom prozirnosti kao u sljedećem primjeru

```
background-color: #0000FF;  
color: white;
```

>Lorem ipsum dolor sit amet, con
dolore magna aliqua. Ut enim ad
ex ea commodo consequat. Duis

ili

```
background-color: #0000FF50;  
color: white;
```

>Lorem ipsum dolor sit amet, c
dolore magna aliqua. Ut enim
ex ea commodo consequat. D

Heksadecimalna vrijednost, kao i heksadecimalna vrijednost s indeksom prozirnosti se sastoji od 3 odnosno 4 dvocifrena heksadecimalna broja koja predstavljaju intenzitet crvene, zelene i plave boje, te indeks prozirnosti.

#0000FF50

crvena – zelena – plava – indeks prozirnosti

Može se primjetiti da numeričke vrijednosti mogu biti u rangi 00-FF za svaku pojedinu boju, odnosno prevedeno u decimalni sistem te vrijednosti mogu biti 0-255.

Na isti način moguće je postaviti boju i korištenjem RGB plete i RGB palete koja uključuje i alfa index opacity.

#rgba(0, 0, 255, 0.5);

crvena – zelena – plava – alfa opacity

Umjesto numeričke vrijednosti, moguće je koristiti i procenat intenziteta boje kao na sljedećem primjeru.

#rgba(0%, 0%, 100%, 50%);

crvena – zelena – plava – alfa opacity

Osim navedenih primjera, moguće je koristiti i HSL i HSLA vrijednosti za obilježje boja elemenata.

#hsla(240,100%,50%,0.5);

Hue – Saturation – Light – alfa opacity

Navedeni primjer HSLA rezultirat će plavom bojom sa 50% alfa opacity vrijednosti.

Fontovi su jedan od važnijih elemenata stiliziranja svakog programera. CSS omogućava stiliziranje familije fonta, stila fonta i njegove veličine. Familije fontova su: Serif, Sans-serif, Cursive, Fantasy i Monospace. Stilovi fontova su: normal, italic i oblique (sličan italicu).

Veličina fonta može biti definisana na više načina. Jedan od načina definisanja je fiksno u vrijednostima piksela npr. „font-size: 20px;“. Također, neki programeri preferiraju definirati veličinu fonta u *em* vrijednosti koja je relativna u odnosu na trenutnu veličinu fonta. Osim navedenih moguće je definirati veličinu fonta i u procentima npr. font-size: 150%;, što će rezultirati navedenim uvećanjem u odnosu na osnovnu veličinu fonta.

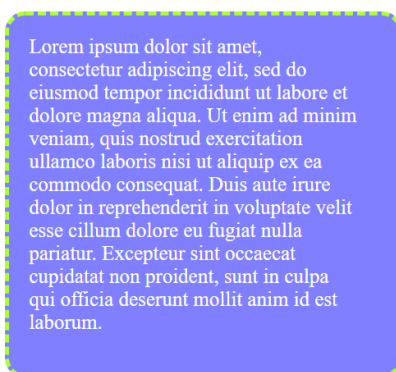
Ukoliko je potrebno postaviti veličinu fonta na način da prati veličinu prozora preglednika, odnosno njegovog viewporta, moguće je koristiti jedinice vw (*vw – viewport width*). Tako npr. „font-size: 2vw;“ će postaviti veličinu fonta na 2% veličine viewporta (1vw = 1% širine viewporta). Koristeći ovu mjernu jedinicu za veličinu

fonta, moguće je ostvariti određenu responsivnost ili prilagodbu veličine fonta prikazu stranice na određenom uređaju.

Border je još jedna od opcija koju programeri često koriste kako bi stilizirali sadržaj. Obilježje border ima više različitih vrijednosti od kojih se najčešće koriste boja, tip (solid, dotted, dashed) i debljina linije. Ukoliko je potrebno moguće je odrediti i radius ugla bordera kako bi element bio prikazan sa zaobljenim ivicama. Tako npr. sljedeći CSS kod

```
p{  
    background-color:hsla(240,100%,50%,0.5);  
    color: white;  
    padding: 5% 10% 10% 5%;  
    font-size: 5vw;  
    border: greenyellow dashed 5px;  
    border-radius: 5vw;  
}
```

rezultira sljedećim prikazom na pregledniku.



Ilustracija 39 - Prikaz opcija bordera

Kako se s primjera vidi, CSS kod *border: greenyellow dashed 5px;* rezultira prikazom na kojem je boja ivice žuto-zelena, ivica je prikazana isprekidanom linijom debljine 5 px. Također, linija *border-radius: 5vw;* rezultira zaobljenošću uglova u iznosu 5% širine viewpota preglednika.

Na primjeru se može vidjeti i da se vrijednosti obilježja padding sastoji od četiri odvojene procentualne vrijednosti. U slučaju ovakve definicije padding, navedene

četiri vrijednosti se odnose na gornji, desni, donj i lijevi padding u redoslijedu kojim je napisano.

```
padding: [gornji] [desni] [donji] [lijevi];
```

3

Osnove JavaScript jezika

3.1 Uvod u JS

Razvojem interneta kao globalne mreže krajem dvadesetog vijeka, privredni subjekti širom svijeta su počeli uviđati potencijale ove mreže za potrebe marketinga te pristupa kupcima. Budući da HTML ima ograničene mogućnosti po pitanju interakcije s korisnicima web-stranica, pokazala se potreba za uključivanjem dodatnih mogućnosti kao što su dinamičke web-stranice, mogućnost interakcije s korisnikom te naprednih multimedijalnih sadržaja. Odgovor na rastuće potrebe je došao u formi skripting jezika. Prvu verziju JavaScript jezika razvio je Brendan Eich 1995-te godine. JavaScript je razvijen za Netscape 2 navigator, odnosno web-preglednik. JavaScript je ostao standardiziran 1997-me godine u okviru ECMA-262⁵ standarda.

JavaScript je objektno orijentirani skriptni programski jezik. Sa stanovišta klasifikacije, JavaScript se može svrstati i u druge kategorije. Jedna od kategorija u

⁵ <https://www.ecma-international.org/publications-and-standards/standards/>

koju se JavaScript svrstava je i u programske jezike visokog nivoa, što govori o visokom nivou apstrakcije. Zatim, JavaScript se svrstava i u tzv. *multi-paradigm* jezike, odnosno jezike u kojima je implementirano više programskih paradigmi. Osim toga, obilježje JavaScript jezika je i to da spada u klasu jezika koji imaju tzv. *just-in-time* kompajliranje. Takvi jezici se prevode u toku izvršavanja, za razliku od drugih programskih jezika koji se odlikuju time da im se izvorni kod prvo prevede/kompajlira, linkuje, a zatim izvršava. Također, JavaScript se smatra *weakly typed* ili *untyped* dinamičkim programkim jezikom.

JavaScript nema poveznica s Java programskim jezikom. JavaScript je primarno namijenjen za programiranje web baziranih aplikacija i funkcionalnosti koje se izvršavaju na internet preglednicima.

Sintaksa JavaScript jezika bazirana je na ECMAScript standardizaciji. Sa stanovišta mjesta izvršenja programa, JavaScript je *client-side* skriptni jezik.

JavaScript omogućava kreiranje dinamičkih web-stranica čiji se sadržaj može promijeniti izvršenjem JavaScript koda. Budući da je ovaj materijal namjenjen početnicima, obrađuju se samo najosnovniji elementi JavaScript jezika, te se ovaj materijal ne može smatrati JavaScript referencom.

3.2 Client Side i Server Side scripting

Da bi se ispravno razumio koncept lokacije izvršenja JavaScript koda, potrebno je razumjeti razliku između *client-side* i *server-side* skriptnih jezika. Nakon poslanog zahtjeva, odnosno HTTP *request-a* od strane klijentskog preglednika, zahtjev se prima na serverskoj strani, te se u slučaju web bazirane aplikacije, procesira zahtjev i izvršava kod. U principu, *server-side* skripte se izvršavaju na procesoru servera i generiraju odgovor koji se vraća klijentskoj strani, najčešće u formi HTML koda.

Nasuprot tome, u slučaju *client-side* skripte koje se također nalaze na web site-u na serveru, prilikom primanja zahtjeva od strane klijenta, server ih šalje klijentskom pregledniku, koji prima izvorni kod te skripte, te ga izvršava na procesoru klijentskog računara, odnosno uređaja kojim se pristupa tom web site-u. Ukoliko se uzme primjer server side skripte napisane npr. u VB Script jeziku kako slijedi:

```
<h2>  
<%  
For brojac As Integer = 0 To 10
```

```
        Response.Write ("Ovo je tekst koji kreira server-side  
scripting<br>")  
        Next brojac  
    %>  
</h2>
```

te ukoliko se navedeni kod nalazi u dokumentu s ekstenzijom .aspx i ukoliko je dokument smješten na odgovarajuću lokaciju na serveru koji podržava tu vrstu web aplikacija, prilikom slanja zahtjeva od strane klijentskog preglednika, to će rezultirati vraćanjem HTML koda koji se sastoji od deset uzastopnih HTML elemenata tipa H2 čiji sadržaj je navedeni tekst.

Međutim, ukoliko se na serveru nalazi sljedećikod pisan u JavaScript jeziku kako slijedi:

```
<h2>  
    <script>  
        for (i=0;i<10;i++)  
        {  
            document.write("Ovo je text koji kreira client-side  
scripting<br>");  
        }  
    </script>  
</h2>
```

to će rezultirati da se navedeni izvorni kod pošalje klijentskom pregledniku. Prilikom učitavanja stranice, preglednik će izvršiti navedeni kod, čije će izvršavanje rezultirati kreiranjem deset uzastopnih HTML elemenata tipa H2 čiji sadržaj je navedeni tekst. Međutim, potrebno je naglasiti da će se na klijentskom pregledniku moći vidjeti i izvorni kod u JavaScript jeziku, što u prethodnom slučaju neće biti moguće, jer izvorni kod ostaje i izvršava se isključivo na serveru.

Početnici često postavljaju pitanje koja opcija je bolja. Bitno je naglasiti da ne postoji bolja ili lošija opcija u smislu *server-side* i *client-side* skripti. One se koriste za potpuno drugačije potrebe i nije ih moguće u tom smislu komparirati.

3.3 Način uključivanja JavaScript u HTML dokument

JavaScript kod se može uključiti u HTML dokument na dva tri osnovna načina:

- interni kod u HTML-u (preko script tag)
- eksterni file (preko Javascript biblioteke)

- kroz atribut (preko html tag atributa).

Mjesto uključivanja JavaScript koda može biti *head* ili *body* sekcija HTML dokumenta ili kombinirano, ovisno o potrebi i zahtjevima koje stranica treba zadovoljiti.

Interni kod u HTML dokumentu je najjednostavniji način uključivanja. Sekcija dokumenta u koji se piše JavaScript kod treba biti uokvirena u `<script></script>` tag. Taj tag će omogućiti pregledniku da prepozna JavaScript kod i da izvrši linije koda koje se nalaze u tom tagu. Ukoliko se u *body* dijelu HTML dokumenta napiše sljedeći JavaScript kod

```
<body>
  <script>
    document.write("Primjer internog JavaScript koda");
  </script>
</body>
```

to će rezultirati sljedećim ispisom u pregledniku.

Primjer internog JavaScript code-a

Ilustracija 40 - Rezultat izvršenja internog JavaScript koda

Naravno, radi se o najjednostavnijem primjeru koji predočava tzv. interni JavaScript kod. Moguće je sve funkcionalnosti koje je potrebno ostvariti na nekoj stranici, programirati u formi internog JavaScript koda. Međutim, kao i u slučaju CSS, najbolja praksa je razdvojiti JavaScript kod od HTML i CSS koda u zasebne dokumente koji se mogu po potrebi uključivati. Nekoliko je prednosti te vrste uključivanja skripti u stranicu od kojih su najvažnije čitkost koda i ponovna upotreba, odnosno ponovna iskoristivost već napisanih programskih skripti. Zbog toga se JavaScript kod piše u zasebnim dokumentima sa .js ekstenzijom. Da bi se eksterni .js dokument uključio u HTML stranicu i time omogućile programirane funkcionalnosti, najčešće se u head sekciji navodi linija `<script type="text/javascript" src="[naziv script file-a sa putanjom]"></script>`. Atribut *src* sadrži naziv datoteke ekstenzijom i putanjom. Npr. ukoliko imamo JavaScript dokument s nazivom *mojSkript.js* i koji se nalazi u istom direktoriju kao i HTML dokument u koji ga uljučujemo, linija koda kojom ćemo uključiti JavaScript dokument je:

```
<script type="text/javascript" src="mojSkript.js"></script>
```

Sadržaj .js dokumenta je tekstualni i sadrži isključivo JavaScript funkcije ili linije koda kao npr.

```
alert("Ovo je poruka koju je generirao kod iz eksterne biblioteke");
```

Treći način uključivanja JavaScript koda je preko atributa HTML elementa. Neki HTML elementi, odnosno njihovi tagovi mogu sadržavati attribute čiji sadržaj može biti poziv JavaScript funkcije ili čak i kraća linija koda kao u sljedećem primjeru:

```
<button onclick="getElementById('vrijeme').innerHTML=Date()  
+'<br/>' + ' Ovo je primjer JavaScript-a u tag-u'  
>Vrijeme?</button>  
<h1 id="vrijeme"></h1>  
<button onclick="poruka()">Poruka?</button>
```

U prvoj liniji koda prikazan je primjer HTML elementa tipa *button* u čiji je atribut *onclick* upisana linija JavaScript koda. Bitno je razumjeti da ovo nije preporučeni način korištenja atributa *onclick*, već samo demonstracija mogućnosti atributa. U trećoj liniji koda, prikazan je način korištenja atributa, kako bi se pozvala i izvršila JavaScript funkcija klikom na dugme. Naravno, nije moguće spoznati puni potencijal JavaScript jezika na ovako jednostavnim primjerima.

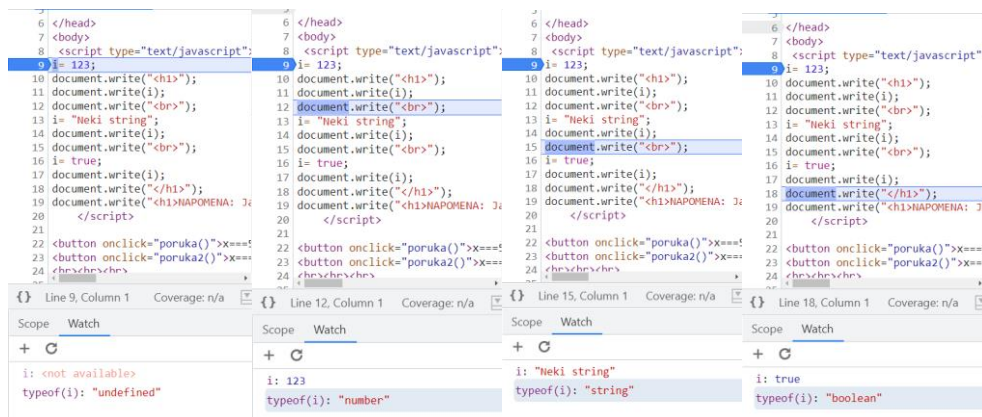
3.3 JS varijable

Kao i u svakom drugom programskom jeziku potrebno se upoznati s varijablama. Za razliku od klasičnih programskih jezika, JavaScript varijable se odlikuju nekim osobinama koje nisu standardne. Specifičnost JavaScript varijabli je jedna od najupečatljivijih posebnosti JavaScript jezika. Kako je navedeno u prethodnim poglavljima, JavaScript je *weakly typed* ili *untyped* programski jezik. Za razliku od C, C++, VB ili drugih klasičnih programskih jezika koji se odlikuju strožijim pravilima u pogledu tipova varijabli, JavaScript varijable mogu dinamički promijeniti tip podatka u vremenu izvršenja programa. Zbog toga se, prilikom deklariranja varijabli u JavaScript jeziku, ne određuje eksplicitno i tip podatka tih varijabli. Tip podatka se dinamički određuje prilikom inicijaliziranja varijable, odnosno prilikom dodjele vrijednosti varijabli. Ukoliko se izvrši sljedeći kod

```
i= 123;  
document.write("<h1>");  
document.write(i);
```

```
document.write("<br>");
i= "Neki string";
document.write(i);
document.write("<br>");
i= true;
document.write(i);
document.write("</h1>");
```

može se primijetiti da je tip podatka varijable i dinamički promijenjen kako je varijabla poprimala druge vrijednosti. Ukoliko se u npr. pregledniku Chrome, postavi break point na liniju `i=123`, može se pratiti izvršenje koda i u Watch dijelu prozora Inspect opcije preglednika pratiti promjena tipa podatka varijable `i`.



Ilustracija 41 - Dinamička promjena tipa podatka dodjelom različitih vrijednosti

U navedenim primjeru, varijabla `i`, dodjelom različitih vrijednosti, mijenja svoj tip podatka od `number`, preko `string` u `boolean`.

Kao i u većini drugih programskih jezika, varijable moraju imati jedinstvena imena koja se nazivaju i *identifiers*. Pravila imenovanja varijabli su sljedeća:

- varijabla mora imati jedinstveno ime u jednom scope-u
- imena varijabli su case-sensitive, odnosno velika i mala slova se razlikuju
- ime varijable mora počinjati slovom, znakom `$` ili znakom `_`.

Kao i u ostalim programskim jezicima, ključne riječi nije moguće koristiti kao imena varijabli.

JavaScript omogućava kreiranje varijabli i konstanti korištenjem ključnih riječi:

- let,
- var
- const.

Osim navedenih ključnih riječi, varijablu je moguće deklarirati i bez navođenja bilo koje ključne riječi, tako što se ime varijable navede u nekoj liniji koda. U tom slučaju, varijabla će biti deklarirana kao globalna varijabla, čiji scope pokriva sve funkcije. Ukoliko programer ne želi koristiti mogućnost implicitnog deklariranja globalne varijable bez navođenja ključne riječi var ili let, moguće je koristiti „use strict“ direktivu kojom se forsira striktni način pisanja JavaScript koda, u kojem, između ostalog, nije dozvoljeno takvo deklariranje varijabli. Direktiva „use strict“ uvedena je u JavaScript u reviziji 5, 2009. godine. Primjer korištenja direktive dat je u sljedećoj funkciji:

```
strikno()=>{
    "use strict"
    a=5;
    console.log(a);
}
```

Izvršavanje navedene funkcije rezultirat će ReferenceError uz poruku da a nije definiran jer u striktnom režimu rada nije moguće implicitno deklarirati globalnu varijablu. Većina web preglednika podržavaju tu direktivu s izuzetkom preglednika Internet Explorer do verzije 9.

Ključna riječ var je najstarija ključna riječ za kreiranje varijabli u JavaScript jeziku. Ona se koristi za deklariranje globalnih varijabli tako što se deklaracija navodi van funkcija, te lokalnih varijabli čiji je domen jedna funkcija. Kao i u ostalim programskim jezicima, razlikuje se deklariranje i inicijaliziranje varijable. Deklariranje s inicijaliziranjem varijable prikazano je u sljedećem primjeru.

```
var mojaVarijabla = 5;
```

Moguće je i samo deklarirati varijablu, te je naknadno inicijalizirati.

```
var mojaVarijabla;
mojaVarijabla = 5;
```

Također, moguće je deklarirati i inicijalizirati više varijabli u jednoj liniji kao npr.

```
var a, b, c=3, tekst="web razvoj";
```

Specifičnost JavaScript jezika u odnosu na ostale programske jezike je i tzv. *hoisting* varijabli. Hoisting je uveden kao termin u ECMAScript 6 (ES6) 2015. godine kako bi se opisala osobina JavaScript jezika, koja omogućava da se deklariranje varijable, funkcije ili objekta pomjeri na početak blok scope-a, čak i u slučajevima kad se varijabla inicijalizira ili koristi prije njenog deklariranja. Ukoliko je dat sljedeći kod:

```
function funkcija1() {  
  
    mojaVarijabla = 5;  
    var a, b, c=3, tekst="web razvoj";  
  
    document.getElementById('proba4').innerHTML += "primjer  
deklariranja varijable";  
    document.getElementById('proba4').innerHTML += "ključnom riječju  
var";  
    document.getElementById('proba4').innerHTML += (mojaVarijabla);  
    var mojaVarijabla;  
}  
function funkcija2() {  
    document.getElementById('proba7').innerHTML =(mojaVarijabla);  
}
```

Može se primijetiti da je varijabla *mojaVarijabla* deklarirana na kraju funkcije *funkcija1()*. Bez obzira na to što će „interpreter“ učitati liniju koda deklaracije varijable na dnu navedene funkcije, varijabla se može koristiti u funkciji *funkcija1()* čak i prije nego je deklarirana. Kao dokaz da je domen odnosno scope varijable samo u funkciji *funkcija1()*, može se izvršiti funkcija *funkcija2()* koja će rezultirati exception-om. Pomenuta osobina JavaScript jezika koja se naziva *hoisting* rezultira time da se čini da je interpreter podigao deklariranje varijable na početak scope-a te varijable.

Ključna riječ *const* se koristi za deklariranje konstantnih referenci na vrijednost ili objekat, za razliku od drugih programskih jezika kao što je C++ u kojem se ključna riječ *const* koristi za deklariranje konstanti. *Const* je uključen u ECMAScript 6 (ES6) 2015. godine. *Const* referenca ne može biti ponovno deklarirana, ponovno definirana ili inicijalizirana unutar jednog blok scope-a.

Ključna riječ *let* forsira striktnije korištenje varijabli. Varijabla deklarirana ključnom riječju *let* ne može biti ponovno deklarirana unutar jednog block scope-a. Takva

varijabla mora biti deklarirana prije korištenja za razliku od varijabli deklariranih ključnom riječju var. Posebna razlika između varijabli deklariranih ključnom riječju var i ključnom riječju let je u domenu varijable u smislu scope-a. Varijable deklarirane ključnom riječju var imaju scope sličan programskim jezicima sa striktnijom sintaksom kao što je C++. Npr. Ukoliko se, unutar neke funkcije, varijabla deklarira ključnom riječju var u dodatnim vitičastim zagradama, scope te varijable će biti samo unutar zagrada, što se može vidjeti na sljedećem primjeru:

```
function primjerVarijabli(value1, value2){
    result_1 = value1+value2; //globalna varijabla
    {
        var result_2 = value1+value2; //lokalna varijabla var
        let result_3= value1+value2; //lokalna varijabla let
        document.getElementById('proba4').innerHTML ="var unutar bock
scope-a "+result_2;
        document.getElementById('proba5').innerHTML ="let unutar bock
scope-a "+result_3;
    }
    document.getElementById('proba6').innerHTML ="var van bock
scope-a "+result_2;
    document.getElementById('proba7').innerHTML ="let van bock
scope-a "+result_3;
    return result_2;
}
```

U navedenom primjeru, varijable result_2 i result_3 su deklarirane kao lokalne varijable unutar funkcije. Međutim, obje varijable su deklarirane unutar dodatnog blok scope-a koji je ostvaren dodatnim vitičastim zagradama. Ukoliko se funkcija izvrši, linija

```
document.getElementById('proba7').innerHTML ="let van bock scope-a
"+result_3;
```

će rezultirati exception-om odnosno greškom, jer varijabla result_3 ne postoji van navedenog block scope-a.

3.4 Operatori

Kao i u drugim programskim jezicima, i u JavaScript jeziku koristimo pojmove operator, operan i izraz. Ukoliko je data sljedeća linija koda

```
rezultat=prviBroj+drugiBroj*(treceBroj/cetvrtiBroj);
```

cijelu linija predstavlja izraz, što je ekvivalent izraza u matematičkom smislu. Varijable rezultat, prviBroj, drugiBroj itd. predstavljaju operande, dok znaci =, +, * i / spadaju u operatore. Operatori su dakle, znaci koji određuju operaciju koja se vrši nad operandima, što u konačnici čini izraz.

Operatori se dijele prema tipu operatora i prema broju operanada. Prema tipu operatora dijele se na:

Tip operatora	Oznaka operatora	Opis
Aritmetički	+	sabiranje
	-	oduzimanje
	*	množenje
	/	dijeljenje
	%	ostatak od dijeljenja (modulo)
	++	inkrement (postfiks i prefiks)
	--	dekrement (postfiks i prefiks)
	**	stepenovanje
Dodjele	=	dodjeljivanje
	+=	dodjeljivanje sa sabiranjem
	-=	dodjeljivanje s oduzimanjem
	*=	dodjeljivanje s množenjem
	/=	dodjeljivanje s dijeljenjem
	%=	dodjeljivanje s modulo operatorom
Komparacije	==	jednako
	!=	nije jednako
	===	striktno jednako (provjerava se i tip podatka)
	!==	striktno nejednako
	>	veće
	<	manje
	>=	veće ili jednako
	<=	manje ili jednako
Logički	&&	logičko I
		logičko ILI
	!	logičko NE
Uvjetni	?:	uvjetni operator
	??	zamjenski operator null vrijednosti
Bitwise	&	bitwise I
		bitwise ILI

	~	bitwise NE
	^	ekskluzivno ILI (XOR)
	<<	shift u lijevo
	>>	shift u desno
	>>>	shift u desno bez predznaka
Grupiranja	()	Identično kao u matematici
i drugi koji se neće spominjati u ovom materijalu		

Naravno, postoje i dodatni operatori, koji na ovom uvodnom nivou neće biti obrađivani, kao što su operatori dodjele sa bitwise operatorima, „in“, „instanceof“, zarez u sekvenci itd. Neki od navedenih operatora su uključeni standardom ECMAScript 6 (ES6) 2015. godine i kasnije.

Osim podjele prema tipu operatora, oni se mogu djeliti i po broju operanada na koje se mogu primijeniti. Tako postoje:

Tip operatora po broju operanada	Primjer operatora
Unarni operatori, odnosno operatori koji se primijenjuju na jedan operand	++ --
Binarni operatori, odnosno operatori koji se primijenjuju na dva operanda	+ - * /
Trojni ili Ternarni operatori, odnosno operatori koji se primijenjuju na tri operanda	?:

Operator + se koristi i kao string operator. U slučajevima operacija na string vrijednostima, operator + predstavlja operator konkatencije, odnosno sjedinjavanja dva stringa. Međutim, ukoliko se operator + koristi u izrazima s mješovitim tipovima podataka, kao što su stringovi i numeričke vrijednosti, njegova uloga će ovisiti o redoslijedu operanada. Tako će npr. evaluacija izraza

```
var rezultat1 = "100" + 200 + 300;
```

rezultirati vrijednošću 100200300, jer će se cijeli izraz evaluirati kao konkatencija stringova, dok će evaluacija izraza

```
var rezultat2 = 100 + 200 + "300";
```

rezultirati vrijednošću 300300, jer će prvi + operator biti primijenjen kao operator sabiranja dok će drugi operator + biti primijenjen kao operator konkatencije stringova.

U skladu s pravilima u matematici i programskim jezicima, primijenjen je princip prioriteta operatora te asocijativnost operatora, kako bi bilo moguće predvidivo evaluirati izraz. U sljedećoj tabeli je dat pregled operatora s indikatorom nivoa prioriteta kao i asocijativnosti.

Prioritet	Operator	Oznaka	Asocijativnost
1	Grupiranje	()	
2	Inkrementi i dekrementi (prefiks)	++, --	unarni
3	Inkrementi i dekrementi (postfix)	++, --	unarni
3	Stepenovanje	**	R-L
4	Modulo	%	L-R
5	Aritmetički (množenje, dijeljenje)	*, /	L-R
6	Aritmetički (sabiranje, oduzimanje)	+, -	L-R
7	Komparatori	>, <, >=, <=...	L-R
7	Dodjela	=, +=, -=	R-L

Navedeni pregled je najosnovniji prikaz detaljnije liste operatora s priotitetima i njihovim asocijativnostima.⁶

3.5 Osnovne programske strukture

Osnovne programske strukture nisu specifičnost nekog programskog jezika. Programske strukture su poznate još od najranijih programskih jezika. Porastom kompleksnosti sisteskkih zahtjeva, pojavila se potreba za definisanjem teorije programiranja u formalno-akademsom smislu. U tom kontekstu se spominje i teorema strukturnog programiranja ili Böhm–Jacopini teorema koja se može, u pojednostavljenoj formi, svesti na to da se svaki konačno rješivi programski zadatak može uraditi korištenjem osnovnih programskih struktura:

- sekvence (uključujući sekvencijalno pozivanje subrutina)
- selekcije (grananja)
- iteracije (ponavljanja).

⁶ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_Precedence

Preduslov korištenja ovog materijala je poznavanje programskih osnova iz drugih programskih jezika kao što je C++, pa se ne ulazi u detalje osnova programiranja u ovom poglavlju.

Programska struktura sekvenca podrazumijeva sekvencijalno izvršenje programskih blokova ili instrukcija. Selekcija predstavlja programski strukturu izbora. U JavaScript koriste se standardne strukture izbora koje podrazumijevaju:

- IF statement
 - o if, if-else i if-else if-else
- switch i
- uvjetni operator ?:

Za razliku od drugih programskih jezika kao što je C++, switch statement u JavaScriptu može sadržavati default opciju i na početku switch statementa.

Programska struktura iteracija rezultira ponavljanjem određenog dijela koda. JavaScript ima dvije osnovne vrste iteracijskih petlji:

- FOR
 - o for
 - o for – in
 - o for – of
- WHILE
 - o while i
 - o do-while

Klasična for petlja se ne razlikuje od for petlje starijih programskih jezika koji nisu bili objektnoorijentirani. For-in petlja podrazumijeva iteraciju kroz obilježja objekta, dok for-of petlja podrazumijeva iteraciju kroz ponavljajuće objekte, odnosno nizove objekata.

3.6 JS funkcije

Funkcije su zasebni dijelovi programskog koda kojim se implementiraju algoritmi koji ulazne podatke preslikavaju u skup izlaznih podataka, implementirajući dio programske logike cjelokupnog programa u kojem su implementirane. Ti zasebni dijelovi koda čine neovisne programske cjeline koje se nazivaju i potprogrami, procedure, rutine, subrutine itd.

Funkcije su nezaobilazan koncept svakog novijeg programskog jezika. Zajedno s osnovnim programskim stukturama omogućavaju rješenje konačno rješivih programskih zadataka implementirajući koncept dekompozicije i usitnjavanja problema na podprobleme sve do nivoa dekompozicije koji omogućava da se algoritam podproblema može zapisati u formi linija programskog koda nekog programskog jezika.

JavaScript, kao i svi ostali programski jezici novije generacije, omogućava korištenje funkcija. Funkcije u JavaScript jeziku su jednostavnije od funkcija u drugim jezicima, kao što je C++,toliko koliko je jednostavniji pristup i koncept varijabli. Specifičnost JavaScript funkcija je u tome što su funkcije zapravo objekti ili „Funkcijski objekti“.

Funkcije u JavaScript nemaju zasebnu deklaraciju koja je odvojena od definicije. Piše se samo definicija funkcije, korištenjem ključne vrijednosti *function*. Sintaksa definisanja funkcije je:

```
function [nazivFunkcije]([param1[, param2[, ..., paramN]]) {  
    codeFunkcije...;  
}
```

JavaScript omogućava kreiranje neimenovanih ili anonimnih funkcija, koje su u formi funkcijskog izraza, što se može primijetiti u prethodnom pseudo-kodu gdje je naziv funkcije naveden u uglastim zagrada, čime se naglašava da je to opcionalni dio sintakse. U slučaju anonimnih funkcija, radi se o funkcijskom izrazu koji se od funkcijske definicije razlikuje samo u definiranju imena. Funkcijski izraz, odnosno neimenovana funkcija se koristi za tzv. IIFE (Immediately Invoked Function Expression) što označava odliku da se funkcija izvršava prilikom nailaska na izraz.⁷ Također, na funkcije definirane na tradicionalni način primjenjuje se tzv. hoisting, koji je objašnjen u poglavlju *Varijable*, dok se za anonimne funkcije ne primjenjuje hoisting.

Funkcijski literal i funkcijske varijable su dva dodatna načina identificiranja funkcije. U slučaju funkcijskog literala, radi se o imenu za koje se veže neimenovana funkcija kao npr.

```
imeFunkcije=function() {  
    alert("poruka");  
}
```

⁷ <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/function>

U slučaju funkcijske varijable, sama varijabla se može koristiti kako bi se pozvalo izvršenje funkcije. Npr. kod

```
function imeFunkcije() {  
    alert("poruka");  
}  
let zamjenskoIme=imeFunkcije;
```

će rezultirati time da se varijabla zamjensko ime koristi umjesto imena funkcije.

Za razliku od C++ jezika u kojem se, prilikom deklariranja i definisanja funkcije, mora navesti tip povratne vrijednosti, JavaScript omogućava da se tip vrijednosti ne navodi u zaglavlju. Jedina razlika između funkcije sa i bez povratne vrijednosti je u postojanju return statementa.

```
function sabiranje(value1, value2){  
    result_1 = value1+value2; //globalna varijabla  
    let result_2 = value1+value2; //lokalna varijabla  
    return result_2;  
}
```

U navedenom primjeru radi se o funkciji koja vraća vrijednost. Za razliku od drugih programskih jezika, kao što je C++, u JavaScript definiciji funkcije nije naveden tip povratne vrijednosti. Tip povratne vrijednosti ovisi o tipu povratne vrijednosti varijable u return statementu.

U sljedećem primjeru data je funkcija koja ne vraća vrijednost.

```
function sabiranjeBezPovratne(value1, value2){  
    result_1 = value1+value2; //globalna varijabla  
    let result_2 = value1+value2; //lokalna varijabla  
    alert(result_2);  
}
```

Kao što se može vidjeti iz navedenog primjera, jedina razlika između funkcija sa i bez povratne vrijednosti je u return statementu.

Funkcije se pozivaju na isti način kako se to radi i u ostalim programskim jezicima, navođenjem imena funkcije i proslijeđivanjem argumenata navedenih u zagradama. Npr. kod

```
<button onclick="document.getElementById('proba').innerHTML
=sabiranje(12,25) ">Sabiranje</button>
```

će rezultirati da se izvršenje funkcije sabiranje veže za klik na dugme. Klikom na dugme pozvat će se funkcija sabiranje kojoj će biti proslijeđena dva argumenta numeričke vrijednosti 12 i 25, te će se povratna vrijednost nakon izvršenja funkcije upisati u HTML element čiji je ID „proba“.

Od ECMAScript 6 (ES6) u JavaScript je uveden koncept *arrow* funkcija. Radi se o kraćoj notaciji funkcije bez korištenja ključne riječi *function*. Umjesto toga, koristi se *arrow* znak, koji se ponekad naziva i fat *arrow*. Primjer *arrow* funkcije dat je u sljedećem kodu

```
racunanje = (c,d)=>{
    let f=c**2+d**2;
    return f;
}
```

Naziv funkcije je računanje i funkcija prima dva argumenta c i d. umjesto ključne riječi *function*, korišten je *arrow* znak između liste argumenata i vitičaste zgrade koja označava početak funkcije. Naziv funkcije je ekvivalent funkcijskom literalu.

Ukoliko se radi o jednostavnijoj funkciji, moguće je dodatno skratiti notaciju, pa se mogu izostaviti vitičaste zgrade, tako da se funkcija može zapisati u obliku

```
racunanje = (a,b) => a**2+b**2;
```

U navedenom primjeru radi se o funkciji koja prima dva parametra a i b, te vraća zbir kvadrata dva primljena argumenta.

U slučaju još jednostavnijih funkcija, moguće je i dalje skratiti zapis funkcije kako je ilustrirano na sljedećem primjeru:

```
racunanje = a => a+5;
```

U navedenom primjeru radi se o funkciji koja prima samo jedan parametar, te vraća vrijednost primljenog argumenta uvećanu za 5. U ovom primjeru izbjegnuto je pisanje zagrada za listu argumenata, te vitičastih zagrada koje označavaju početak i kraj funkcije.

JavaScript podržava i rekurzivne pozive funkcije kao npr.

```
function rekurzivna (n) {  
    if (n <= 1) {    //bazni slučaj - prekid rekurzije  
        return 1;  
    }  
    return n * rekurzivna(n-1); //rekurzivni poziv  
}
```

U navedenom primjeru radi se o funkciji koja računa faktorijel datog broja n.

Naravno, kao i u ostalim programskim jezicima, potrebno je voditi računa o beskonačnim petljama u iteracijama i o *stack overflow* u rekurzijama, što se može izazvati nepostojanjem ili nepravilno osmišljenim baznim slučajem.

3.7 JavaScript i DOM

Jedna od najpoznatijih mogućnosti i najveći razlog korištenja JavaScript jezika je manipuliranje *Document Object Modelom* (DOM). DOM koncept je originalno vezan za HTML dokument i objašnjen je u poglavlju 1.2 *Struktura HTML dokumenta – DOM*. Ukoliko bi se definisao DOM, može se reći da je DOM „Document Object Model je jezički i platformski neovisna struktura/interface koji omogućava programima i skriptama da dinamički pristupe i izmjene/update sadržaj, strukturu i stil HTML dokumenta ili nekog njegovog dijela“.

JavaScript kodom se stoga može pristupiti HTML dokumentu kako bi se manipulirano njegovim sadržajem. Moglo bi se razumijeti kako JavaScript „vidi“ HTML dokument kao DOM strukturu. Zbog toga Javascript preko DOM može:

- promijeniti sve HTML elemente na stranici
- promijeniti sve HTML attribute svih elemenata
- dodati nove elemente i attribute na stranicu
- ukloniti elemente sa stranice
- promijeniti sve CSS stilove
- reagirati na sve HTML evente/događaje
- kreirati nove HTML evente/događaje.

Postoji više standarda DOM-a:

- Core DOM – standardni model za sve dokumente
- XML DOM – standardni model za XML

- HTML DOM – standardni model za HTML.

Naravno, u ovom materijalu se obrađuje samo HTML DOM.

Sa stanovišta JavaScript jezika, kompletan HTML dokument se može posmatrati kao objekat, kao i svi njegovi elementi. Zbog toga, način pristupanja HTML elementima se odvija preko:

- HTML DOM Metoda – akcija koje se izvršavaju na elementima
- HTML DOM Properties – obilježja HTML elemenata koji se postavljaju ili mijenjaju.

Dakle, metode i obilježja svakog objekta predstavljaju programsko sučelje preko kojih skripte pristupaju elementima.

Osim sadržaja samog dokumenta, JavaScript jezikom se može manipulirati čak i prozorom preglednika, tako da se mogu otvarati novi prozori preglednika, otvarati novi tab-ovi unutar preglednika, te se može manipulirati veličinom i pozicijom prozora preglednika na ekranu uređaja na kojem se prikazuje stranica.

Načelna podjela načina pristupanja JavaScript koda HTML elementima je:

- metode za adresiranje elemenata
- metode za izmjenu elemenata
- kreiranje i uklanjanje elemenata
- korištenje event-a odnosno događaja određene grupe HTML elemenata koji mogu reagovati korisničku ili neku drugu akciju.

Metode za adresiranje elemenata se koriste kako bi se dohvatio, odnosno izabrao, element na kojem će se vršiti određena operacija.

Najjednostavniji primjer manipulacije elementima DOM-a je modificiranje sadržaja jednog HTML elementa. To se može uraditi adresiranjem odnosno dohvaćanjem HTML elementa po njegovom ID, te korištenjem obilježja „innerHTML“ kako bi se promijenio sadržaj elementa kao npr.:

```
document.getElementById("poruka").innerHTML="Dobrodošli";
```

U ovoj liniji koda adresira se element čiji je ID = „poruka“ tako što se navede `document.getElementById("poruka")`., te mu se, putem obilježja `innerHTML` mijenja sadržaj u tekst „Dobrodošli“. Ukoliko se želi zadržati postojeći sadržaj, te

dodati još neki tekst to se može uraditi korištenjem operatora += koji se u slučajevima string vrijednosti koristi za konkatenciju stringova.

Ukoliko bi se detaljnije analizirala sintaksa, može se primijetiti da se koristi metoda getElementById objekta dokument koji se smatra globalnim objektom. Na ovaj način se može i razumijeti cijeli DOM kao *tree* struktura, odnosno struktura drveta u čijem je korijenu dokument objekt kako je prikazano na ilustraciji 5 u poglavlju 1.2.

Načini adresiranja, odnosno dohvaćanja elemenata na DOM-U su:

- dohvaćanje po imenu - document.getElementsByName(„ime“)
- dohvaćanje po nazivu klase - document.getElementsByClassName(„ime“)
- dohvaćanje po imenu taga - document.getElementsByTagName(„ime“)
- dohvaćanje po ID elementa - document.getElementById(„ime“)
- pretraga elemenata po kriteriju - document.querySelector(„.dno“).

Budući da se preporučuje da na jednoj stranici ID elementa bude jedinstven, za očekivati je da adresiranje elementa po ID rezultira dohvaćanjem samo jednog elementa. Međutim, ukoliko se elementi dohvaćaju po nekom drugom kriteriju za koji se ne očekuje da bude jedinstven na stranici, kao npr. class, dohvaćanje elemenata po tom kriteriju obično rezultira dohvaćanjem više od jednog elementa. U tom slučaju radi se o nizu elemenata u kojem, kako bi se pristupilo svakom pojedinom elementu, se koristi indeksiranje. Pretpostavimo da se na stranici nalaze tri HTML input elementa tipa text, čiji je atribut name=„godina“. Komanda document.getElementsByName(„godina“) će adresirati niz od tri HTML elementa kojima se može pristupiti na sljedeći način:

```
let godine=document.getElementsByName("godina");
for (let i = 0; i<godine.length;i++){
    godine[i].value=i;
}
```

ili koristeći for-of inačicu petlje:

```
let i=0;
for (godine of document.getElementsByName("godina")){
    godine.value=i++;
}
```

Kako je navedeno, DOM komande se mogu koristiti i za manipuliranje ostalim atributima elementa. Tako se npr. mogu adresiranom elementu promijeniti ostale osobine kao npr. boja slova, boja pozadine itd.

Sljedeća linija koda dohvaća element čija je klasa „dno“ i postavlja boju pozadine na crvenu:

```
document.querySelector(".dno").style.backgroundColor = "red";
```

koristeći obilježje `style.background`.

Linija koda

```
document.querySelector("#proba").setAttribute("style", "background-color:blue");
```

će rezultirati postavljanjem plave boje pozadine elementa čiji je ID=“proba“, ali putem metode *setAttribute*. U ovom slučaju radi se o atributu „style“. Metoda *setAttribute* može biti korištena za postavljanje bilo kojeg atributa HTML elementa kao npr „class“ kao u sljedećem primjeru:

```
document.querySelector("#proba1").setAttribute("class", "glavaDonji2");
```

Ukoliko je potrebno ukloniti atribut s određenog elementa, to se može uraditi korištenjem metode *removeAttribute()* na isti način kako je atribut i dodan.

Manipulacija DOM-om podrazumijeva kreiranje i uklanjanje HTML elemenata na stranici. Jednostavni primjer kreiranja HTML elementa, na način koji nije preporučen i ne smatra se dobrom praksom, prikazan je u sljedećem kodu.

```
document.getElementById("poruka").innerHTML="<h2>dobrodošli</h2>";
```

U navedenom primjeru, dohvaća se HTML element čiji je ID=“poruka“, te mu se upisuje sadržaj koji ima HTML tag oznake. To će, poslije upisivanja sadržaja, na pregledniku biti protumačeno kao novi tag. Naravno, više nego očito je da se ne radi o dobrom načinu kreiranja HTML elemenata jer se, između ostalog, lako može pogriješiti u pisanju HTML koda u formi stringa.

Umjesto toga, preporučeno je dodavati elemente metodama `createElement` i `appendChild` kako je demonstrirano u sljedećem primjeru:

```
let selekcija=document.getElementById("poruka");
let noviElement=document.createElement("h2");
selekcija.appendChild(noviElement);
noviElement.innerHTML="Kreiran je novi element";
```

U navedenom primjeru, u prvoj liniji koda, varijabla `selekcija` se referira na HTML element čiji je ID="poruka". Zatim se u sljedećoj liniji kreira novi HTML element tipa H2. Bitno je razumijeti da se u ovoj liniji koda, kreirani element još uvijek ne pojavljuje na stranici, što je moguće vidjeti korištenjem debug režima u *source tabu inspect* element prozora preglednika Chrome. Tek trećoj liniji koda se kreirani element i dodaje na stranicu kao *child* element ranije adresiranog elementa, dok se u četvrtoj liniji koda dodaje sadržaj upravo kreiranog elementa.

Element se može ukloniti korištenjem metode `removeChild` na isti način kako se element i dodaje. Metode za manipulacijom elementima su:

- `document.createElement()`
- `document.appendChild()`
- `document.removeChild()`
- `document.replaceChild()`.

Manipuliranjem stilovima moguće je i pozicioniranje elementa na stranici bilo primjenom nekog već definisanog CSS stila bilo dodavanjem nekog atributa stila kao npr.

```
document.getElementById("mojEl").style.top =
Math.random()*300+1+"px";
document.getElementById("mojEl ").style.right =
Math.random()*500+1+"px";
```

Izvršenje navedenog koda će rezultirati postavljanjem elementa na stranici korištenjem slučajnih vrijednosti koje se generišu metodom `random()` objekta `Math`.

Metode za modificiranje elementa su:

- `element.innerHTML`
- `element.stil.vrijednost`

- element.**setAttribute** (naziv,vrijednost)
- element.**value**.

JavaScript DOM metodama je moguće modificirati, dodati i ukloniti sve elementa na HTML stranici.

3.8 Dijaloški prozori

Osim standardne komunikacije s korisnikom putem tekstualnih input polja, kako je to objašnjeno u poglavlju koje govori o HTML formama, korištenja dugmadi, te korištenja ostalih HTML elemenata putem promjene njihovog sadržaja, JavaScript ima ugrađena tri dijaloška prozora putem kojih se pruža dodatna mogućnost komunikacije s korisnikom stranice.

Dijaloški prozori su:

Naziv dijaloškog prozora	Primjer sintakse	Opis
Alert	<code>alert("Poruka korisniku");</code>	Dijaloški prozor kojim se prikazuje poruka korisniku
Confirm	<code>confirm("Odgovorite na pitanje?");</code>	Dijaloški prozor kojim se upućuje pitanje korisniku na koje se može odgovoriti s OK i Cancel, što rezultira vraćanjem true ili false vrijednosti u program
Prompt	<code>prompt("Kako se zovete?", "Ovdje upiši svoje ime");</code>	Dijaloški prozor kojim se daje mogućnost korisniku upisivanja neke vrijednosti u tekstualno polje na tom prozoru. U ovom slučaju, u program se vraća vrijednost koju je korisnik unio u tekstualno polje.

Na sljedećem primjeru demonstrirani su načini korištenja navedenih dijaloških prozora.

```
<head>
  <script>
    primjerAlert=()=>alert("Poruka korisniku");
```

```

        primjerConfirm={()=>document.getElementById("poruka").innerHTML=confirm("Odgovorite na pitanje?")};
        primjerPrompt={()=>document.getElementById("poruka").innerHTML="Dobrodošli "+prompt("Kako se zovete?", "Ovdje upiši svoje ime")};
    </script>
</head>
<body>
    <button onclick="primjerAlert()">Klikni za alert</button>
    <button onclick="primjerConfirm()">Klikni za
confirm</button>
    <button onclick="primjerPrompt()">Klikni za prompt</button>
    <h1 id="poruka"></h1>
</body>

```

U primjeru su korištene tzv. *arrow* funkcije koje su pojašnjene u prethodnom poglavlju. Naravno, funkcija `alert` se može direktno pozvati pa nije bilo neophodno kreirati zasebnu funkciju `primjerAlert`. Klikom na svako od tri dugmeta definirana u tijelu (engl. `body`) sekcije izvršit će se odgovarajuća funkcija koja će demonstrirati funkcionalnost dijaloških prozora. Ukoliko se žele poslati podaci na web servis kako bi se izvršila akcija unosa ili ažuriranja podataka, također se koristi *fetch* na sljedeći način:

```

snimi = (podaci) => {
    let adresa = 'api/Osobe/unosPodataka';
    var strJson = JSON.stringify(podaci);
    fetch(adresa, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: strJson
    })
    .then(
        (response) => {
            if (response.status < 200 || response.status > 300)
            {
                console.log('Geska: ' +
                    response.status);
                return;
            }
        }
    )
}

```

```
        response.json().then((data) => {
            console.log(data);
            window.location="prikaz.html";
        });
    }
)
.catch((err) => {
    console.log('Fetch Greska :-S', err);
});
}
```

Može se primijetiti da se podaci u ovom primjeru šalju, također, u JSON formatu, pasu podržani i ostali formati podataka. I u slučaju slanja podataka, prima se response s web servera, te se na osnovu njegovog statusa tumači da li su podaci uspješno procesirani od strane web servisa.

3.9 Objekti

Obične varijable se deklariraju upotrebom riječi `var` ili `let`, nazivaju se i *JavaScript Primitives* kao što su:

- string
- number
- boolean
- null
- undefined.

Za razliku od njih, kompleksni tipovi podataka ili objekti su također varijable, objekti mogu sadržavati više vrijednosti (property) i funkcija (methods). Objekti ili kompleksni tipovi podataka u JavaScript su nezaobilazan dio koji je potrebno razumijeti kako bi se razumio JavaScript jezik. Naime, JavaScript manipulira svim elementima HTML stranice kao objektima, funkcije u JavaScript se ponašaju kao objekti, nizovi u JavaScript su objekti itd.

Kompletan HTML dokument je objekat formiran u hijerarhijsku strukturu drveta (DOM). Zapravo, puna sintaksa dijaloškog prozora alert bi bila:

```
window.alert("Ovo je alert");
```

što znači da je alert metoda *window* objekta kao i u slučaju `console.log` komande koja

omogućava prikazivanje neke poruke ili podataka u konzolnom djelu web preglednika. Puna sintaksa bi bila:

```
window.console.log(window.screen);
```

U navedenom primjeru, u konzolnom prozoru će se ispisati informacije o screen-u kao npr:

```
Screen {availWidth: 1366, availHeight: 728, width: 1366, height: 768, colorDepth: 24, ...}
```

To omogućava programerima da pomoću JavaScript jezika kreiraju skripte koje će prilikom izvršenja moći čak i kreirati potpuno novi prozor preglednika u kojem će moći učitati neku stranicu kao npr.:

```
novi=window.open("http://www.unmo.ba", '', 'width=400, height=300');
```

ili pozicionirati prozor na neku lokaciju kao npr.:

```
novi.moveTo(50, 60);
```

te mu promijeniti dimenzije kao npr.:

```
novi.resizeTo(600, 400);
```

Kao što se može vidjeti, JavaScript-om se može ostvariti puna kontrola kako nad HTML dokumentom, tako i nad prozorom web preglednika.

Naravno, najinteresantniji dio za programere je kreiranje objekata kao kompleksnih tipova podataka. U JavaScript jeziku je moguće kreirati korisnički definisane objekte, kao i kod ostalih objektno-orijentiranih jezika. JavaScript ima predefinirane objekte kao što su:

- Array,
- Date,
- Math
- RegExp itd.

Ukoliko programer želi kreirati tzv. "custom" objekte za potrebe aplikacije, potrebno je pravilno definisati tzv. apstrakciju objekta. Taj koncept se smatra jednim od četiri

najznačajnija koncepta u objektno-orijentiranom programiranju. Ostali koncepti su polimorfizam, enkapsulacija i naslijeđivanje, koji se ne obrađuju u ovom materijalu. Apstrakcija kao koncept nije direktno vezana za sam proces programiranja, niti za neki poseban programski jezik. Proces apstrakcije predstavlja identificiranje obilježja i metoda objekta iz realnog svijeta koji su relevantni za aplikaciju. Nedostatak vještine u procesu apstrakcije dovodi do razvoja aplikacije s nepotrebnim elementima, te teže implementacije i održavanja.

Načelno, objekat se može kreirati na sljedeće načine:

- korištenjem objektnog literala
- korištenjem ključne riječi *new*
- definisanjem objektnog konstruktora.

Korištenje objektnog literala je slično korištenju funkcijskog literala. Primjer objekta kreiranog objektnim literalom je:

```
let automobil = {boja:"Crna", gorivo:"Benzin", godina:"2015",  
vrata:"5 vrata", puntika:"1+4 puntika"};
```

Može se primijetiti da se u ovom načinu kreiranja objekta koristi JSON notacija.

JSON notacija predstavlja standard za razmjenu podataka. JSON je skraćenica za *JavaScript Object Notation*. Koristi se intenzivno kako za razmjenu podataka, tako i kod kreiranja objekata. JSON postaje mnogo češće korištena notacija od npr. XML, mada se i XML još uvijek često koristi. Više informacija o JSON notaciji koja je standardizirana ECMA-262 standardom može se naći na službenoj stranici ECMA International⁸. JavaScript ima i tzv. JSON objekat koji ima svoje dvije glavne metode *parse* i *stringify*. Za potrebe razvoja razlikujemo JSON objekat i JSON niz objekata. Sintaksa JSON objekta je data u prethodnom primjeru, gdje se mogu primijetiti parovi obilježje:vrijednost koji su odvojeni zarezom i ovičeni vitičastim zagradaama.

U primjeru je dat objekat čiji je naziv *automobil*. Obilježjima navedenog objekta se može pristupiti na sljedeće načine:

```
document.getElementById("godina").innerHTML=automobil.godina;
```

⁸ https://www.ecma-international.org/wp-content/uploads/ECMA-262_12th_edition_june_2021.pdf

ili

```
document.getElementById("godina").innerHTML=automobil["godina"];
```

Preporučeni način pristupanja obilježjima je korištenjem tačke između naziva objekta i njegovog obilježja, mada se mogu koristiti i uglaste zagrade s obilježjem navedenim pod navodnim znacima.

Drugi način kreiranja objekta je korištenjem ključne riječi *new* kao što je prikazano u sljedećem primjeru:

```
var automobil2 = new Object();
automobil2.boja="Plava";
automobil2.gorivo="Dizel"
automobil2.godina="2016";
```

Treći način je korištenjem objektnog konstruktora kao npr.:

```
function objekatAutomobil(boja,gorivo,godina){
    this.gorivo = gorivo;
    this.boja = boja;
    this.godina = godina;
}
```

Od ECMAScript 6 iz 2015. godine, u JavaScript je uključena i ključna riječ *class*, te se objekat može kreirati i na način sličan kako se to radi i u C++ jeziku. Naravno, sintaksa C++ jezika je mnogo striktnija, a mogućnosti klasa u ovom smislu mnogo naprednije.

Objekti mogu imati i svoje metode, odnosno ugrađene funkcije koje se odnose na dati objekat. Tako npr.

```
let automobil = {
    boja:"Crna",
    gorivo:"Benzin",
    godina:"2015",
    vrata:"5 vrata",
    puntika:"1+4 puntika",
    potrosnjaNa100km:8,
    trenutnoPotroseno: function(predjeniPut){
        return predjeniPut*this.potrosnjaNa100km/100;
```

```
}  
}
```

Kao što se vidi u navedenom primjeru, objekat automobil, pored obilježja kao što su boja, gorivo itd., ima i svoju metodu trenutnoPotroseno, kojoj se može prisupiti na isti način kako se pristupa obilježjima, npr.:

```
document.getElementById("potr").innerHTML=automobil.trenutnoPotroseno(625);
```

U ovoj liniji koda, u HTML element čiji je ID="potr" upisuje se vrijednost koju vrati metoda trenutnoPotroseno objekta automobil.

Posebno interesantna činjenica programerima koji dolaze iz oblasti tradicionalnih programskih jezika, kao što je C++, je mogućnost JavaScript-a da objektima može dodati obilježje, čak i poslije kreiranja objekta. Dovoljno je da se pozove novo obilježje i ono će biti kreirano kao obilježje datog objekta. Npr. ukoliko u objektu automobil koji sadrži obilježja kao u prethodnom primjeru ne postoji obilježje „stanje“, bit će dovoljno napisati liniju koda

```
automobil.stanje="Izvršno";
```

te će se objektu automobil dodati obilježje „stanje“ čija će vrijednost biti string „Izvršno“.

Primjeri korištenja ugrađenog datumskog, Math kao i String objekata dati su u sljedećem kodu:

```
//datumi  
var datum = new Date();  
document.getElementById("proba1").innerHTML= datum; //primjer get metode  
document.getElementById("proba2").innerHTML= datum.getDate();  
//metoda koja vraća dan u mjesecu  
document.getElementById("proba3").innerHTML= datum.getDay();  
//metoda koja vraća dan u sedmici  
datum.setMonth(9); //primjer set metode  
document.getElementById("proba4").innerHTML= datum;  
//ili inicijalizacija prilikom deklaracije  
var rođendan = new Date("1973/04/14");
```

```

document.getElementById("proba4").innerHTML=rodjendan;

//math
document.getElementById("proba5").innerHTML=Math.PI;
document.getElementById("proba6").innerHTML=Math.round(Math.PI);
document.getElementById("proba7").innerHTML=Math.sqrt(144);

//parse
document.getElementById("proba8").innerHTML=parseInt(document.getElementById("kilometara").value);

//string
var brUgovora = "IPA/2017/123-456";
document.getElementById("proba9").innerHTML=brUgovora;
document.getElementById("proba9").innerHTML+=" Budget instrument: " + brUgovora.substring(0,3); //substring(pocetak,kraj)
document.getElementById("proba9").innerHTML+=" Godina: " + brUgovora.substr(4,4); //substr(pocetak,duzina)
document.getElementById("proba9").innerHTML+=" Ugovor ID: " + brUgovora.slice(-7); //slice moze koristiti sa pozitivnom i negativnom vrijednoscu, odnosno od pocetka ili od kraja stringa
//ili
var kratkiBroj = brUgovora.slice(-7);
kratkiBroj = kratkiBroj.replace("-", ""); //mijenjanje dijela stringa
document.getElementById("proba9").innerHTML+=" Kratki ID: " + kratkiBroj;

```

Nizovi u JavaScript su također objekti. Specifičnost JavaScript nizova je u tome što članovi niza ne moraju biti istoga tipa, kao npr. u C++ jeziku. Elementi niza mogu biti varijable, ali i kompletni objekti. Nizovi se mogu deklarirati i inicijalizirati na dva načina:

- klasični
 - `mojNiz = new Array();`
 - ili sa inicijalizacijom
 - `mojNiz = new Array(vrijednost1, vrijednost2, vrijednost2, ...);`
- JSON
 - `mojNiz = [vrijednost1, vrijednost2, vrijednost2, ...];` //best practice.

Preporučeni način kreiranja niza je korištenjem JSON array notacije.

Niz kao objekat ima svoje metode koje omogućavaju lakše upravljanje nizom kao što su:

Metoda	Opis
.sort()	metoda za sortiranje niza koja može kao argument primiti neimenovanu funkciju kojom se određuje da li će niz biti sortiran uzlazno ili silazno
. push()	metoda koja primljeni argument dodaje kao zadnji član niza
. pop()	metoda koja uklanja zadnji član niza
.indexOf()	metoda za pretragu koja prima kao argument kriterij pretrage te vraća indekspronađenog člana niza
.splice	metoda koja uklanja elemente niza prema proslijeđenim argumentima od kojeg do kojeg člana niza treba ukloniti, te može vratiti uklonjene članove u drugi niz
i mnoge druge	

Primjer korištenja metoda za manipuliranje nizom dat je u sljedećem kodu:

```
let niz = [45,12,128,64,32,12,15,19,0,8,-1,-19,180,1];
for (let i=0;i<niz.length;i++){ //obratite paznju na niz.length
    koji vraca duzinu niza
    document.getElementById("proba1").innerHTML+=niz[i]+", ";
}
//ili
document.getElementById("proba2").innerHTML = niz;
niz.sort(); //po defaultu sortira string
document.getElementById("proba3").innerHTML=niz;
niz.sort((a, b)=>{return a-b});
document.getElementById("proba4").innerHTML=niz;
niz.sort((a, b)=>{return b-a}); //silazni redoslijed
document.getElementById("proba5").innerHTML=niz;
document.getElementById("proba6").innerHTML=typeof(niz); //niz je
objekat
for (i=0;i<niz.length;i++){
    document.getElementById("proba7").innerHTML+=typeof(niz[i])+",
";
}
niz.push("FIT"); //dodavanje clana niza - moze biti drugog tipa
podataka
```

```

niz.push(Date());
document.getElementById("proba8").innerHTML= niz;
    for (i=0;i<niz.length;i++){
        document.getElementById("proba9").innerHTML+=typeof(niz[i])+",
";
    }

var niz2 = niz.splice(3,7); //izbacivanje clanova niza
niz.pop(); //uklanja zadnji clan
document.getElementById("proba10").innerHTML= niz;
document.getElementById("proba11").innerHTML = niz.indexOf("FIT");
document.getElementById("proba12").innerHTML = niz2;

```

3.10 HTTP request

Jedna od najvažnijih odlika JavaScript jezika je mogućnost komuniciranja s web serverom. JavaScript kao *client-side scripting jezik* može pristupiti web servisima i izvršiti CRUD operacije putem tih web servisa. Ne ulazeći u server stranu, s klijentske strane dva najpoznatija načina komuniciranja s web servisima su:

- putem XMLHttpRequest
- koristeći Fetch.

Zbog razumijevanja ovih načina interakcije s web serverima, potrebno je razumijeti i pojam AJAX koji se usko veže za ovakve zahtjeve (engl.request). AJAX je dosta star pojam u JavaScript-u i skraćenica je za *Asynchronous JavaScript and XML*. AJAX nije neki posebni programski jezik, niti je neka posebna tehnologija, metoda ili razvojni okvir. AJAX-om se smatra skup JavaScript mogućnosti koje koriste ugrađene mogućnosti web preglednika i DOM koncepta čime omogućavaju programerima da:

- Asinhrono šalju i primaju podatke, odnosno ostvaruju interakciju sa web serverom.
- Dinamički osvježavaju i ažuriraju (engl.update)dijelove web-stranice bez ponovnog učitavanja kompletne stranice, čime omogućavaju tzv. *Single Page Application* itd.

XMLHttpRequest je stariji način i u mnogim aspektima se smatra prevaziđenim. Bez obzira što je u imenu metode sadržan XML, danas se većina klijent-server komunikacije u smislu razmjene podataka odvija u JSON formatu. XMLHttpRequest ima sljedeću najosnovniju sintaksu:

```

var zahtjev = new XMLHttpRequest();
zahtjev.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        //Odgovor sa web servisa se naazi u zahtjev.responseText;
    }
};
zahtjev.open("GET", "http..... adresa", true);
zahtjev.send();

```

Budući da se radi o zastarjelom načinu komuniciranja i interakcije sa serverom, koristi se Fetch API interface. Fetch je uvršten u ECMAScript 6 2015. godine. Fetch podržava većina web preglednika izuzev Internet Explorera. Fetch je kasnije podržan od strane preglednika Edge.

Nekoliko je prednosti *Fetch* metode u odnosu na XMLHttpRequest od kojih je najpoznatija činjenica da se *fetch* bazira na tzv. promises u JavaScript. Puna sintaksa fetch metode je⁹:

```

fetch(url, {
    method: 'POST', // *GET, POST, PUT, DELETE, itd.
    mode: 'cors', // no-cors, *cors, same-origin
    cache: 'no-cache', // *default, no-cache, reload, force-cache,
only-if-cached
    credentials: 'same-origin', // include, *same-origin, omit
    headers: {
        'Content-Type': 'application/json'
        // 'Content-Type': 'application/x-www-form-urlencoded',
    },
    redirect: 'follow', // manual, *follow, error
    referrerPolicy: 'no-referrer', // no-referrer, *no-referrer-
when-downgrade, origin, origin-when-cross-origin, same-origin,
strict-origin, strict-origin-when-cross-origin, unsafe-url
    body: JSON.stringify(data) // body data type must match
"Content-Type" header
});

```

Primjer korištenja *fetch* metode dat je u sljedećem kodu:

⁹ https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

```

podaci=()=>{

fetch('http://onlineshop.wrd.app.fit.ba/api/ispit20190914/Narudzba/GetProizvodiAll')
    .then(
        (response) =>{
            if (response.status !== 200) {
                console.log('Geska: ' +
                    response.status);
                return;
            }
            response.json().then((podat)=> {
                console.log(podat);
                sviProizvodi=podat;
                popuniPodatke(podat);
            });
        }
    )
    .catch((err) =>{
        console.log('Fetch Greska :-S', err);
    });
}

```

U navedenom primjeru *fetch* metoda prima argument URL web servisa koji vraća podatke, te ih prima u sekciji *.then* u varijablu *response*, koja predstavlja argument neimenovane funkcije. Zatim se ispituje status *response* odnosno status odgovora web servisa, te ako je status neispravan obradi (engl.handle) se greška, dok s, ako je status jednak 200, podaci se pretvaraju u JSON format te se mogu koristiti za ispis na stranici ili u neke druge svrhe.

3.11 Greške - exception

Prilikom procesa razvoja bilo kakve aplikacije u bilo kojem razvojnom okruženju, koristeći bilo koji programski jezik, taj proces nije imun na greške. U većini programskih jezika postoje opcije upravljanja greškama. U JavaScript se, između ostalog, koristi *try-catch*. Kad se greška desi, izvršenje JavaScript koda se zaustavlja i generiše se „poruka greške“. Na stranici se najčešće ne ispiše nikakva poruka, te se često desi da korisnik vidi šta se dešava. U slučaju generiranja greške, kaže se da je JavaScript „throw an exception“, odnosno u prijevodu „izbacio grešku“. Zapravo, JavaScript kreira objekat s imenom i porukom greške. Taj objekat treba „uhvatiti“

(engl. catch) da bi se vidjela greška. *Try-catch* se može koristiti i za otklanjanje grešaka (engl. debug) kao i za realizaciju određenih logičkih zahtjeva preko namjernih throw akcija.

Sintaksa *try-catch* mehanizma se sastoji od dva dijela:

```
try {  
}  
catch (varijabla)  
{  
}
```

U *try* sekciji se upisuje kod čije izvršenje se želi testirati dok se u *catch* sekciji u varijablu „hvata“ odnosno upisuje objekat grške ukoliko je ona generirana, odnosno ukoliko se desio „throw and exeption“. Zatim se u uglastim zagradama *catch* sekcije upisuje kod koji se treba izvršiti ukoliko se desi greška. Objekat greške ima obilježja name i message u kojima je smješteno ime i poruka greške.

```
try {  
    for(let i=0;i<varijablaKojaNePostoji;i++){  
        console.log(i);  
    }  
}  
catch (greska)  
{  
    alert(greska);  
    alert("Naziv greske: "+greska.name);  
    alert("Poruka: "+greska.message);  
}
```

U ovom slučaju, neće se ispisati poruka u konzolnom prozoru jer je greška kontrolisana odnosno obrađena (engl. handle).

Try-catch može biti korišten i za druge potrebe kao npr. kontrolu unosa vrijednosti od strane korisnika kao u sljedećem primjeru:

```
let unos;  
unos=prompt("Unesi svoj prosjek","Prosjek?");  
try {  
    if(unos == "") throw "Prazan unos";  
    if(isNaN(unos)) throw "Unesena vrijednost nije numerička";
```

```

        if (unos>10||unos<5){
            throw "Nije moguće";
        }
        else if(unos > 8){
            throw "Dobro!";
        }
        else if(unos > 6){
            throw "Nije lose!";
        }
        else{
            throw "Moze i bolje!";
        }

    } catch (varijabla) {
        alert("Unos rezultira sa: " + varijabla);
    }
    finally{
        document.getElementById("proba3").innerHTML=unos;
    }
}

```

Try-catch sekcije rade na isti način kao i u prethodnom primjeru, s tim da se u varijablu upisuje vrijednost koja je namjenski poslana iz *try* sekcije pomoću ključne riječi *throw*. Navedeni primjer sadrži i sekciju *finally* koja se u svakom slučaju izvršava neovisno o izvršenju gornjeg dijela koda. Ta sekcija se može koristiti za dodatne akcije.

3.12 Event listener

Većina HTML elemenata ima obilježja koja se odnose na događaje koji se mogu desiti i na koje se može reagirati nekim JavaScript kodom, odnosno nekom akcijom. Postoje tri osnovna načina dodavanja takvih akcija na HTML elemente, odnosno uvezivanja eventa HTML elementa s odgovarajućom akcijom. Ti načini su:

- preko obilježja HTML elementa u HTML tagu, kao što je npr. obilježje „onclick“
- preko obilježja HTML objekta
- dodavanjem *Event listener-a*.

U sljedećem primjeru mogu se vidjeti sva tri načina. Dat je JavaScript kod:

```

dodajListener=(()=>{

```

```

document.getElementById("prviDiv").addEventListener("wheel",()=>{alert("tockic")});

document.getElementById("prviDiv").addEventListener("click",()=>{document.getElementById("drugi").innerHTML="15";});

window.addEventListener("resize",()=>{document.getElementById("prviDiv").innerHTML=window.innerWidth})

document.getElementById("sve").addEventListener("mousemove",()=>{

document.getElementById("novi").innerHTML=window.event.screenX+"
"+window.event.screenY + "<br>" + window.event.clientX+"
"+window.event.clientY

document.getElementById("pracenje").style.top=window.event.clientY+1
+"px";

document.getElementById("pracenje").style.left=window.event.clientX+
1+"px";

document.getElementById("pracenje1").style.top=window.event.screenY+
"px";

document.getElementById("pracenje1").style.left=window.event.screenX
+"px";
})

//dohvaćamo element
let nasElement=document.getElementById("viseNacinaOkidanja");
//dodavanje preko event listener-a
nasElement.addEventListener("wheel",()=>{alert('Dodavanje eventa
pomocu listener-a')});
//dodavanje kao property
nasElement.onmouseleave = napustanje =()=>{alert('Dodavanje
eventa kao property objekta')};
}

document.addEventListener("load", dodajListener());

```

koji je linkovan s HTML stranicom čija je *body* sekcija prikazana u sljedećem kodu:

```
...
<body>
  <div ID="sve">
    <h1>Dodavanje event listener-a</h1>
    <div ID="prviDiv">
      Ovo je sadržaj jednog DIV-a
    </div>
    <div ID="drugi"></div>
    <div ID="pracenje"></div>
    <div ID="pracenje1"></div>
    <div id="novi">

      </div>
    <!-->
    <input type="text" ID="viseNacinaOkidanja"
onclick='alert("Dodavanje eventa kao atributa HTML elementa")' />

  </div>
  <script type="text/javascript" src="skript.js"></script>
</body>
...
```

Na primjeru se vidi način implementacije funkcije koja se poziva na okidanje događaja koji se dešava na nekom HTML elementu. Primjer implementacije kroz atribut HTML elementa prikazan je u HTML kodu na liniji `input` postavljanjem atributa „`onclick`“. To je ujedno i najjednostavniji način uvezivanja događaja na HTML elementu s određenom akcijom.

Drugi uvezivanja akcije sa događajem na HTML elementu je preko obilježja objekta. Kao što je već navedeno, svi HTML elementi predstavljaju objekte čije metode i obilježja čine programski interface ili sučelje preko kojeg skripte pristupaju elementima. U linijama koda je to urađeno na slijedeći način:

```
let nasElement=document.getElementById("viseNacinaOkidanja"); -
linija koda u kojoj se dohvaća ili adresira element na način da se varijabla nasElement
referira na HTML element čiji je ID = „viseNacinaOkidanja“
nasElement.onmouseleave = napustanje =>{alert('Dodavanje
eventa kao property objekta')}; - linija koda u kojoj se dodaje funkcija koja
```

će se izvršiti kao reakcija na *event* u trenutku kad se taj događaj desi. U ovom slučaju radi se o kliku na polju za unos teksta.

Treći i najbolji način uvezivanja akcije s događajem na HTML elementu je putem dodavanja tzv. *Event listener-a*. Dodavanje event listener-a demonstrirano je linijama koda:

```
nasElement.addEventListener("wheel",()=>{alert('Dodavanje eventa  
pomocu listener-a')}});
```

ili npr:

```
document.addEventListener("load", dodajListener());
```

Prva linija predstavlja primjer dodavanja akcije u formi neimenovane ili anonimne funkcije na event pomjeranja tačaka na mišu u trenutku kad se pokazivačmiša nalazi na HTML elementu na koji se referira varijabla *nasElement*. Na primjeru se vidi, da je *addEventListener*, također metoda objekta.

U drugom primjeru, može se vidjeti dodavanje *event listener-a* na HTML dokument koji predstavlja globalni objekat. U primjeru se dodaje *event listener* koji reaguje na događaj učitavanja stranice. U ovom slučaju uvezuje se imenovana funkcija „*dodajListener()*“ sa događajem učitavanja stranice. Ovaj događaj učitavanja stranice je bitan za neke JavaScript funkcije, za čije je ispravno izvršavanje preduslov potpuno učitana stranica sa svim elementima.

4

JavaScript razvojni okviri

Riječ standard zasigurno ulijeva povjerenje i sigurnost, a kada je riječ o standardima u području *web* tehnologija onda se moraju spomenuti: *HTML*, *CSS* i *JavaScript*. U okviru prethodnih poglavlja svi spomenuti koncepti su detaljnije predstavljani, a u ovom poglavlju će biti mnogo više riječi o *JavaScript* programskom jeziku koji predstavlja jedan od najzastupljenijih jezika korištenih za razvoj ne samo *web*, već i drugih vrsta aplikacija.

Zahvaljujući *JavaScript*-u *web* stranice su dobile mehanizam za ostvarenje dinamičnijeg prikaza podataka, što prvenstveno podrazumijeva ažuriranja sadržaja u realnom vremenu, prikaz interaktivnih mapa i video zapisa u odnosu na jednostavno prikazivanje statičkih podataka. Implementacija kompleksnijih korisničkih interfejsa, dinamičnost, interakcija, specijalni efekti koji se trebaju prikazati u *web* pregledniku je izuzetno kompleksna i dugotrajna, a slična logika je vjerovatno napisana već mnogo puta. Upravo je potreba za dinamičnošću ubrzo pozicionirala *JavaScript* kao

esencijalnu komponentu savremenih *web* aplikacija, te je naredni logičan korak predstavljalo pisanje biblioteka koje će automatizovati svakodnevne programerske zadatke i razvoj interaktivnih aplikacija učiniti što bržim i efikasnijim. Neke od zadataka koje biblioteke olakšavaju bi mogle biti automatsko popunjavanje tj. dovršavanje riječi prilikom kucanja, učitavanje različitih tipova sadržaja (audio, video i dr.), interakciju s *web* servisima i sl. Sve veći zahtjevi su doveli do toga da je količina potrebnog JavaScript koda uzrokovala poteškoće u organizaciji, te u konačnici čak i u brzini izvršavanja pojedinih dijelova. U tom kontekstu, biblioteke, kao osnovne gradivne jedinice, laganim, ali sigurnim koracima, prerasle su u razvojne okvire (engl. *Framework*) koji osiguravaju strukturiranost projekta, te programerima nude čitav set različitih funkcionalnosti koje su višestruko testirane i provjerene u toku rada procesno intenzivnih aplikacija. Bitno je naglasiti da osnovna razlika između biblioteke i razvojnog okvira leži u činjenici da biblioteke programerima ostavljaju mnogo više kontrole nad izvršenjem, dok razvojni okvir uglavnom zahtijeva ispunjenje određenih uslova nakon čega u potpunosti preuzima nadležnost nad izvršenjem određene akcije odnosno aplikacije. Također je važno napomenuti da se korištenjem razvojnih okvira osiguravaju najznačajniji koraci u životnom ciklusu razvoja softvera, a oni su: predvidivost izvršenja određenih akcija, poštivanje standarda, te nadogradnja i održavanje.

Inicijalno namijenjen uglavnom osiguranju dinamičnosti i interaktivnosti korisničkog interfejsa, te izvršavanju unutar preglednika tj. na klijentskoj strani, JavaScript je vremenom prerastao u programski jezik koji se može koristiti kako na klijentskoj tako i na serverskoj strani. Spomenuto podrazumijeva činjenicu da JavaScript posjeduje mogućnost prihvatanja i obrade zahtjeva pristiglih od strane *web* preglednika. Drugim riječima, *JavaScript* je u stanju podržati „full-stack“ razvoj aplikacija.

Nakon posjete određenoj *web* adresi (npr. www.fit.ba), *web* preglednik, od servera na kome se hostira pomenuta *web* stranica, preuzima generisanu HTML datoteku¹⁰, te krećući se od početka prema kraju, red po red, interpretira njen sadržaj. Također, *web* preglednici posjeduju komponente koje su u stanju izvršiti *JavaScript* kod, te zahvaljujući njima, uglavnom, statički HTML kod i njegove komponente doživljavaju željenu transformaciju. Nailaskom na određeni *JavaScript* kod, komponenta *web* preglednika ga izvršava u cijelosti prije nego što nastavi interpretirati ostatak HTML dokumenta. Spomenuto se može odnositi na modifikaciju određenog HTML sadržaja

¹⁰ Svaki HTTP zahtjev, u ovom kontekstu, rezultira generisanjem adekvatnog odgovora u HTML formatu

ili povezivanje određene kontrole s događajem koji će se izvršiti prilikom interakcije sa tom kontrolom.

U želji da osigura dominaciju u svom području, *JavaScript* danas postaje osnova alata za testiranje, razvoj mobilnih aplikacija, interakciju s bazama podataka i mnoge druge segmente razvoja.

Kao što je već na početku naglašeno, potreba za različitim pristupima i načinima realizacije pojedinih koncepata dovela je do pojave velikog broja biblioteka i razvojnih okvira, a neki od najpopularnijih su:

- React
- Angular
- Vue
- Ember
- Backbone
- Polymer i dr.

U pobrojanim opcijama se nalaze biblioteke i razvojni okviri. Međutim, pošto je u prethodnom dijelu poglavlja pojašnjena razlika između ta dva koncepta, te uzimajući u obzir činjenicu da za savladavanje sadržaja ovog poglavlja nisu pretjerano značajne razlike između njih, u nastavku rada će se koristiti pojam *razvojni okvir*.

Veliki broj dostupnih razvojnih okvira sa sobom donosi mnoštvo nedoumica, posebno kod osoba koji se tek upoznaju sa *JavaScript*-om, a neka od najčešćih su: „Da li mi je uopšte potreban neki od razvojnih okvira?“, „Koji od razvojnih okvira odabrati?“ i sl.

Potreba za korištenjem nekog od razvojnih okvira se može opisati na mnogo načina, a jedan od njih bi mogao biti i primjer kupovine pojedinih komponenti računara i njihovo sastavljanje u jednu cjelinu što zasigurno predstavlja čestu i uobičajenu pojavu. Ipak, spomenuta aktivnost će zahtijevati više vremena, istraživanja i znanja jer, na primjer, željena grafička kartica možda ne odgovara kupljenoj matičnoj ploči ili u okviru odabrane matične ploče grafička kartica ne može ostvariti svoj puni potencijal. Također, može se desiti da grafička kartica ne posjeduje adekvatne izlaze za odabrani monitor, što obično zahtijeva dodatne troškove i kupovnu odgovarajućeg konvertera, pa sve do mogućnosti da monitor limitira ostvarenje punog potencijala grafičke kartice. Sve spomenuto su zasigurno pitanja s kojima su se susreli oni koji žele samostalno sklopiti novi računar, a sve u odnosu na mogućnost kupovine pripremljene konfiguracije koja je već testirana, sa instaliranim

operativnim sistemom i svim potrebnim drajverima. U kontekstu nabavke računara, korištenje razvojnih okvira bi se moglo poistovjetiti s kupovinom brendirane, testirane i pripremljene računarske konfiguracije koja osigurava njenu instantnu upotrebu u smislu obavljanja uobičajenih aktivnosti: pretrage interneta, posjete socijalnim mrežama, igranja omiljenih igara i dr., dok bi sve pobrojane aktivnosti vjerovatno morale biti pauzirane sve dok se, pojedinačno kupljene komponente ne povežu, provjere eventualni nedostaci u njihovom radu, te izvrši instalacija operativnog sistema i svih pripadajućih drajvera. Drugim riječima, korištenje razvojnih okvira osigurava uštedu vremena i novca jer se razvojni tim kompanije može odmah posvetiti rješavanju problema iz domene za koju je određeno softversko rješenje namijenjeno, a ne da se bave osiguranjem uobičajenih funkcionalnosti.

U nastavku poglavlja će biti ukratko predstavljene osnovne karakteristike nekih od ranije pobrojanih razvojnih okvira, a sve u cilju uvida u jednostavnost i logiku njihovog rada. Ne treba zaboraviti činjenicu da bi detaljnije predstavljanje svakog od razvojnih okvira zahtijevalo nekoliko knjiga, te je potrebno imati razumna očekivanja od jednog poglavlja koji opisuje nekoliko njih.

Upoznavanje s *JavaScript* razvojnim okvirima započinje s *Ember*-om u okviru koga će biti opisane potrebne komponente za razvoj koje su, također, potrebne i za većinu drugih razvojnih okvira. Nakon toga, sa malo više detalja, će biti predstavljen *React*, a na kraju poglavlja će biti predstavljen *Vue*. Svaki od, u nastavku predstavljenih, razvojnih okvira će biti ukratko opisan do nivoa kreiranja osnovnih gradivnih jedinica gotovo svih aplikacija, a to su komponente. *Angular*, kao zasigurno jedan od najpopularnijih razvojnih okvira, neće biti predstavljen u ovom poglavlju jer se on detaljnije izučava na drugim predmetima.

4.1 Ember.JS

Ember.JS (u nastavku *Ember*) je nasljednik danas manje popularnog razvojnog okvira *SproutCore* koji je kreiran od strane kompanije Apple i korišten u razvoju platformi za efikasniju interakciju i upravljanje iOS uređajima, ali i drugih vrsta aplikacija u tom području. Sve veće potrebe i zahtjevi tržišta doveli su do zaključka da bi naredna verzija *SproutCorea* zahtijevala toliko dramatične promjene da je mnogo efikasnije razmišljati o kreiranju potpuno novog razvojnog okvira koji je nazvan *Ember*.

Još od najranijih verzija, a u odnosu na druge razvojne okvire, kao najznačajnije karakteristike *Embera* se navode (Jesse Cravens, 2014): jednostavan mehanizam

kreiranja dvosmjerne veze s izvorom podataka, podrška radu za različitim vrstama ORM (*Object–Relational Mapping*) alata, te ugrađena podrška za upravljanje historijom ruta. Pored kompanije Apple, ne treba zaboraviti činjenicu da je *Ember* itekako popularan razvojni okvir i unutar drugih IT kompanija današnjice kao što su *Microsoft* i *Netflix*.

Istraživanje o *Ember* razvojnem okviru zasigurno će nametnuti pitanje zbog čega je simpatični hrčak njegov zaštitni znak ili ikonica? Naime, odabirom spomenutog zaštitnog znaka se nastojala poslati poruka da se radi o razvojnem okviru koji će programerima omogućiti da odmah krenu na posao kao što obično maleni hrčak kreće u dugotrajno trčanje unutar svog koluta.

Ember se također može predstaviti kao open-source *JavaScript* razvojni okvir koji je zasnovan na dokazanim praksama, te omogućava jednostavan razvoj modularnih *web* aplikacija. Spomenuta jednostavnost i modularnost je prije svega zasnovana na korištenju skupa API-ja koji omogućavaju implementaciju čak i najkompleksnijih zadataka. Osobina modularnosti je zasluga *Ember* CLI (*Command Line Interface*) koji predstavlja oficijelni i najkorišteniji način za kreiranje, izgradnju, testiranje i izvršavanje/hostiranje datoteka koje čine *Ember* aplikaciju ili ekstenziju. CLI ne predstavlja komponentu koja je vezana samo za *Ember*, već i neizostavna i u radu drugih razvojnih okvira.

4.1.1 Instalacija potrebnih *Ember* komponenti

Da bi se pokrenula jedna *Ember* aplikacija, potrebno je nekoliko komponenti za čiju je konfiguraciju i usklađivanje prvenstveno zaslužan pomenuti CLI. Ipak, prije CLI-a, kao osnovnu komponenta za razvoj *Ember* aplikacija treba navesti *Node.js* koji predstavlja asinhrono *JavaScript* okruženje (*JavaScript runtime*) namijenjeno razvoju skalabilnih aplikacija. Za potrebe windows okruženja, dovoljno je preuzeti instalacijski fajl s oficijelne *Node.js* stranice i u svega par klikova mišem glavna komponenta za razvoj aplikacija je dostupna na lokalnom računaru.

Nakon instalacije, korištenjem sljedećih komandi, unutar neke od konzolnih aplikacija, moguće je provjeriti da li je sve proteklo bez problema odnosno koja je verzija (-v) *Node.js*-a instalirana na lokalnom računaru:

```
PS C:\Users\User> node -v
v16.13.0
PS C:\Users\User> npm -v
8.1.0
```

S instalacijom Node.js-a, a što se može vidjeti u prethodnom primjeru, dolazi i odgovarajuća verzija **npm**-a (*Node Package Manager*) koji predstavlja podrazumijevani menadžer paketa za Node.js. Paketi konceptualno predstavljaju izuzetno jednostavan način za distribuciju različitih funkcionalnosti, odnosno biblioteka, te su aktualni u gotovo svim programskim jezicima i razvojnim okruženjima. Također, još jednom je bitno naglasiti da Node.js i npm nisu osnova za razvoj samo *Ember* aplikacija, već i aplikacija za veliki broj drugih *JavaScript* razvojnih okvira. Upravo zbog toga, komponente i procedura instalacije opisane u ovom dijelu se može iskoristiti i za ostale razvojne okvire opisane u nastavku poglavlja.

Nakon *Node.js*-a, potrebno je posjedovati i ranije spomenuti CLI kojeg je moguće instalirati korištenjem sljedeće komande:

```
PS C:\Users\User> npm install -g ember-cli
```

Instalacija paketa se može vršiti lokalno (lokalni folder projekta) ili globalno (globalni node folder). Globalno instalirani paketi su dostupni s bilo koje lokacije, te ih je zbog toga potrebno instalirati samo jednom. Uzimajući u obzir spomenuto, u okviru prethodne komande je korišten dodatak (flag) *-g* kojim se označava da se određeni paket instalira globalno. Pored globalnih, podrazumijevano, aplikacije referenciraju lokalne pakete.

Prethodnim koracima trebalo bi da su instalirane sve komponente neophodne za razvoj *Ember* aplikacija. Verifikaciju uspješnosti instalacije tj. provjeru verzije instaliranog CLI-a, moguće je izvršiti korištenjem komande prikazane u nastavku.

```
PS C:\Users\User> ember -v
ember-cli: 3.28.4
node: 16.13.0
os: win32 x64
```

Potrebno je napomenu da će, u slučaju korištenja *PowerShell* konzole, a u zavisnosti od postavki na računaru, možda biti potrebno modifikovati neke od sigurnosnih postavki koje se odnose na manipulisanje konfiguracijskim fajlovima ili skriptama.

Ukoliko su sve ranije komponente uspješno instalirane na lokalnom računaru, još je jedino preostalo da se odabere adekvatno razvojno okruženje ili editor programskog koda. Za primjere programskog koda opisane u ovom poglavlju korišten je sve popularniji besplatni alat *Microsoft Visual Studio Code*.

Demonstracija korištenja prethodno instaliranih komponenti i alata će započeti kreiranjem odgovarajućeg foldera na željenoj lokaciji diska, npr. C:\Users\User\Ember\Intro. Nakon pozicioniranja u prethodno kreirani folder, koristeći *PowerShell* ili neku drugu konzolnu aplikaciju¹¹, potrebno je izvršiti komandu *init* koja poziva istoimenu funkciju.

```
PS C:\Users\User\Ember\Intro> ember init
installing app
Ember CLI v3.28.4
...
```

Sadržaj Intro foldera jasno ukazuje na činjenicu da je generisana, te ujedno i povezana, kompletna struktura projekta, a osnovne informacije o strukturi foldera su date u narednoj tabeli.

Folder	Opis
app	Jedan od naznačajnijih foldera u projektu koji sadrži modele, komponente, predloške (engl. templates), logiku, rute, stilove i dr.
config	Konfiguracijski folder koji sadrži varijable za podešavanje samog okruženja.
public	Folder namijenjen čuvanju fajlova kao što su slike, fontovi i drugi dokumenti
tests	Unutar ovog foldera se čuvaju fajlovi za testiranje aplikacije, počevši od jediničnih pa sve do integracijskih testova.
vendor	Folder koji najčešće sadrži fajlove instalirane korištenjem npm-a, odnosno pakete referencirane u projektu.

Generisanje foldera i osnovne strukture projekta, kao što je to učinjeno s komandom *init*, je moguće izvršiti i korištenjem komande *new* kao što je prikazano u narednom primjeru.

```
PS C:\Users\User\Ember> ember new Intro
```

Bez obzira da li se radi o inicijalnoj verziji ili verziji koja je spremna za produkcijsko okruženje, pokretanje *Ember* aplikacije se može izvršiti na nekoliko načina, a jedan od njih je izvršenje komande *npm start*. Procedura za pokretanje aplikacije će biti identična i za aplikacije kreirane u drugim *JavaScript* razvonim okvirima opisanim u nastavku poglavlja.

¹¹ Na raspolaganju je i Terminal komponenta unutar Microsoft Visual Studio Code-a

```
PS C:\Users\User\Ember\Intro> npm start
```

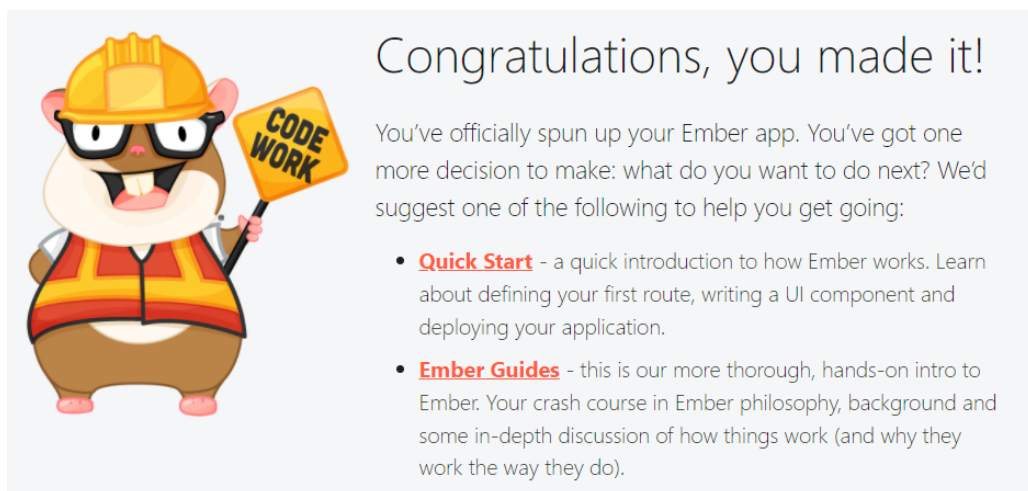
```
> intro@0.0.0 start  
> ember serve
```

```
Build successful (12325ms) - Serving on http://localhost:4200/
```

Nakon uspješnog okončanja prethodno opisanog procesa, aplikacija postaje dostupna na, u ispisu, naznačenoj adresi, odnosno u konkretnom primjeru na adresi: <http://localhost:4200/>. U cilju što jednostavnijeg predstavljanja glavnih karakteristika *Embera*, projektu nisu dodavani konfiguracijski fajlovi koji bi zahtijevali mnogo dodatnih pojašnjenja.

4.1.2 Razvoj Ember aplikacije

Posjetom lokaciji *localhost:4200* u *web* pregledniku bi se trebao prikazati sadržaj kao na Ilustracija 42.



Ilustracija 42 - Izgled početne stranice Ember projekta sa generisanim postavkama

Prikazani sadržaj je opisan fajlom *application.hbs* koji se nalazi u folderu *templates* (lokacija unutar projekta: *intro->app->templates*), a glavni sadržaj prikaza je opisan komponentom *WelcomePage*.

```
{{page-title "Intro"}}
```

```
{{!-- The following component displays Ember's default welcome  
message.--}}
```

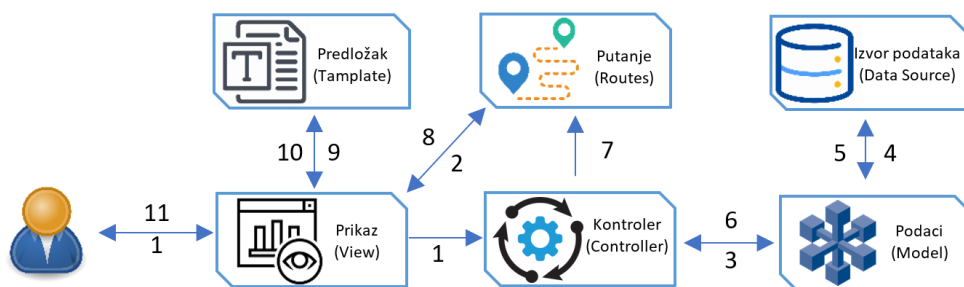
```
<WelcomePage />
{{!-- Feel free to remove this! --}}

{{outlet}}
```

Unutar kreiranog projekta se može ukloniti instanciranje *WelcomePage* komponente jer će u nastavku biti kreirana nova komponenta koja će biti zadužena za prikaz podataka o predmetima.

Određenu dozu iznenađenja prilikom prvog susreta s fajlovima unutar *Ember* projekta obično uzrokuje ekstenzija *.hbs* (skraćenica od termina *Handlebars*) koji predstavlja osnovu za kreiranje predložaka (engl. *template*), a omogućava korištenje direktiva koji osiguravaju upravljanje sadržajem na napredniji način u odnosu na HTML. U tom kontekstu se veoma često može pronaći konstatacija da predlošci u *Ember*-u posjeduju supermoć. Prilikom pokretanja, predlošci se kompajliraju u format koji je jednostavnije procesirati okruženju unutar koga se aplikacija izvršava. Također, treba naglasiti da *.hbs* fajlovi mogu sadržavati i standardni HTML kod.

Ember u potpunosti podržava MVC (*Model-View-Controller*) implementaciju, koja se može smatrati opšteprihvaćenim pristupom u razvoju aplikacija, a prema kojoj *Model* komponenta ima zadatak osigura pristup podacima, *View* je zadužen da osigura adekvatan prikaz zahtijevanih podataka, dok je *Controller* odgovoran za povezivanje i interakciju među ostalim komponentama. Unutar *Embera*, *Controller* komponenta uključuje rute i kontrolere, dok *View* uključuje prikaze i predloške. Većina pomenutih stavki se automatski generiše u momentu dodavanja određene rute, ali prije nego što detaljnije bude obrađena procedura kreiranja rute i pristupa podacima, na Ilustracija 43 je prikazan redoslijed interakcije između pojedinih dijelova *Ember* aplikacije. Glavne komponente predstavljene sa Ilustracija 43 će ukratko biti opisane u nastavku poglavlja kao i osnove njihovog korištenja.



Ilustracija 43 - Redoslijed interakcije komponenti unutar *Ember* aplikacije

Pristup određenoj funkcionalnosti ili stranici *web* aplikacije je obično definisan nekom putanjom ili rutom (*URL*-om) koja će korisnika usmjeriti na željenu lokaciju. *Ember* posjeduje mehanizam generisanja i upravljanja rutama koji automatski povezuje sve komponente neophodne za navigaciju. U inicijalnoj strukturi *Ember* aplikacije definisana je podrazumijevana ruta *application* koja je povezana sa ranije pomenutim *application.hbs* fajlom. Na taj način, *application.hbs* fajl predstavlja početnu ili ulaznu tačku u *web* aplikaciju, te ukoliko se želi kreirati nova ulazna tačka potrebno je definisati novu rutu. Za potrebe kreiranja nove rute na raspolaganju je komanda *generate route*:

```
PS C:\Users\User\Ember\Intro> ember generate route predmeti
installing route
  create app\routes\predmeti.js
  create app\templates\predmeti.hbs
updating router
  add route predmeti
installing route-test
  create tests\unit\routes\predmeti-test.js
```

Ispis nakon pokretanja komande za generisanje rute, koja se naziva *predmeti*, jasno ukazuje na infrastrukturu koja je kreirana za potrebe dostupnosti stranice <http://localhost:4200/predmeti> u okviru koje će se, nakon u nastavku prikazane implementacije, prikazati predmeti dostupni studentima za odabir. Drugim riječima, kreiranje rute podrazumijeva automatsko kreiranje: predložka koji će biti prikazan korisnicima, rute do nove lokacije definane unutar *router.js* fajla, *route* objekta koji je zadužen za dostupnost modela korištenog od strane predložka, te jedinični test za novu rutu.

Generisani *predmeti.js* fajl predstavlja lokaciju putem koje će se dobavljati podaci za potrebe prikaza unutar predložka ili komponente na nivou određene rute. Bez obzira da li podaci dolazili od nekog *web* servisa ili predefinisane liste, bitno je spomenuti da oni postaju dostupni unutar *.hbs* fajla putem ključne riječi *model*, što je na sličan način implementirano i u velikom broju drugih razvojnih okruženja. Naime, *model* može sadržavati različite tipove podataka, počevši od primitivnih, kolekcija, nizova ili listi, pa sve do kompleksnijih objekata koji unutar sebe objedinjuju sve spomenuto. U nastavku je prikazan sadržaj fajla *predmeti.js* u okviru koga metoda *model* vraća listu objekata pri čemu svaki objekat opisuje tri osobine karakteristične za određeni predmet. *PredmetiRoute* predstavlja klasu koja proširuje ili nasljeđuje klasu *Route*, koja je, kao bazna klasa, definisala glavne osobine potrebne za rad aplikacije.

```
import Route from '@ember/routing/route';

export default class PredmetiRoute extends Route {
  model() {
    return [
      { Id: 1, Naziv: 'Ember.js programiranje', Semestar: 2 },
      { Id: 2, Naziv: 'Web razvoj i dizajn', Semestar: 3 },
      { Id: 3, Naziv: 'Embedded i RT sistemi', Semestar: 4 }
    ];
  }
}
```

Konzumiranje podataka, u narednom primjeru samo ispis, dostupnih putem modela, unutar pripadajućeg *.hbs* fajla (*predmeti.hbs*) se na jednostavan način može izvršiti korištenjem helpera *each*. Svaka vrijednost unutar modela (*@model*) će se referencirati nazivom *predmet*, a pošto se radi o objektima svakom svojstvu se pristupa preko operatora tačka. Helper *each* je jako sličan *foreach* petlji koju posjeduje većina savremenih programskih jezika.

```
{{page-title "Akademska 2021/22"}}
<ol>
  {{#each @model as |predmet|}}
    <li>{{predmet.Naziv}} (Semestar: {{predmet.Semestar}})</li>
  {{/each}}
</ol>
```

Prethodnim primjerom su samo ispisane vrijednosti osobina svih objekata koji se nalaze unutar modela, a ukoliko se korisniku treba omogućiti selekcija predmeta koje želi pratiti u određenom semestru, onda je neophodno napraviti proširenje na način da im se doda *checkbox* html element.

```
{{page-title "Akademska 2021/22"}}
<p>Odaberite izborne predmete koje želite slušati: </p>
<ul>
  {{#each @model as |predmet|}}
    <input type="checkbox" id={{predmet.Id}}>
    <label for={{predmet.Naziv}}>{{predmet.Naziv}} (Semestar:
{{predmet.Semestar}})</label>
  <br>
```

```
{{/each}}  
</ul>
```

Prethodnim primjerom korisniku je ponuđena mogućnost odabira (Ilustracija 44), ali se njegov izbor ni na koji način ne pohranjuje niti utječe na druge elemente prikaza ili buduće akcije.

Odaberite izborne predmete koje želite slušati:

- ☒ Ember.js programiranje (Semestar: 2)
- ☒ Web razvoj i dizajn (Semestar: 3)
- ☐ Embedded i RT sistemi (Semestar: 4)

Ilustracija 44 Prikaz objekata iz modela koji predstavljaju listu predmeta

Detaljnija interakcija s izvorom podataka će biti opisana u dijelovima poglavlja koji govori o React i Vue razvojnim okvirima. Za kraj ovog dijela koji govori o *Ember-u*, ukratko će biti predstavljene komponente kao jedan od osnovnih gradivnih elemenata savremenih aplikacija. Naime, komponente se mogu posmatrati kao jedinstvene cjeline koje objedinjuju sve elemente neophodne za njihovo funkcionisanje, počevši od načina prikaza i stilova, podataka, interakcije s drugim komponentama i sl. Pored toga, kad god se javi potreba za istom funkcionalnošću ili prikazom na različitim lokacijama, odnosno dijeljenjem iste vrste podataka u različitim prikazima, onda komponente predstavljaju idealno rješenje.

Proces kreiranja komponente je jako sličan kreiranju putanje što se može vidjeti iz naredne komande koja kreira komponentu pod nazivom *ListaPredmeta*.

```
PS C:\Users\User\Ember\Intro> ember generate component ListaPredmeta
```

Kreiranje komponente je podrazumijevalo i generisanje pripadajućeg *.hbs* fajla unutar koga će biti definisana struktura komponente. Komponenta ne bi trebala biti u potpunosti prilagođena bilo kojem prikazu, odnosno, trebala bi biti definisana na način da se što više podataka dinamički učitava, odnosno ustupa komponenti sa lokacije na kojoj se ona koristi ili iz nekog drugog izvora podataka.

U nastavku je prikazana vrlo jednostavna definicija komponente *ListaPredmeta* zasnovana na prethodnom primjeru, a koja omogućava da se sa lokacije njenog

korištenja prosljede sljedeći argumenti: *naslov* (zaglavlje sadržaj kojeg je potrebno prikazati korisniku), *tekst* (tekstualni sadržaj u formi pojašnjenja ili pitanja koji upućuje na očekivanja od korisnika u kontekstu interakcije s podacima), te *podaci* koji su unutar komponente referencirani kao *@predmeti*.

```
<h3> {{@naslov}} </h3>
<p> {{@tekst}} </p>
<ul>
  {{#each @predmeti as |predmet|}}
    <input type="checkbox" id={{predmet.Id}}>
    <label for={{predmet.Naziv}}>{{predmet.Naziv}}
      (Semestar:
{{predmet.Semestar}}</label>
    <br>
  {{/each}}
</ul>
```

Nakon definisanja njene strukture, komponenta *ListaPredmeta* je dostupna za korištenje, odnosno selekciju predmeta koji se prosljede unutar *@predmeti* argumenta. U nastavku je, unutar fajla *predmeti.hbs*, prikazan način korištenja komponente *ListaPredmeta* odnosno način slanja vrijednosti koji će biti prikazani unutar komponente.

```
<ListaPredmeta
  @naslov = "Akademska 2021/22"
  @tekst = "Odaberite izborne predmete koje želite slušati: "
  @predmeti = {{@model}}
/>
```

Prikaz ponuđenih predmeta bi trebao biti gotovo identičan ranijim primjerima, a pored gotovo identične strukture bitno je primijetiti da se komponenti, kao izvor podataka, šalje *@model* koji je dostupan zahvaljujući ranije pojašnjenoj metodi *model*. Nakon definisanja vlastite komponente trebala bi biti jasnija stranica dobrodošlice koja je bila prikazana nakon uspostavljanja *Ember* infrastrukture, odnosno korištenjem komponente *WelcomePage* (Ilustracija 42).

4.2 React.JS

Razvojni okviri koje kreiraju neki od najvećih tehnoloških kompanija današnjice od samog starta imaju značajnije predispozicije za uspjeh. Kompanija Facebook je, sada

već davne, 2013. godine objavila novi razvojni okviru pod nazivom *React* koji se, za razliku od dostupnih razvojnih okvira, fokusira samo na *View* segment, odnosno prikaz ili slovo *V* unutar MVC arhitekture. Upravo zbog svega spomenutog se veoma često kaže da *React* predstavlja *JavaScript* biblioteku namijenjenu razvoju korisničkog interfejsa.

Kao i kod velikog broja drugih razvojnih okvira, osnovu razvoja *React* aplikacija predstavlja razvoj komponenti kao samostalnih modula, koje su konceptualno ukratko opisane u prethodnom dijelu poglavlja. Komponenta može biti bilo koji element interfejsa, počevši od kontrole za automatsko pretragu podataka pa sve do kompleksnijih komponenti koje objedinjuju veći broj drugih komponenti i kontrola. Kreiranje interfejsa ne predstavlja ništa drugo već korištenje i međusobnu integraciju različitih kontrola. Podrazumijevano, svaka *React* aplikacija posjeduje jednu glavnu *App* komponentu, te sve ostale predstavljaju njenu djecu. S implementacijskog aspekta, komponenta ne predstavlja ništa drugo nego klasu koju opisuje njeno stanje predstavljeno osobinom (engl. *property*) *state* i metoda koja je zadužena za njen prikaz (metoda *render*). Ipak, prije kreiranja prve *React* aplikacije potrebno se podsjetiti do sada naučenih koncepata bez kojih se teško mogu savladati noviteti predstavljeni u nastavku poglavlja.

Unutar prvih poglavlja ove knjige detaljnije je opisan DOM (*Document Object Model*) kao koncept koji predstavlja logičku reprezentaciju *web* stranice na način da omogućava interakciju sa svim elementima i njihovim osobinama. Svaka promjena u vrijednostima osobina je obično zahtijevala osvježavanje kompletnog prikaza, što se u zavisnosti od kompleksnosti interfejsa *web* stranice, pokazalo kao proces koji vremenski i performansno nije zanemariv. Za potrebe unapređenja spomenutog procesa, predložen je *Virtual DOM* (skr. VDOM) kojim je uveden koncept memorijske (engl. *in-memory*) ili virtualne reprezentacije elemenata korisničkog interfejsa koja se konstantno sinhronizuje s originalnom verzijom DOM-a. Drugim riječima, VDOM predstavlja apstrakciju DOM-a koja osigurava mnogo brži odgovor na promjene u stanju pojedinih elementima interfejsa, a proces sinhronizacije između njih, VDOM-a i DOM-a, obavlja biblioteka koja se naziva *React DOM*. Zahvaljujući ovom pristupu, više nije potrebno pisati kod koji će manipulirati DOM-om, već se svaka promjena stanja komponente automatski reflektuje na DOM element.

4.2.1 Instalacija potrebnih React komponenti

Razvoj aplikacija korištenjem *React* razvojnog okvira zahtijeva gotovo identičnu infrastrukturu kao i opisani *Ember*. Dakle, potrebno je instalirati *Node.js* sa kojim

dolazi i *npm*, a pošto je kompletna procedura opisana u prethodnom dijelu poglavlja u ovom dijelu se neće ponavljati.

Nakon kreiranja foldera unutar koga će biti generisana inicijalna struktura projekta, instalaciju biblioteka neophodnih za razvoj *React* aplikacija, unutar neke od konzonlihi aplikacija, moguće je izvršiti sljedećom komandom:

```
PS C:\Users\User\React\Intro> npm install react
```

Proces kreiranja aplikacije obično podrazumijeva podešavanje većeg broja različitih komponenti od kojih su najznačajniji: *Babel* (*JavaScript* kompajler), *Development Server* (server koji omogućava izvršenje aplikacije tokom razvoja), te *WebPack* (zadužen je za pakovanje svih *JavaScript* fajlova neophodnih za izvršavanje aplikacije unutar *web* preglednika)¹². Manualno podešavanje i povezivanje pobrojanih komponenti će u određenim slučajevima predstavljati preduslov za uspješnu realizaciju projekta. Međutim, početne faze ovladavanja *React*-om zasigurno nisu adekvatan momenat za manualno podešavanje pojedinih komponenti, te će biti iskorišten mehanizam koji se naziva *create-react-app* koga je prethodno potrebno instalirati.

```
PS C:\Users\User\React\Intro> npm install -g create-react-app
```

Generisanje kompletne strukture projekta i povezivanje svih njegovih komponenti zahtijeva jednostavni poziv prethodno instaliranog mehanizma i dodjeljivanje adekvatnog imena projektu. U prikazanom primjeru, projektu je dodijeljen naziv *react-book-app*.

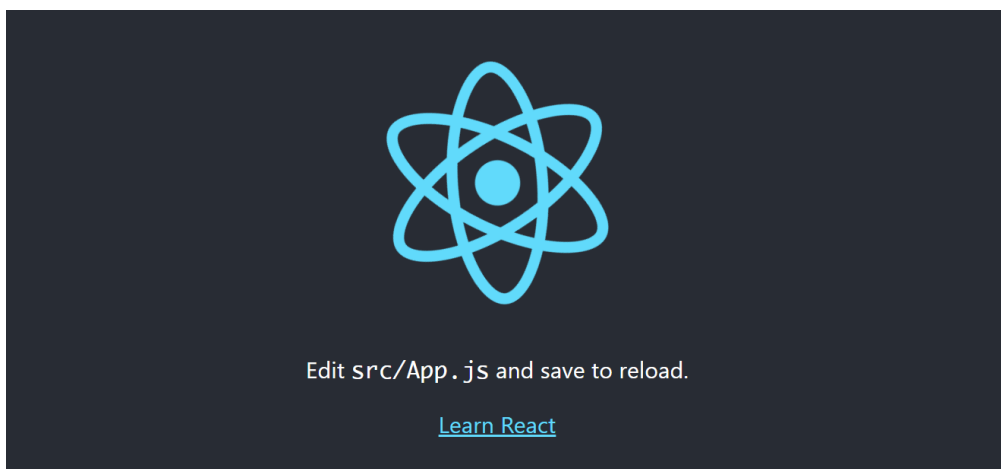
```
PS C:\Users\User\React\Intro> create-react-app react-book-app
```

Pokretanje *React*, baš kao i *Ember* aplikacije, zahtijeva izvršenje *npm start* komande unutar prethodno kreiranog aplikacijskog foldera.

```
PS C:\Users\User\React\Intro\react-book-app> npm start
```

Ukoliko su svi koraci vezano za kreiranje *React* aplikacije uspješno okončani, odabrani *web* preglednik bi, na adresi <http://localhost:3000/> trebao prikazati podrazumijevanu stranicu kao na ilustracija 455.

¹² Pomenute komponente se koriste i u okviru drugih razvojnih okvira, ali će u ovom dijelu biti malo detaljnije predstavljene.



Ilustracija 45 - Izgled početne stranice React projekta sa generisanim postavkama

Folder *react-book-app* sadrži sve povezane elemente generisane prilikom kreiranja projekta, a informacije o glavnim folderima su date u narednoj tabeli.

Folder/Fajl	Opis
node_modules	Sadržajem ovog foldera je obično u potpunoj nadležnosti npm-a, odnosno njime se upravlja korištenjem odgovarajućih komandi. Unutar njega se nalazi lista node paketa koji su neophodni za izvršenje aplikacije. Upravo zbog toga, veoma su rijetke manualne intervencije nad sadržajem ovog foldera.
package.json	Unutar ovog fajla se nalazi lista međuzavivosti node paketa i drugih konfiguracijskih detalja projekata.
public	Fajlovi koji nastaju prilikom razvoja <i>React</i> aplikacije se obično smještaju unutar ovog foldera. Prikaz s Ilustracija 45 se dobije inicijalnom postavkom projekta, a definisan je unutar fajla <i>index.html</i> koji se nalazi u ovom folderu.

4.2.2 Razvoj React aplikacija

Dublje razumijevanje interakcije koja se mora ostvariti kako bi se određeni sadržaj prikazao na strani korisnika zahtijeva poznavanje uloge dva termina: ES* i JSX. ES se odnosi na *ECMAScript* i predstavlja *JavaScript* standard koji osigurava interoperabilnost *web* stranica u okviru različitih *web* preglednika. ES je doživio veliki broj nadogradnji odnosno verzija, a način njihovog numerisanja je u najmanju ruku bio zanimljiv. Naime, do 2016. godine, *ECMAScript*-e su označavane verzijama ES1 – ES6, da bi se nakon toga u konvenciju imenovanja uključile kalendarske godine, tako

da je posljednja¹³, 12. verzija, označena kao ECMAScript 2021, a neki od noviteta koje je sa sobom donijela odnose se na metodu *replaceAll* dostupnu u okviru string-ova, logičke operatore dodjele i dr.

JSX (*JavaScript eXtension*¹⁴) predstavlja *React* ekstenziju koja omogućava pisanje *JavaScript* koda koji je jako sličan XML i HTML-u. Spomenuta ekstenzija proširuje *ECMAScript* specifikaciju na način da sadržaj sličan XML i HTML-u može koegzistirati zajedno s *React* kodom. JSX sadržaj predstavlja ulaz u neki od kompajlera, kao što je ranije pomenuti *Babel*, koji će sve definisane izraze transformisati u odgovarajući *JavaScript* kod. U nastavku je prikazan primjer JSX sadržaja, te rezultata njegovog transformisanja od strane *Babela*.

Ulaz	<pre>const zaglavlje = (<div> <h2>Akademska 2021/22</h2> <p>Odaberite izborne predmete koje želite slušati:</p> </div>);</pre> 
Izlaz	<pre>const zaglavlje = /*#__PURE__*/ React.createElement("div", null, /*#__PURE__*/ React.createElement("h2", null, "Akademska 2021/22"), /*#__PURE__*/ React.createElement("p", null, "Odaberite izborne predmete koje \u017Eelite slu\u0161ati:"));</pre>

Unutar *index.html* fajla, koji je lociran u folderu *public*, nalazi se *div* element sa *id*-om *root*. Spomenuti element trenutno predstavlja kontejner za izvršenje *React* aplikacije na način da se unutar njega učitava *App* komponenta, a što je definisano unutar *index.js* fajla.

```
ReactDOM.render(
  <React.StrictMode>
```

¹³ Verzija koja je aktualna u vrijeme pisanja ove knjige

¹⁴ JSX se ponekad označava i kao JavaScript XML

```
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

Koncept *React* DOM-a je ranije predstavljen, a njegova *render* metoda ima zadatak da unutar *root* elementa učitava *App* komponentu. *StrictMode* omogućava da se izvrši verifikacija korištenja najboljih praksi i prihvatljivih verzija metoda, te ukaže na potencijalne rizike u implementaciji određenih komponenti. Prateći opisanu proceduru, umjesto *App* komponente se na vrlo jednostavan način može učitati neki drugi sadržaj:

```
const zaglavlje =
  (<div>
    <h2>Akademska 2021/22</h2>
    <p>Odaberite izborne predmete koje želite slušati:</p>
  </div>);
ReactDOM.render(zaglavlje, document.getElementById('root'));
```

Poprilično jednostavan primjer JSX koda je prikazan unutar objekta zaglavlje. Malo kompleksniji primjer se može pronaći unutar *App* funkcijske komponente koja vraća *React* objektat inicijalno prikazan na početnoj stranici generisanog projekta (Ilustracija 45), a koji je prikazan u nastavku.

```
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a className="App-link" href="https://reactjs.org"
          target="_blank" rel="noopener noreferrer">
          Learn React
        </a>
      </header>
    </div>
  );
}
```

U slučaju da nije potrebno čuvati stanja određenih akcija, odnosno objekata nad kojim se izvršavaju, umjesto klasa se mogu koristiti funkcijske komponente. Primjer funkcijske komponente je upravo ranije pomenuta *App* funkcija.

Komponente implementirane u obliku klasa obično posjeduju osobinu za čuvanje potrebnih stanja (*state*) i *render* metodu koja je zadužena za prikaz komponente. Za potrebe demonstriranja rada s komponentama, prethodno kreiranom projektu, unutar *src* foldera, potrebno je dodati novi fajl pod nazivom *predmeti.jsx*¹⁵ sa sljedećim sadržajem:

```
import React, { Component } from 'react';

class Predmeti extends React.Component {
  state = { }
  render() {
    return <div><h2>Akademska 2021/22</h2>
      <p>Odaberite izborne predmete koje želite
slušati:</p></div>;
  }
}

export default Predmeti;
```

Na samom početku se iz *react* modula uvozi *React* objekat koji je neophodan za konverziju JSX sadržaja koji se vraća iz metode *render*, a ranije je pojašnjeno da se JSX sadržaj konvertuje u odgovarajuće pozive *React.createElement* metoda. Kao što je spomenuto u primjerima s *Ember*-om, nasljeđivanje se realizuje korištenjem ključne riječi *extends*, te je zbog toga i uvezena klasa *Component*. Komponente i drugi dijelovi koda koji se logički odnose na određenu funkcionalnost ili cjelinu se obično posmatraju i organizuju u okviru *modula*. Također, modul se posmatra kao samostalna jedinica aplikacije koja svoje dijelove ili komponente, putem ključne riječi *export*, može učiniti dostupnim drugim modulima, odnosno referencirati dijelove drugih modula putem ključne riječi *import*. U prethodnom primjeru komponenta *Predmeti* je označena dostupnom za druge module, te će njeno korištenje, npr.

¹⁵ Preporuka je da se komponente logički organizuju unutar odgovarajućeg foldera, najčešće pod nazivom *components*

unutar *index.js* fajla, prvenstveno zahtijevati referenciranje a nakon toga i proslijeđivanje *render* metodi koja će se pobrinuti za njen prikaz.

```
// ...
import Predmeti from './components/predmeti';
// ...
ReactDOM.render(<Predmeti/>, document.getElementById('root'));
```

Ukoliko bi se željela kreirati komponenta koja će korisniku omogućiti selekciju izbornih predmeta, unutar *state*-a će biti dodata i inicijalizovana nova osobina odnosno niz objekata koji opisuju predmete.

```
class Predmeti extends React.Component {
  state = {
    predmeti: [
      { Id: 1, Naziv: 'React.js programiranje', Semestar: 2 },
      { Id: 2, Naziv: 'Web razvoj i dizajn', Semestar: 3 },
      { Id: 3, Naziv: 'Embedded i RT sistemi', Semestar: 4 }
    ]
  };
  render() {
    return <div><h2>Akademska 2021/22</h2>
      <p>Odaberite izborne predmete koje želite slušati:</p>
      {
        this.state.predmeti.map(predmet => (
          <div>
            <label>
              <input type="checkbox" id={predmet.Id}/>
              {predmet.Naziv}
            </label>
          </div>
        ))
      }
    </div>
  };
} ;
```

Osobinu *state* je moguće referencirati s *this.state* pri čemu *this*, kao i u drugim programskim jezicima, ukazuje na objekat koji je pozvao metodu. Prateći definisani mehanizam, predmeti koji se trebaju ispisati se referenciraju navođenjem

this.state.predmeti. Dostupna lista objekata, bez obzira da li je ona statička ili potječe iz nekog drugog izvora, je iskorištena unutar *render* metode kako bi se njeni članovi, korištenjem metode *map*, ispisali korisniku. Metoda *map*, za svakog člana niza, kreira novi *checkbox* element kojim se, opisan nazivom predmeta, korisniku omogućuje odabir.

Analizirajući prethodni primjer može se uočiti upozorenje koje se prikazuje u konzoli alata *web* preglednika¹⁶, a odnosi se na potrebu da se svakom članu liste dodijeli jedinstvena vrijednost svojstva *key* koji se koristi od strane *Reacta*¹⁷ za referenciranje pojedinih elemenata, te njena vrijednost nije dostupna unutar drugih komponenti. Posebno u slučajevima kada im se dodijele iste vrijednosti, veoma često se pravi asocijacija između osobina *id* i *key* pri čemu ne treba zaboraviti njihovu (ne)dostupnost u različitim kontekstima. Zbog svega navedenog, elementu *checkbox* iz ranijeg primjera je potrebno inicijalizovati vrijednost osobine *key*.

Svaka komponenta u *React*-u posjeduje specijalnu osobinu pod nazivom *props* (skraćenica od *properties*) koja omogućava da se prilikom instanciranja kontrole proslijede željene vrijednosti. Spomenuta osobina će biti iskorištena za dinamičko definisanje vrijednosti *naslov* i *tekst* koji se prikazuju neposredno iznad liste predmeta dostupnih za odabir, kao što je to bio slučaj i kod unutar *Ember* komponente.

```
ReactDOM.render(<Predmeti
                    naslov="Akademska 2021/22"
                    tekst="Odaberite izborne predmete koje
želite slušati:"/>,
                    document.getElementById('root'));
```

Pristup proslijeđenim vrijednostima se vrši na identičan način kao i *state* osobini, te su one iskorištene unutar *render* metode.

```
//...
render() {
  return <div><h2>{this.props.naslov}</h2>
    <p>{this.props.tekst}</p>
```

¹⁶ Većina web preglednika u podrazumijevanoj instalaciji, bez naknadno instaliranih komponenti, dolazi sa određenim setom alata koji programerima olakšavaju debugiranje aplikacija. Pomenuti alati su najčešće dostupni pritiskom na tipku F12.

¹⁷ Inicijalizacija svojstva *key* je neophodna i u okviru drugih JavaScript razvojnih okruženja

```
//...
```

Glavna razlika između *state* i *props* osobine je u tome što *state* predstavlja privatni segment koji nije dostupan nikome osim članovima komponente. S druge strane, *props* se može posmatrati kao transportni mehanizam kojim omogućava inicijalizaciju vrijednosti s lokacije instanciranja objekta. Također, jedna od značajnijih karakteristika *props* osobine je da se njegovi članovi, nakon inicijalizacije, mogu samo čitati a nikako modifikovati. Ispis komponente *Predmeti*, nakon napravljenih izmjena, bi trebao izgledati kao na Ilustracija 46.

Akadska 2021/22

Odaberite izborne predmete koje želite slušati:

- ☐ React.js programiranje
- ☐ Web razvoj i dizajn
- ☐ Embedded i RT sistemi

Ilustracija 46 - Prikaz komponente *Predmeti*

U prethodnom dijelu potpoglavlja je kreirana jedna vrlo jednostavna komponenta koja je zadužena za prikaz podataka o predmetima. Glavna snaga komponenti upravo leži u činjenici da se one mogu kombinovati i na taj način formirati mnogo kompleksnije strukture pa čak i kompletne funkcionalnosti. U nastavku će biti kreirana komponenta *AkadskaGodina* koja će imati zadatak da studentu omogući odabir predmeta koje će slušati tokom određene akademske godine. Na samom početku će biti kreiran novi fajl pod nazivom *npp2018.jsx*¹⁸ koji će sadržavati listu svih dostupnih predmeta u nastavanom planu i programu. Naravno, u cilju održanja preglednosti navedeno je svega nekoliko predmeta iz nastavnog plana i programa. Zahvaljujući ključnoj riječi *export*, predmeti postaju dostupni drugim modulima unutar aplikacije.

```
export const Npp2018 = [  
  { Id: 1, Naziv: 'Racunarstvo i informatika', Semestar: 1 },  
  { Id: 2, Naziv: 'Programiranje I', Semestar: 1 },  
  { Id: 3, Naziv: 'Inžinjerska matematika', Semestar: 1 },
```

¹⁸ Fajlovi ovog tipa se obično organizuju u zasebne foldere, te je novokreirani fajl potrebno pohraniti u folder *data* ili *podaci*

```

    { Id: 4, Naziv: 'Programiranje II', Semestar: 2 },
    { Id: 5, Naziv: 'React.js programiranje', Semestar: 2 },
    { Id: 6, Naziv: 'Web razvoj i dizajn', Semestar: 3 },
    { Id: 7, Naziv: 'Embedded i RT sistemi', Semestar: 4 }
  ];

```

Komponenti *Predmeti* je dodijeljen određeni dio zaduženja koje bi sada trebale pripasti komponenti *AkadskaGodina*, te će zbog toga biti kreirana nova komponenta *Predmet* (potrebno je kreirati novi fajl pod nazivom *predmet.jsx*) koja će se brinuti o prikazu podataka o samo jednom predmetu. Naravno, definicija ove komponente bi u realnijem kontekstu sadržavala mnogo više detalja, te će se zbog potrebe unapređenja u odnosu na raniji format ispisa, korištenjem komande prikazane u nastavku, dodati *Bootstrap* biblioteka.

```
PS C:\Users\User\React\Intro\react-book-app> npm install react-bootstrap
```

U cilju demonstracije inicijalizacije vrijednosti unutar osobine *state*, komponenta *Predmet* će putem *props*-a dobiti objekat odnosno *predmet* kojeg je potrebno ispisati. Unutar metode *render* noviteti se odnose na korištenje ključnih riječi *className* umjesto *class*, te *htmlFor* umjesto *for*. Također, kontrolama je dodijeljena odgovarajuća *Bootstrap* klasa kako bi se dobio malo drugačiji prikaz.

```

import React, { Component } from 'react';
import 'bootstrap/dist/css/bootstrap.min.css';

class Predmet extends React.Component {
  state = {
    predmet : this.props.predmet
  };
  render() {
    return (
      <div className="form-check">
        <input className="form-check-input"
          type="checkbox" key={this.state.predmet.Id}
          id={this.state.predmet.Id} />
        <label htmlFor={this.state.predmet.Id}>
          {this.state.predmet.Naziv}
          (Semestar: {this.state.predmet.Semestar})
        </label>
      </div>
    );
  }
}

```

```

    )
  };
} ;
export default Predmet;

```

Komponenta *AkademaskaGodina* će, neposredno iznad ponuđene liste predmeta, preuzeti obavezu ispisa akademske godine i adekvatne poruke koji su proslijeđeni putem osobine *props*. Također, unutar ove komponente će se čuvati podatak o semestrima, a te vrijednosti će biti korištene za filtriranje predmeta koji se izvode u željenom semestru, a nalaze se u nizu *Npp2018*. Prije nego što bude definisana komponenta *AkademaskaGodine*, a kako bi se stekao dojam o implementaciji komponente, u nastavku je prikazan način njenog instanciranja tj. ustupanja podataka koji će biti iskorišteni za potrebe filtriranja i prikaza.

```

ReactDOM.render(<AkademaskaGodina
    semestarOD = "1" semestarDO = "2" naslov =
    "Akademaska 2021/22"
    tekst = "Odaberite predmete koje želite slušati:"/>,
    document.getElementById('root'));

```

Nakon što se komponenti *AkademaskaGodina* proslijedi opseg semestara, čija bi razlika vrijednosti trebala biti manja ili jednaka od 1, filtriranje predmeta *p* iz liste *Npp2018* se vrši korištenjem metode *filter* unutar koje se navodi uslov prema kome semstar, u kojem se predmet izvodi, mora biti u opsegu definisanom vrijednostima *semstarOD* i *semestarDO*.

```

Npp2018.filter(p => p.Semestar >= this.state.semestarOD
    && p.Semestar <= this.state.semestarDO)

```

Filtriranjem predmeta stvaraju se preduslovi za mapiranje¹⁹ koje podrazumijeva da se za svaki objekat, nastao kao rezultat filtriranja, generiše zasebna komponenta *Predmet* kojoj se proslijeđuje *predmet* odnosno objekat kojeg je potrebno ispisati. Također, svakoj komponenti *Predmet* se, pored osobine *id*, dodjeljuje vrijednost i osobini *key*.

```

map(predmet => <Predmet id={predmet.Id} key={predmet.Id}
    predmet={predmet}/>)

```

¹⁹ Metoda *map* se izvršava na povratnoj vrijednosti metode *filter* tj. nad objektima koji su zadovoljili postavljeni uslov

Povezivanjem svega prethodno opisanog u jednu cjelinu, u nastavku je prikazana definicija komponente *AkademskiGodina* koja se nalazi unutar fajla *AkademskiGodina.jsx*.

```
import React, { Component } from 'react';
import { Npp2018 } from '../data/npp2018';
import Predmet from './predmet';

class AkademskiGodina extends React.Component {
  state = {
    semestarOD : this.props.semestarOD,
    semestarDO : this.props.semestarDO,
    opis : this.props.opis,
    tekst : this.props.tekst
  }
  render() {
    return (<div>
      <h2>{this.state.opis}</h2>
      <p>{this.state.tekst}</p> {
        Npp2018.filter(p => p.Semestar >=
this.state.semestarOD
                                && p.Semestar <=
this.state.semestarDO)
                                .map(predmet=><Predmet id={predmet.Id}
key={predmet.Id}
                                predmet={predmet}/>)
      } </div>)
  }
}
export default AkademskiGodina;
```

Podaci o predmetima, odnosno fajl *npp2018.jsx* se nalazi u folderu *data*, što naravno nije nikakva obaveza već ukazuje na potrebu modifikovanja rute ukoliko se tokom implementacije koristila neka druga lokacija.

Kao što je u više navrata do sada naglašeno, kombinujući dvije ili više komponenti moguće je kreirati kompleksnije interfejske koji se, zahvaljujući modularnosti, mogu na jednostavan način proširivati i održavati. Za kraj upoznavanja s *React*-om, prethodni primjer će biti proširen dodavanjem metode koja će biti zadužena za

evidenciju predmeta koje je student odabrao. Pošto je komponenta *Predmet* zadužena za ispis podataka o predmetu, prilikom njenog instanciranja bit će proslijeđena referenca na metodu *modifikujListuOdabranih* koju je potrebno pozvati, u komponenti *AkadskaGodina*, prilikom odabira nekog od predmeta. Proslijeđivanje reference će se izvršiti korištenjem svojstva *props* koji se naziva *naOdabir*.

```
.map(predmet=><Predmet id={predmet.Id} key={predmet.Id}  
    predmet={predmet}  
    naOdabir = {this.modifikujListuOdabranih} />)
```

Pošto je referenca na metodu, koja je implementirana unutar komponente *AkadskaGodina*, proslijeđena komponenti *Predmet*, ona će biti iskorištena za inicijalizaciju *onClick* događaja u elementu *checkbox*. Inicijalizacija je izvršena korištenjem *inline* funkcije koja će, klikom na *checkbox*, pozvati metodu proslijeđenu unutar svojstva *naOdabir* i tom prilikom joj proslijediti *predmet* koji je asociran sa komponentom tj. *predmet* koji je proslijeđen prilikom instanciranja te komponente (predmet je nakon instanciranja pohranjen u okviru *state*-a komponente).

```
<input className = "form-check-input" type = "checkbox"  
    id = {this.state.predmet.Id} key = {this.state.predmet.Id}  
    onClick={ () => this.props.naOdabir (this.state.predmet)}  
/>
```

Kompletna logika posljednjeg primjera je sadržana u okviru metode *modifikujListuOdabranih* koja treba osigurati manipulaciju nizom odabranih predmeta. Za potrebe čuvanja odabranih predmeta u okviru *state* osobine komponente *AkadskaGodina* je dodat niz pod nazivom *odabraniPredmeti*.

```
state = {  
    //...  
    tekst: this.props.tekst,  
    odabraniPredmeti: []  
}
```

Zahvaljujući spomenutom nizu stvorene su pretpostavke za ispis odabranih predmeta neposredno ispod liste ponuđenih. Treba naglasiti da implementacija u nastavku nije najoptimalnije rješenje, ali je s aspekta jednostavnosti i količine obrađene materije najpogodnija u ovom momentu. Naime, metoda

modifikujListuOdabranih kao parametar prihvata *predmet* koji je korisnik odabrao, te prvo provjerava da li *predmet* već postoji u listi odabranih. Provjera se vrši kreiranjem novog niza pod nazivom *filtriraniPredmeti* koji sadrži sve predmete osim onoga koji je trenutno proslijeđen. Zahvaljujući novokreiranom nizu, jedan od načina da se provjeri da li je predmet već dodat jeste provjera broja elemenata oba niz. Ako je broj elemenata u nizovima isti, to znači da *predmet* ne postoji u nizu odabranih predmeta, te se on, korištenjem metode *push*, dodaje u niz *filtriraniPredmeti*. Na kraju, bez obzira da li je predmet dodat (metoda *push*) ili uklonjen (metoda *filter*) korištenjem metode *setState*, vrijednost niza *odabraniPredmeti* se postavlja na *filtriraniPredmeti* jer je kompletna manipulacija (*filter* i *push*) vršena nad njegovim članovima.

```
modifikujListuOdabranih = predmet => {  
    let filtriraniPredmeti = this.state.odabraniPredmeti  
        .filter(p => p !==  
predmet);  
    if(filtriraniPredmeti.length===this.state.odabraniPredmeti.lengt  
h)  
        filtriraniPredmeti.push(predmet);  
    this.setState(this.state.odabraniPredmeti =  
filtriraniPredmeti);  
};
```

Mnogo adekvatnija, ali i nešto kompleksnija, implementacija bi zahtijevala reagovanje na događaje u kojima je *checkbox* element (ne)označen, te na osnovu toga vršiti uklanjanje ili dodavanje predmeta. Također, uklanjanje članova niza se moglo realizovati korištenjem metode *splice*.

Kako bi se odabrani predmeti mogli prikazati potrebno je napraviti i određene modifikacije u *render* metodi na način da se nakon liste predmeta filtriranih iz niza *Npp2018* prikažu i članovi niza *odabraniPredmeti*. U nastavku je prikazan samo sadržaj koji je dodat *render* metodi, dok je postojeći ostao nepromijenjen. Ispis odabranih predmeta je formatiran korištenjem *card* i *list* CSS klasa.

```
//...  
<div className="card mt-3 col-sm-3">  
    <div className="card-header">  
        Odabrani predmeti:  
    </div>  
    <ul className="list-group list-group-flush">  
    {
```

```

        this.state.odabraniPredmeti.map(predmet => (
            <li className="list-group-item"
                key={predmet.Id}> {predmet.Naziv}</li> ))
        )
    </ul>
</div>
//...

```

Nakon svih opisanih modifikacija, kombinacija komponenti *AkadskaGodina* i *Predmet* bi trebala rezultirati prikazom kao na ilustracija 47, tačnije lijeva slika na spomenutoj ilustraciji. Desna slika se razlikuje samo po dodatku *form-switch* u nazivu CSS klase. Mada se, funkcionalno, dobija isti rezultat, odabir adekvatnih elemenata s kojima će korisnici ostvarivati interakciju bi trebao pratiti dokazane prakse koje će biti ukratko opisane u narednom, ujedno posljednjem, poglavlju ove knjige.

Akadska 2021/22

Odaberite predmete koje želite slušati:

- ☐ **Racunarstvo i informatika** (Semestar: 1)
- ☒ **Programiranje I** (Semestar: 1)
- ☐ **Inžinjerska matematika** (Semestar: 1)
- ☐ **Programiranje II** (Semestar: 2)
- ☒ **React.js programiranje** (Semestar: 2)

Odabrani predmeti:
Programiranje I
React.js programiranje

Akadska 2021/22

Odaberite predmete koje želite slušati:

- ☐ **Racunarstvo i informatika** (Semestar: 1)
- ☒ **Programiranje I** (Semestar: 1)
- ☐ **Inžinjerska matematika** (Semestar: 1)
- ☐ **Programiranje II** (Semestar: 2)
- ☒ **React.js programiranje** (Semestar: 2)

Odabrani predmeti:
Programiranje I
React.js programiranje

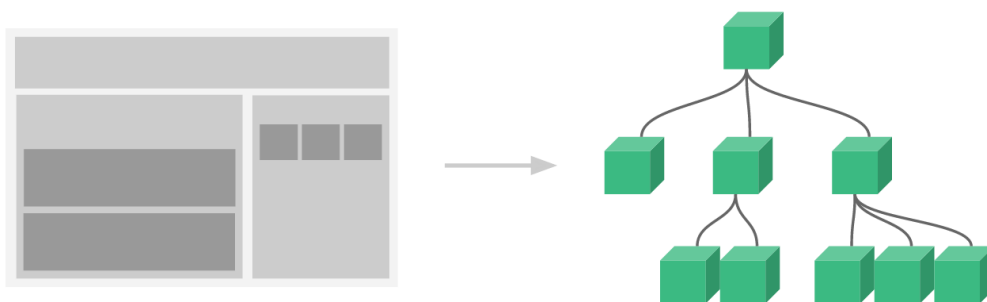
Ilustracija 47 - Prikaz ponuđenih i odabranih predmeta korištenjem komponenti u React-u

Provjeru stepena razumijevanja osnovnih koncepata *Reacta* moguće je izvršiti nadogradnjom prethodno opisanog primjera na način da se odabrani predmeti također prikazuju koristeći komponentu *Predmet* ili čak neku novu komponentu koja će omogućiti korisniku da ukloniti predmet iz liste odabranih.

U narednom dijelu poglavlja će biti detaljnije opisan *Vue.js* kao jedna od *JavaScript* razvojnih okvira koja svakim danom stječe sve veću popularnost u razvoju aplikacija.

4.3 Vue.JS

Područje razvoja aplikacija kao i očekivanja korisnika postaje sve zahtjevnije, što direktno utječe i na količinu *JavaScript* koda koja se nalazi unutar razvojnih okvira. Spomenuto postaje ne tako zanemariv problem, ali se u tom smislu ističe, kao jedan od „najlakših“ razvojnih okvira, *Vue.JS* (u nastavku *Vue*) kojeg je 2014. godine kreirao Evan You. Od samog početka je označen kao progresivan, što podrazumijeva da se u bilo kojoj fazi razvoja može uključiti u projekat, te progresivno koristiti u preostalom dijelu implementacije. Kao i ranije pomenuti razvojni okviri, *Vue* se bazira na komponentama sačinjenih od osnovnih gradivnih blokova *HTML*, *CSS* i *JavaScripta*, te su zahvaljujući tome u stanju egzistirati kao zasebne cjeline. Kao što se može vidjeti na Ilustracija 48 jedna aplikacija bi mogla biti sastavljena od tri komponente (npr. zaglavlje, glavni dio, meni), pri čemu bi svaka komponenta mogla imati komponente djecu koji će služiti za realizaciju specifičnih funkcionalnosti npr. ispisa kursne liste, nadolazećih ispita, novoobjavljenih predmeta i sl.



Ilustracija 48 - Komponente u kontekstu razvoja *Vue* aplikacija²⁰

Osobina razvojnih okvira, bez obzira da li se radilo o *JavaScript* ili nekom drugom programskom jeziku, je da omogućava više različitih načina za interakciju tj. njihovo uključivanje u projekat. Prilikom upoznavanja s *Ember* i *React* razvojnim okvirima izvršena je instalacija svih komponenti neophodnih za razvoj željene aplikacije. Na kraju ovog poglavlja će biti pojašnjena procedura instalacije svih komponenti neophodnih za razvoj *Vue* aplikacija, ali će primjeri koda biti kreirani na nešto drugačiji način. Naime, u cilju demonstracije dostupnih opcija, u nastavku će biti korišten CDN (*Content Delivery Network*) pristup koji podrazumijeva činjenicu da kompanije javno publiciraju potrebne fajlove kako bi oni bili dostupni korisnicima za

²⁰ <https://vuejs.org/v2/guide/components.html>

referenciranje. Drugim riječima, zahvaljujući CDN-u, kreiranje *Vue* reference podrazumijeva jednostavno dodavanje *script* taga s odgovarajućom putanjom.

```
<script  
src="https://cdn.jsdelivr.net/npm/vue@2.6.14/dist/vue.js"></script>
```

Prethodno naveda putanja se obično koristi za potrebe razvoja i testiranja, dok je na drugoj putanji dostupna stabilna verzija koja se preporučuje za produkcijsko okruženje.

4.3.1 Razvoj *Vue* aplikacija

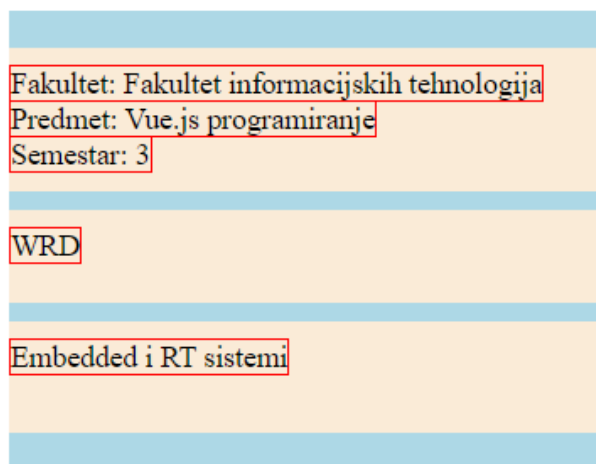
Nakon referenciranja, dovoljno je instancirati *Vue* objekat koji bi se trebao povezati s nekim od elemnta unutar HTML dokumenta. Za potrebe demonstracije, zadržavajući se na istoj tematici kao i prilikom predstavljanja drugih razvojnih okvira, će biti korišten HTML kod iz Ilustracija 31. U željenom direktoriju kreirajte fajl *index.html* i unutar njega dodajte sljedeći sadržaj.

```
<head>  
  <style>  
    /*...*/  
  </style>  
  <script  
src="https://cdn.jsdelivr.net/npm/vue@2.6.14/dist/vue.js"></script>  
</head>  
<body>  
  <div class="wrapper">  
    <div class="prvi" id="divPredmeti">  
      <span>Fakultet: {{fakultet}}</span><br/>  
      <span>Predmet: {{predmet}}</span><br/>  
      <span>Semestar: {{semestar}}</span><br/>  
    </div>  
    <div class="drugi">  
      //...  
    </div>  
  </div>  
</body>  
  
<script>  
  new Vue(  
    {  
      el: '#divPredmeti',  
      data: {  
        fakultet: 'Fakultet informacijskih tehnologija',  
        predmet: 'Vue.js programiranje',  
        semestar: 3  
      }  
    }  
  })
```

```
</script>
```

Instanciranje *Vue* objekta se vrši putem izraza *new Vue*, te se kao parametar proslijeđuje objekat koji posjeduje set predefinisanih osobina. Prije svega, *el* koji predstavlja skraćenicu od *element*, a omogućava referenciranje HTML elementa koji će predstavljati kontejner za izvršenje kreiranog objekta, kao što je to bio *root* element unutar primjera *React* aplikacije. U okviru *data* sekcije, koja zasigurno podsjeća na *state* sekciju *React* komponenti, su kreirane osobine (*fakultet*, *predmet* i *semestar*) čije vrijednosti će biti dostupne unutar referenciranog HTML elementa. Na osnovu prethodnog primjera, a što je također bio slučaj i u ranijim razvojnim okvirima, interakcija s pojedinim osobinama, u okviru *divPredmeti*, zahtijeva jednostruke ili dvostruke vitičaste zagrade.

Prikazivanjem prethodno kreiranog HTML dokumenta u *web* pregledniku trebalo rezultirati prikazom kao na Ilustracija 49. Svi stilovi i HTML kod iz Ilustracija 31 su zadržani, ali zbog preglednosti, te činjenice da su ranije detaljno objašnjeni, nisu ponovo prikazani u primjeru koda koji producira Ilustracija 49.



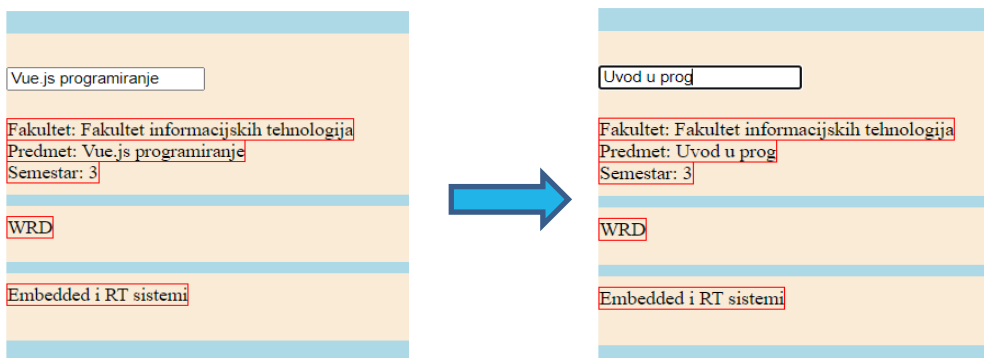
Ilustracija 49 - Pristup osobinama *Vue* objekta

Prethodni primjer demonstrira jednostavnost načina na koji se iz *Vue* razvojnog okvira može izvršiti modifikacija neke od vrijednosti HTML elemenata. Ništa kompleksnija nije ni procedura u okviru koje je moguće uspostaviti dvosmjernu komunikaciju između određenog HTML elementa i nekod od osobina *Vue* objekta. Pod dvosmjernošću se podrazumijava mogućnost da se promjena vrijednosti HTML elementa automatski reflektuje na vrijednost osobine i obrnuto.

U narednom primjeru, Ilustracija 50, korisniku je omogućeno da izvrši modifikaciju naziva predmeta pri čemu je inicijalni naziv, kako osobine tako i input polja, postavljen na vrijednost *Vue.js programiranje*. Modifikacijom vrijednosti bilo kojeg aktera uzrokuje promjene na obje strane. Promjenom inicijalne vrijednosti i unosom sadržaja *Uvod u prog* automatski je izvršena modifikacija vrijednosti osobine predmet unutar *span* elementa.

```
<!-- ... -->
<div class="wrapper">
  <div class="prvi" id="divPredmeti">
    <div>
      <!--direktiva v-model se povezuje sa nazivom osobine Vue.js
objekta -->
      <input type="text" v-model="predmet" />
    </div>
    <span>Fakultet: {{fakultet}}</span><br />
    <span>Predmet: {{predmet}}</span><br />
    <span>Semestar: {{semestar}}</span><br />
  </div>
<!-- ... -->
</div>
<!-- ... -->
```

Dvosmjernost je osigurana korištenjem direktive *v-model* koja se inicijalizuje vrijednošću osobine s kojom treba obavljati interakciju.



Ilustracija 50 - Modifikacija vrijednosti HTML elemenata putem Vue objekta

Korištenjem konzole unutar razvojnog alata *web* preglednika moguće je na jednostavan način modifikovati vrijednost *Vue* objekta i na taj način testirati dvosmjernu komunikaciju *HTML* elementa i osobine objekta.

Povezivanje se može ostvariti i s većim brojem elemenata, te je u tom smislu, unutar *Vue* objekta, potrebno kreirati niz koji će, u zavisnosti od vrste elemenata, čuvati

potrebne vrijednosti. U narednom primjeru će biti uklonjeni nepotrebni dijelovi HTML-a, a korisniku će se omogućiti odabir predmeta koje želi slušati u tekućem semestru. Za potrebe implementacije spomenute funkcionalnosti kreirana su dva niza, od kojih prvi sadrži listu dostupnih (niz *dostupniPredmeti*), a drugi listu odabranih predmeta (niz *odabraniPredmeti*).

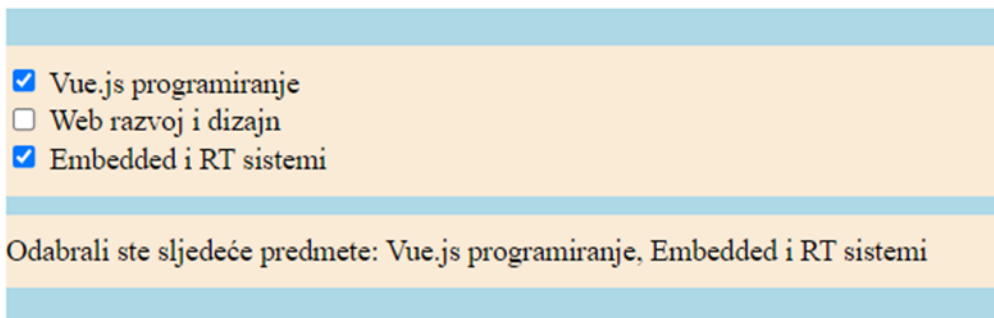
```
<!-- ... -->
<body>
  <div class="wrapper" id="divPredmeti">
    <div class="prvi">
      <input type="checkbox" id="vue" value="Vue.js
programiranje" v-model="odabraniPredmeti" />
      <!--koristi se prvi clan niza dostupniPredmeti -->
      <label for="vue">{{dostupniPredmeti[0]}}</label> <br />

      <input type="checkbox" id="wrđ" value="Web razvoj i dizajn"
v-model="odabraniPredmeti" />
      <label for="wrđ">{{dostupniPredmeti[1]}}</label> <br />

      <input type="checkbox" id="erts" value="Embedded i RT
sistemi" v-model="odabraniPredmeti" />
      <label for="erts">{{dostupniPredmeti[2]}}</label> <br />
    </div>
    <div>
      <!-- sve vrijednosti tj. clanovi niza odabraniPredmeti se
povezuju u jedan string odvojen zarezom -->
      Odabrali ste sljedeće predmete: {{odabraniPredmeti.join(',
')}}
    </div>
  </div>
  <script>
    new Vue(
      {
        el: '#divPredmeti',
        data: {
          dostupniPredmeti: [ 'Vue.js programiranje', 'Web
razvoj i dizajn',
                                'Embedded i RT sistemi'],
          odabraniPredmeti: []
        }
      })
  </script>
</body>
<!-- ... -->
```

U prethodnom primjeru je fiksiran broj dostupnih predmeta, te se oni ispisuju koristeći indeks ili lokaciju unutar niza. Nakon ponuđene liste, korisniku se ispisuje sadržaj niza *odabraniPredmeti* koji je inicijalno prazan, a njegovi članovi se dodaju i uklanjaju svakom interakcijom s *checkbox* elementa. Ispis odabranih predmeta se

vrši korištenjem metode *join*, kojoj se kao argument šalje separator što je u konkretnom primjeru znak zarez.



The screenshot shows a web interface with a list of subjects. Three subjects are listed with checkboxes: 'Vue.js programiranje' (checked), 'Web razvoj i dizajn' (unchecked), and 'Embedded i RT sistemi' (checked). Below the list, a summary line states: 'Odabrali ste sljedeće predmete: Vue.js programiranje, Embedded i RT sistemi'.

Ilustracija 51 - Pohrana podataka nastalih interakcijom sa korisnikom

Pored *v-model*, na raspolaganju je čitavi set direktiva koje omogućavaju povezivanje određenih događaja s osobinama objekta ili drugim akcijama. Kao primjer se mogu navesti slučajevi u kojima bi klik na određeni element trebao pokrenuti izvršenje akcije, te je za te potrebe na raspolaganju *v-on* direktiva koja je u stanju prepoznati da se desio neki DOM događaj. Ukoliko se govori o klik događaju, onda bi se *v-on* direktiva, u okviru *checkbox* elementa, mogla iskoristiti na sljedeći način:

```
<input type="checkbox" id="vue" value="Vue.js programiranje" v-model="odabraniPredmeti" v-on:click="{brojac++}"/>
```

U prethodnom primjeru, svaka promjena stanja elementa *checkbox* će uzrokovati uvećavanje vrijednosti osobine *brojac* koja je dodata *data* sekciji *Vue* objekta, a koja će biti iskorištena za informativni prikaz broja promjena stanja. Pored *v-on:click* dostupna je i skraćena verzija *@click*.

```
<!-- ... -->
<body>
<!-- ... -->
    <input type="checkbox" id="vue" value="Vue.js
programiranje" v-model="odabraniPredmeti" @click="{brojac++}" />
    <!-- ... -->
    Broj promjena stanja: <b>{{brojac}}</b>
</div>
</div>
<script>
    //...
    data: {
        brojac: 0,
        //...
    }
}
```

```

    })
  </script>
<!-- ... -->

```

Uvećavanje vrijednosti se ne čini pretjerano značajnom akcijom, ali je bitno shvatiti način interakcije koji je gotovo identičan za bilo koju drugu vrstu događaja. U slučaju da se javi potreba za promjenom stila prikaza određenih elemenata, u nastavku su, prvenstveno zbog referenciranja primjera iz ranijih poglavlja knjige, kreirane dvije CSS klase koje će biti dodijeljene *checkbox* elementima u zavisnosti od njihovog stanja.

```

.odabran { color: blue; font-weight: bold; }
.nijeOdabran { color: black; font-style: italic; }

```

Mada se provjera može ostvariti na više načina, u cilju praćenja stanja pojedinih *checkbox* elemenata, koji predstavljaju predmete, će biti dodat niz *odabraniStatus* čije će vrijednosti inicijalno biti postavljene na *false*.

```

odabraniStatus: [false, false, false]

```

Promjena vrijednosti unutar niza *odabraniStatus* će se vršiti prilikom svakog klika na neki od *checkbox* elemenata što je osigurano negacijom postojeće vrijednosti. Također, zadržano je i uvećavanje osobine *brojac* prilikom svake interakcije.

```

@click="odabraniStatus[0] = !odabraniStatus[0], brojac++"

```

Zahvaljujući ranije dodatim elementima moguće je ostvariti dodjelu određene klase kako bi se osigurao željeni stil, a za to je na raspolaganju direktiva *v-bind:class* ili skraćena verzija *:class*.

```

<label for="vue" :class="[odabraniStatus[0] ? 'odabran' :
'nijeOdabran']">{{dostupniPredmeti[0]}}</label> <br />

```

Određivanje koja od klasa će biti dodata je prepušteno uslovnom operatoru koji provjerava status elementa u nizu *odabraniStatus* i na osnovu toga dodijeljuje odgovarajuću klasu.

Stilovi se mogu definisati i kao osobine objekta, te se kao takvi dodijeliti elementu u zavisnosti od njegovog stanja. U nastavku su objektu dodate dvije osobine tj. objekta koje su iskorištene unutar *:style* atributa.

```

<!-- ... -->
<style>

```

```

        /*...*/
        .odabran { color: blue; font-weight: bold; }
        .nijeOdabran { color: black; font-style: italic; }
    </style>
<!-- ... -->
<body>
    <div class="wrapper" id="divPredmeti">
        <div class="prvi">
            <input type="checkbox" id="vue" value="Vue.js
programiranje"
                    v-model="odabraniPredmeti"
                    @click="odabraniStatus[0] = !odabraniStatus[0],
brojac++"/>
            <label for="vue"
                    :class="[odabraniStatus[0] ? 'odabran' :
'nijeOdabran']">
                                {{dostupniPredmeti[0]}}</label> <br />

            <input type="checkbox" id="wrđ" value="Web razvoj i
dizajn"
                    v-model="odabraniPredmeti"
                    @click="odabraniStatus[1] = !odabraniStatus[1],
brojac++"/>
            <label for="wrđ"
                    :style="[odabraniStatus[1] ? odabran :
nijeOdabran]">
                                {{dostupniPredmeti[1]}} </label> <br />

            <input type="checkbox" id="erts" value="Embedded i RT
sistemi"
                    v-model="odabraniPredmeti"
                    @click="odabraniStatus[2] = !odabraniStatus[2],
brojac++" />
            <label for="erts"
                    :style="[odabraniStatus[2] ? odabran :
nijeOdabran]">
                                {{dostupniPredmeti[2]}}</label> <br />
        </div>
        <div>
            Odabrali ste sljedeće predmete:
            {{odabraniPredmeti.join(', ')}} <br />
            Broj promjena stanja: <b>{{brojac}}</b>
        </div>
    </div>
    <script>
        new Vue(
            {
                el: '#divPredmeti',
                data: {
                    brojac: 0,

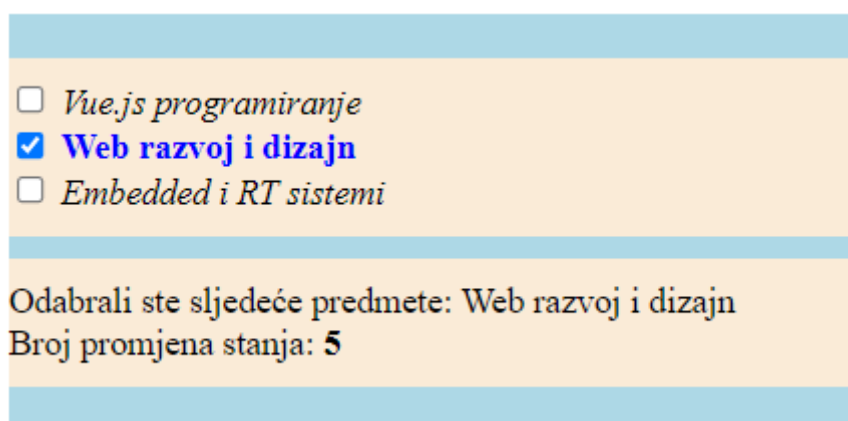
```

```

        dostupniPredmeti: ['Vue.js programiranje',
                           'Web razvoj i dizajn',
                           'Embedded i RT sistemi'],
        odabraniStatus: [false, false, false],
        odabraniPredmeti: [],
        odabran: { color: 'blue', fontWeight: 'bold' },
        nijeOdabran: { color: 'black', fontStyle:
'italic' }
    }
  })
</script>
</body>
<!-- ... -->

```

Za potrebe demonstracije, u slučaju prvog *combobox* elementa je zadržano korištenje CSS klasa definisanih unutar *style* taga, dok preostala dva *combobox* elementa koriste vrijednosti stilova definisane unutar objekta.



Ilustracija 52 - Upravljanje stilovima elemenata

U gotovo svim dijelovima ovog poglavlja je naglašeno da većina razvojnih okvira, pa tako i *Vue*, izuzetno veliki značaj poklanjaju mehanizmu izrade i korištenja komponenti. Komponente se mogu posmatrati kao *Vue* instance koje je, referencirajući njihov naziv, moguće koristiti i kombinovati u kompleksnije strukture. Osnova za kreiranje komponenti je poziv *component* metode prikazane u nastavku.

```

Vue.component('nazivKomponente',
  {
    metapodaci,
    ...
  }
);

```

Nakon kreiranja komponente, njen naziv postaje validan element koji se može koristiti kad god je neophodna implementirana funkcionalnost, a za potrebe demonstracije u nastavku je kreirana komponenta pod nazivom *predmet* koja će svaki put, prilikom instanciranja, ispisati sadržaj tj. naziv predmeta *Vue.js programiranje*. Ispisu je dodijeljen i određeni stil koji se za potrebe vježbe, a u kontekstu cijele komponente, treba zamijeniti odgovarajućom *Bootstrap* klasom.

```
<!-- ... -->
<body>
  <div id="divPredmeti">
    <predmeti></predmeti>
    <predmeti></predmeti>
  </div>
  <script>
    Vue.component('predmeti', {
      template: '<p style="color:blue; font-family:Helvetica">Vue.js programiranje</p>'
    });
    new Vue({ el: '#divPredmeti' })
  </script>
</body>
<!-- ... -->
```

Na osnovu primjera se može zaključiti da se unutar osobine *template* definiše struktura prikaza komponente. U *body* dijelu stranice, unutar elementa *divPredmeti*, komponenta *predmeti* se, navođenjem naziva, dva puta koristi, te se može očekivati da naziv predmeta bude dva puta ispisan. Predefinisanje naziva predmeta unutar komponente bi zasigurno narušilo njenu efikasnost i svrhu postojanja, te je u tom kontekstu na raspolaganju osobina *props*, koja je također intenzivno korištena pri radu s React komponentama, a koji omogućava definisanje niz osobina s kojim se može ostvariti interakcija prilikom instanciranja komponente. U narednom primjeru je definisana osobina *naziv* koja je proslijeđena s lokacije gdje se koristi komponenta.

```
<div id="divPredmeti">
  <predmeti naziv="Vue.js programiranje"></predmeti>
  <predmeti naziv="Web razvoj i dizajn"></predmeti>
<!-- ... -->
</div>
<script>
  Vue.component('predmeti', {
    props: ['naziv'],
    template: '<p style="color:blue; font-family:Helvetica">{{naziv}}</p>'
  })
</script>
```

```
});
new Vue({ el: '#divPredmeti' })
</script>
```

Prethodni primjer se na vrlo jednostavan način može proširiti i kreirati komponentu koja će omogućiti odabir predmeta koje student želi slušati. Umjesto klasičnog ispisa proslijeđenog naziva predmeta, struktura komponente sada definiše *checkbox* i *label* elemente koji će osigurati adekvatan opis i potrebnu funkcionalnost.

```
<div id="divPredmeti">
  <predmeti oznaka="vue" naziv="Vue.js
programiranje"></predmeti>
  <!-- ... -->
</div>
<script>
  Vue.component('predmeti', {
    props: ['oznaka', 'naziv'],
    template:
      '<div><input type="checkbox" :name="oznaka">
        <label>{{naziv}}</label></div>'
  });
  new Vue({ el: '#divPredmeti' })
</script>
```

Bitno je napomenuti da bi svaki *template* komponente trebao imati glavni ili korijenski (engl. *root*) element, kao što je u prethodnom primjeru iskorišten *div*. Spomenuto je karakteristično za većinu *JavaScript* razvojnih okvira, te će, nepostojanje korijenskog elementa, uzrokovati grešku.

Ukoliko sadržaj *template*-a prelazi u više redova onda je na početak i kraj sadržaja neophodno staviti znak ``` (ASCII: 96). Svi primjeri u nastavku su, samo zbog preglednosti, prikazani u više redova, ali je znak na početku sadržaja namijenjen za jedan red. Ukoliko se želi zadržati prikaz kao u primjerima, potrebno je zamijeniti spomenuti znak.

Rad s podacima unutar komponente se obično realizuje u okviru *data* funkcije koja se poziva svaki put prilikom instanciranja objekta, te obično vraća neku vrijednost bilo da se radi o jednom ili nizu objekata. U narednom primjeru, *data* segment vraća jedan objekt koji opisuje predmet, te pored naziva posjeduje osobine: *id*, *semestar* i *ime nastavnika*. *Template* dio je formatiran na način da prikazuje samo *naziv* vraćenog objekta, te bi se u prikazu trebao pojaviti samo sadržaj *Osnove programiranja*.

```
Vue.component('predmeti', {
  template: '<p>{{naziv}}</p>',
  data() {
    return {
      id: 1,
      naziv: "Osnove programiranja",
      semestar: 1,
      nastavnik: "prof.dr. Elmir Babovic"
    }
  }
});
```

Izvor podataka, bez obzira da li se direktno komunicira sa bazom podataka ili nekim od servisa, najčešće vraća niz ili listu koji je potrebno prikazati. U nastavku je prethodni primjer proširen kako bi ispisao nazive predmeta i imena nastavnika za sve predmete koji se izučavaju na FIT-u. Predmeti su organizovani unutar niza, te se za prikaz može koristiti *v-for* direktiva koja je dosta slična *foreach* petlji i ima sljedeću sintaksu:

```
v-for="predmetFIT in npp2018"
```

pri čemu *npp2018* predstavljaju dostupan niz podataka, koji u ovom primjeru vraća *data* funkcija, a *predmetFIT* ²¹ će biti naziv za svakog od članova niza tj. za svaki objekat koji se nalazi u nizu *npp2018*. Za pristup vrijednostima pojedinih objekata, članova niza, koristi se uobičajena logika tj. *nazivObjekta.nazivOsobine*. Za svaki od predmeta se, ranije korištenim pristupom, ispisuje samo naziv i ime nastavnika.

```
<div id="divPredmeti">
  <predmeti></predmeti>
</div>
<!-- ... -->
<script>
  Vue.component('predmeti', {
    template: '<div>
      <p v-for="predmetFIT in npp2018">
        {{predmetFIT.naziv}}-
        {{predmetFIT.nastavnik}}
      </p></div>',
    data() {
      return {
        npp2018: [
```

²¹ Mnogo adekvatniji naziv bi bio predmet, ali je on rezervisan za potrebe imenovanja nove komponente koja će biti kreirana u nastavku poglavlja

```

        { id: 1, naziv: "Osnove programiranja",
semestar: 1,
          nastavnik: "prof.dr. Elmir Babovic" },
        { id: 2, naziv: "Diskretna matematika",
semestar: 1,
          nastavnik: "prof.dr. Nina Bijedic" },
        { id: 3, naziv: "Operativni sistemi",
semestar: 1,
          nastavnik: "prof.dr. Senad Rahimic" }
      ]
    }
  });
  //...
</script>

```

Komponenta može kombinovati i druge elemente, te na taj način formirati mnogo učinkovitije i iskoristivije cjeline. U nastavku je prethodni primjer proširen na način se za svaki od predmeta kreira *checkbox* element koji omogućava selekciju predmeta koji se žele pratiti. U cilju eliminisanja nepotrebnog ponavljanja identičnih dijelova koda, u nastavku su naznačeni samo modifikovani dijelovi. Za vježbu inicijalizujte i druge vrijednosti osobina pojedinih elemenata kao što su *id*, *key* i dr.

```

template: '<div>
  <p v-for="predmetFIT in npp2018">
    <input type="checkbox" :name="predmetFIT.id">
      <label>{{predmetFIT.naziv}}-
{{predmetFIT.nastavnik}}</label>
    </p>
  </div>',

```

Proširenje u kontekstu vezivanja korisnikovog odabira za neki niz unutar komponente se može izvršiti korištenjem *v-model* direktive. Koja od vrijednosti će biti pohranjena prvenstveno zavisi od osobine koja se navede unutar *value* atributa. U konkretnom slučaju se pohranjuje kompletan odabrani objekat, jer je navedeno *:value="predmetFIT"*, a što se može vidjeti i prilikom ispisa niza *student.odabraniPredmeti*.

```

<script>
  Vue.component('predmeti', {
    template: '
<div>
  <h3>{{student.indeks}}</h3>
  <p v-for= "predmetFIT in npp2018" >
    <input type="checkbox"

```

```

        v-model="student.odabraniPredmeti"
        :value = "predmetFIT">
        <label>{{predmetFIT.naziv}} -
        {{predmetFIT.nastavnik}}</label>
        </p>{{student.odabraniPredmeti}}</p>
    </div > ',
    data() {
        return {
            student: {
                indeks: 'IB200002',
                odabraniPredmeti: []
            },
            npp2018: [
                { id: 1, naziv: "Osnove programiranja",
/*.....*/

```

Unutar *data* sekcije je dodat i novi objekat *student* unutar koga se pohranjuju broj indeksa i lista odabranih predmeta. Svaki put kada korisnik izvrši promjenu u selekciji ponuđenih predmeta, vrši se modifikacija sadržaja niza *odabraniPredmeti* kao i ispisa koji je osiguran na kraju *template*-a.

Prikaz i upravljanje podacima određenog entiteta, kao što je u prethodnom slučaju bio entitet *predmetFIT* iz niza *npp2018*, se može prepustiti zasebnoj komponenti koja će definisati sve detalje potrebne za prikaz i upravljanje podacima. U nastavku je kreirana nova komponenta koja se naziva *predmet*, a koja je iskorištena za prikaz svakog pojedinog predmeta te se referencira iz postojeće komponente *predmeti*²².

```

<!-->
<script>
    Vue.component('predmeti', {
        template: '<div><predmet v-for="predmetFIT in npp2018">
                //..
                </predmet>
                //..
            </div>',
        data() {
            return {
                student: {
                    indeks: 'IB200002',
                    odabraniPredmeti: []
                },
                npp2018: [

```

²² Umesto *predmeti*, adekvatniji naziv za komponentu bi bio, u duhu ranijih primjera, *AkadskaGodina*

```

        { id: 1, naziv: "Osnove programiranja",
/*...*/
    },
    },
});
Vue.component('predmet', {
    template: '<p>
        <label><slot></slot></label>
    </p>'
});
new Vue({ el: '#divPredmeti' })
//...

```

Interakcija između komponenti prvenstveno podrazumijeva razmjenu podataka koju će pomoćna komponenta *predmet* prikazati, a za tu namjenu je korišten *slot* element. Spomenuti element predstavlja lokaciju u okviru koje će biti prikazane vrijednosti proslijeđene prilikom instanciranja komponente *predmet*. Unutar *v-for* direktive se kreira *checkbox* element za svaki od predmeta, iz niza *npp2018*, koji će se ispisati u dijelu *slot* sekcije komponente *predmet*.

Ispis sadržaja *odabraniPredmeti*, Ilustracija 53, podrazumijeva ispis vrijednosti svih osobina objekta, što nije najadekvatnija opcija kada se uzme u obzir činjenica da će taj prikaz biti dostupan krajnjim korisnicima. Prilagođavanje ispisa se može izvršiti modifikovanjem vrijednosti *:value* atributa u *predmetFIT.naziv*, pri čemu će se pohranjivati samo naziv, a ne i kompletan predmet. Pored spomenute, a na osnovu do sada obrađenog, prilikom ispisa na raspolaganju je mogućnost korištenja *v-for* direktive što će omogućiti selekciju vrijednosti koje je potrebno ispisati.

IB200002

☒ Osnove programiranja - prof.dr. Elmir Babovic

☐ Diskretna matematika - prof.dr. Nina Bijedic

☐ Operativni sistemi - prof.dr. Senad Rahimic

```
[ { "id": 1, "naziv": "Osnove programiranja", "semestar": 1, "nastavnik": "prof.dr. Elmir Babovic" } ]
```

Ilustracija 53 - Odabir i ispis predmeta

4.3.2 Instalacija potrebnih Vue komponenti

Programski kod korišten u svim do sada predstavljenim *Vue* primjerima nalazio se unutar jednog fajla i bio je zasnovan na CDN-u. Pošto se radi o gotovo identičnoj

proceduri kao i kod Emer-a i React-a, uz manje razlike u komandama, u nastavku će biti ukratko opisana procedura instalacije komponenti neophodnih za razvoj *Vue* aplikacija. Pod uslovom da je *Node.js* prethodno instaliran, instalacija *Vue* CLI-a se vrši korištenjem sljedeće komande:

```
PS C:\Users\User\Vue\Intro> npm install -g @vue/cli
```

Jedan od načina za generisanje početne strukture projekta, kao na *Ember* primjeru, je korištenje *init* funkcije koju je također potrebno instalirati korištenjem sljedeće komande:

```
PS C:\Users\User\Vue\Intro> npm install -g @vue/cli-init
```

Kao što je ranije detaljnije opisano, sa CLI-om je postala dostupna kompletna infrastruktura neophodna za izvršenje aplikacije, počevši od *Babela*, *WebPacka* i dr. Za kreiranje novog projekta, pod nazivom *vue-book-app*, se neophodno pozicionirati na željenu lokaciju, te putem sljedeće komande, korištenjem *webpack* predloška, generisati početnu strukturu.

```
PS C:\Users\User\Vue\Intro> vue init webpack vue-book-app
```

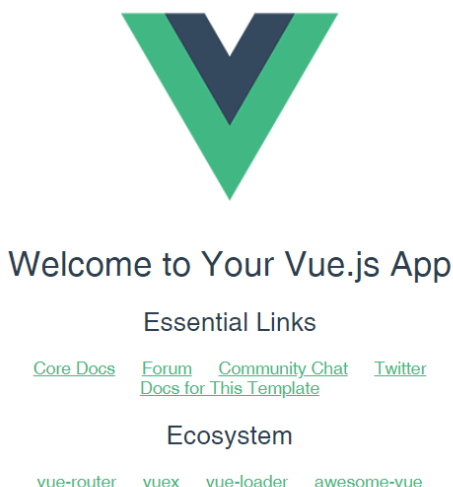
Procedura kreiranja projekta će podrazumijevati mini QA sesiju u okviru koje se na sva pitanja može odabrati ponuđena opcija ili odričan odgovor, a na posljednje se može odabrati korištenje *npm* opcije.

```
? Project name vue-book-app
? Project description A Vue.js project
? Author user <user@edu.fit.ba>
? Vue build standalone
? Install vue-router? No
? Use ESLint to lint your code? No
? Set up unit tests No
? Setup e2e tests with Nightwatch? No
? Should we run `npm install` for you after the project has been
created? (recommended) npm
```

```
vue-cli · Generated "vue-book-app".
```

Ukoliko je sve uspješno okončano, opisana procedura bi trebala rezultirati generisanjem osnovne strukture projekta koju je moguće pokrenuti korištenjem

komande `npm start` unutar foldera aplikacije. Podrazumijevana stranica projekta je dostupna na adresi <http://localhost:8080/>, a njen izgled bi trebao biti kao na Ilustracija 54.



Ilustracija 54- Izgled početne stranice Vue projekta sa generisanim postavkama

Struktura generisanog projekta je dosta slična *Ember* i *React* projektima, te je nije potrebno detaljnije opisivati. Unutar fajla `index.html` se nalazi `div` element s `id`-om `app` koji predstavlja kontejner referenciran iz fajla `src/main.js` unutar koga se instancira *Vue* objekat. Unutar spomenutog objekta se kao template koristi komponenta `App`, definisana u fajlu `src/App.vue`, a koja referencira komponentu `HelloWorld` definisanu u fajlu `components/HelloWorld.vue`.

Zbog velikog stepena sličnosti s projektima drugih razvojnih okvira, za vježbu, ranije implementirane *Vue* komponente je potrebno prebaciti u projekat pod nazivom *vue-book-app* i tom prilikom koristiti *Bootstrap* klase kako bi se uljepšao prikaz ponude i odabira predmeta.

4.4 Osnovne karakteristike razvojnih okvira

Potreba za razvojnim okvirima je rijetko diskutabilna u odnosu na odabir najboljeg ili najefikasnijeg razvojnog okvira. Uzimajući u obzir sve do sada spomenuto, vjerovatno je najadekvatniji odgovor da *niti jedan od JavaScript razvojnih okvira ne predstavlja univerzalno rješenje za sve slučajeve i probleme*. Neki od razvojnih okvira pokazuju veći stepen fleksibilnosti, lakoće u integraciji i interakciji sa drugim komponentama softvera. U tom kontekstu, u nastavku su spomenute osnovne

karakteristike u kojima pojedini razvojni okviri pokazuju superiornost u odnosu na svoju konkurenciju.

Ember – za njega se može reći da, u odnosu na druge koji su ukratko opisani u ovom poglavlju, predstavlja razvojni okvir u punom smislu te riječi. Upravo zbog spomenutog, programerima je neophodno malo više vremena za savladavanje *Embera*. Pokazao se kao dobar kandidat za razvoj dinamičkih SPA (*Single Page Application*), a baziran je na MVC-u. *Ember* je korišten prilikom razvoja aplikacije *LinkedIn*.

React – svoje najbolje karakteristike pokazuje pri razvoju SPA što ukazuje na činjenicu da je pogodan za projekte koji se mogu okarakterisati kao oni manje ili srednje veličine. Jedna od njegovih najvećih prednosti jeste čitava lepeza paketa koji omogućavaju da se implementira određena funkcionalnost čime se osigurava izuzetno velika fleksibilnost. Baziran je na DOM modelu, a jedna od najpopularnijih aplikacija razvijena korištenjem *Reacta* je *Netflix*.

Vue – veliki broj funkcionalnosti je već podržan paketima koji dolaze sa njegovom distribucijom, ali to ni na koji način ne znači da nije dostupno mnoštvo drugih paketa koji se mogu iskorisiti za implementaciju potrebnih funkcionalnosti. Mada se, kao i *React*, preporučuje za projekte manje i srednje veličine, jednostavnost i preglednost su glavne karakteristike ovog razvojnog okvira. Tokom izvršenja uobičajenih akcija, te upravljanja setovima podataka koji nisu pretjerano intenzivni, *Vue* pokazuje zavidan stepen performansi. Neke od *web* aplikacija u okviru kojih je korišten *Vue* su *9gag* i *Nintendo*.

Angular – mada nije detaljnije opisivan u ovom poglavlju, a zbog njegovog značaja, ukratko će biti pomenuto nekoliko njegovih glavnih karakteristika. Ukoliko se razvija aplikacija s velikim brojem funkcionalnosti, dakle govori se o velikim projektima, onda je *Angular* zasigurno jedan od glavnih kandidata. Spomenuto je vezano za njegove mehanizme i dostupne pakete koji omogućavaju upravljanja većim količinama podataka. S druge strane, ne treba zaboraviti činjenicu da s velikom moći dolaze i velike obaveze, pa *Angular* ponekad zahtijeva više podešavanja i upravljanja velikim brojem paketa koji opravdavaju njegovu trenutnu popularnost. *UpWork* i *PayPal* su samo neke od aplikacija razvijenih korištenjem *Angulara*.

5

Principi interakcijskog i vizualnog dizajna

Uspjeh određenog softverskog projekta se teško može predvidjeti, ali je praksa pokazala da određeni koncepti mogu imati značajan utjecaj u osiguranju uspjeha. Spomenuto se prije svega odnosi na činjenicu da korisnici neke aplikacije, bez obzira da li su one besplatne ili se plaćaju, sa zadovoljstvom koriste, dok neke druge, koje imaju istu namjenu i funkcionalnosti, nisu predmet njihovog pretjeranog interesovanja. Jedan od glavnih razloga za to vjerovatno je dizajn aplikacije koji se može sagledati s različitih aspekata, a najznačajniji su dizajn interakcije i vizualni dizajn. Treba naglasiti da se sadržaj ovog poglavlja ne fokusira na specifičnu vrstu aplikacija, već se nastoje predstaviti osnovne norme i prakse koje bi trebale ispuniti očekivanja krajnjih korisnika.

5.1 Dizajn interakcije

Dizajn interakcije se posmatra kao proces kreiranja proizvoda u okviru koga se detaljno analizira način na koji će korisnik ostvarivati interakciju s posmatranim proizvodom. Drugim riječima, kvalitetan dizajn interakcije podrazumijeva sposobnost dizajnera da se uživi u ulogu korisnika i pokuša što realnije simulirati

korisničku interakciju s interfejsom aplikacije koja se dizajnira. Spomenuto podrazumijeva prepoznavanje korisničkih želja i očekivanja. U cilju kreiranja kvalitetnog dizajna interakcije koriste se modeli koji će simulirati korisničku interakciju, te modifikovanje korisničkog interfejsa u skladu s rezultatima korištenih modela. Tom prilikom treba voditi računa da korisnici ostvaruju interakciju sa različitim uređajima i aplikacijama, te tom prilikom formiraju vlastite standarde i očekivanja za koje neupitno naslućuju da će biti ispunjeni i prilikom korištenja aplikacije koja se pokušava dizajnirati.

Kreiranje adekvatnih modela koji će biti korišteni za dizajniranje interakcije je direktno vezano za psihologiju čovjeka i način na koji on doživljava interakciju s okruženjem, bez obzira da li se radi o živoj ili neživoj prirodi. Danas se veoma često mogu susresti najmlađi članovi naše populacije kako pokušavaju ostvariti neku vrstu interakcije s atraktivnim reklamama postavljenim uz šetališta i parkove. Dodirujući pojedine dijelove reklame, koje sadrže kvalitetno obrađene slike, mališani očekuju neku vrstu interakcije i povratne informacije, jer su korištenjem različitih uređaja, usvojili norme za interakciju. Postavljanje određenih normi i očekivanja zahtijeva detaljniju razradu, odnosno korištenje modela kojima se dizajnira interakcija s određenom komponentom²³ i tom prilikom se u obzir uzima sljedeće karakteristike: opažanje, predvidljivost, povratne informacije, intuitivnost i unificiranost.

Opažanje – komponente trebaju biti lako uočljive korisniku, te nedvosmisleno odavati dojam da su namijenjene za interakciju. Pozicioniranje komponenti na neočekivane lokacije, posebno ako one nisu vidljive prilikom inicijalnog učitavanja interfejsa, korisnicima će zasigurno uzrokovati poteškoće u korištenju aplikacije. Također, korištenje neadekvatnih boja, kao što su nijanse sive, mogu korisnika navesti na zaključak da komponenta, zbog neposjedovanja dovoljnih privilegija ili potrebnih podataka, nije dostupna za korištenje ili interakciju.

Predvidljivost – lokacije i izgled određene komponente trebaju korisnika navesti na ispravan zaključak o svojoj namjeni tj. korisnik treba biti siguran šta će biti rezultat interakcije s određenom komponentom. Kao primjer bi se mogao navesti stil *checkbox* elemenata na ilustracija 47 pri čemu korištenje *form-switch* klase korisnika više asocira na funkcionalnost unutar koje treba izvršiti podešavanje nekih postavki aplikacije nego selekciju nekih od ponuđenih predmeta koje želi slušati u određenom

²³ Pojam komponenta se može odnositi na određeni kontrolu ili komponentu s kojom korisnik ostvaruje interakciju. O komponentama, u okviru JavaScript razvojnih okvira, bilo je više riječi u prethodnom poglavlju ove knjige.

semestru. Mada bi se moglo diskutovati koji stil *checkbox* elementa izgleda adekvatnije, nikako se ne treba izostaviti namjena podataka s kojima se upravlja i kontekstu na koji se ti podaci odnose.

Povratna informacija – nakon što je korisnik obavio interakciju s određenom komponentom na predvidljiv način, aplikacija bi u adekvatnoj formi trebala producirati odgovor tj. neku vrstu povratne informacije. Spomenute informacije moraju biti što preciznije kako korisnik ni u jednom momentu ne bi došao u nedoumicu da li je određena akcija uspješno realizovana, zahtijeva ponavljanje ili neke dodatne korake. Kao primjer se može navesti slučaj u kome korisnik, zbog nepostojanja adekvatne povratne informacije (npr. promjene boje dijela komponente, informativne poruke, ponovnog učitavanja podataka i sl.), više puta odabere određenu opciju ili pokrene izvršenje neke akcije. U zavisnosti od načina implementacije, spomenuti slučajevi mogu imati negativne efekte u smislu višestruke modifikacije podataka, a u najblažem slučaju nepotrebno korištenje resursa jer se određene akcije izvršavaju ili pokušaju izvršiti bez stvarne potrebe za njima. Izostanak adekvatnih provjera i implementacije spomenutih akcija bi mogao rezultirati dupliranjem podataka koji bi, u zavisnosti od konteksta, mogli zahtijevati kompleksnu proceduru njihove korekcije. Na primjer, ukoliko bi, zbog neadekvatne implementacije i povratne informacije, višestruka interakcija s određenim dugmićem ili nekom drugom kontrolom, uzrokovala kreiranje više fiskalnih računa, koji bi odmah bili sinhronizovani s Poreskom upravom ili ERP-om kompanije, onda bi česte korekcije (storniranje, poglašavanje računa neaktivnim i sl.) mogle ukazivati na pokušaj malverzacija ili drugih neželjenih aktivnosti.

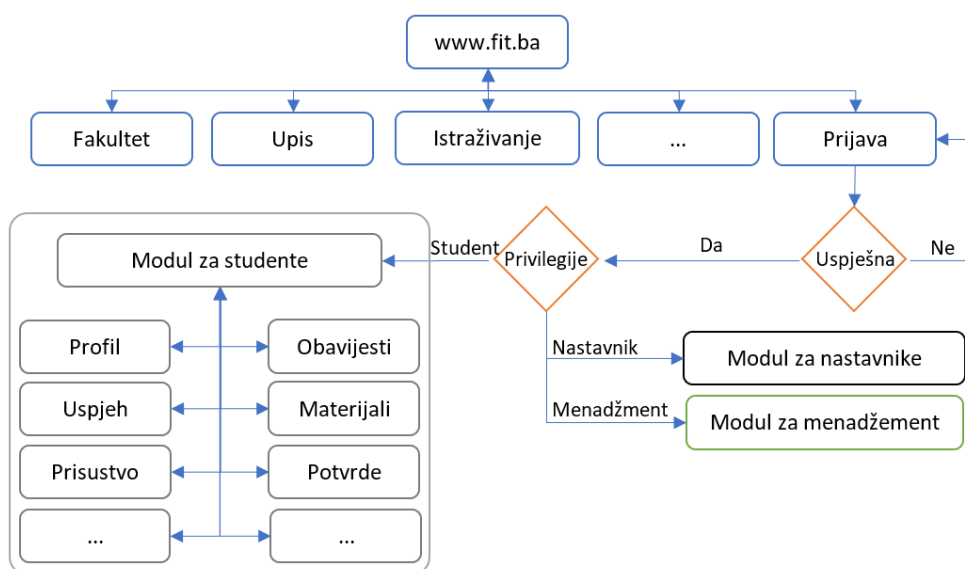
Intuitivnost – kompletna interakcija s korisnikom mora biti implementirana na intuitivan način što podrazumijeva i brzo savladavanje konkretnog procesa ili korištenja komponente. Ukoliko korisnik, prilikom ponovljenog procesa, zastaje na svakom ili većini polja koji zahtijevaju interakciju, onda je intuitivnost poprilično upitna. Spomenuto je posebno karakteristično za osobe starije životne dobi kojima savremene tehnologije i nisu predmet pretjeranog interesovanja ili čak i za mlađe osobe koje nisu navikle na interakciju s kompleksnijim interfejsima. Upravo zbog toga, većina savremenih aplikacija prati opšteprihvaćene norme koje osiguravaju da se korisnici osjećaju ugodno i u poznatom okruženju čak i pored činjenice da prvi put koriste aplikaciju.

Unificiranost – bez obzira da li je korisnik napravio dužu ili kraću pauzu u interakciji s aplikacijom koja se razvija, ponavljanje određenog procesa ne bi trebao predstavljati problem. Spomenuti postulat je potrebno zadržati i prilikom interakcije sa svim

dijelovima aplikacije, a to nije problematično postići ukoliko se koriste standardi i najbolje prakse koje se često susreću kod najkorištenijih aplikacija današnjice. Ipak, pretjerana unificiranost može graničiti s monotonošću, te svakako treba ostaviti mogućnost fokusiranja korisničke pažnje na određene elemente u adekvatnom momentu o čemu će biti više riječi u nastavku poglavlja koji govori o vizualnom dizajnu.

Prilikom planiranja razvoja veliki značaj treba posvetiti definisanju toka interakcije ili slijeda koraka koji bi trebao biti logičan, pri čemu, unutar definisanog toka, sve korištene komponente trebaju biti implementirane na način da posjeduju ranije pobrojane karakteristike. U cilju postizanja što adekvatnijeg i unificiranijeg interakcijskog slijeda, uobičajeno je da se kreiraju vizualni modeli interakcije koji mogu poslužiti i drugim akterima u procesu razvoja da steknu dojam o budućem izgledu i načinu interakcije s aplikacijom. Spomenuto predstavlja i idealan momenat za ukazivanje na eventualna odstupanja ili potencijalne probleme u implementaciji.

Prilikom kreiranja globalnog modela interakcije ne razmatraju se detalji vezani za dizajn, već se prvenstveno definiše slijed interakcije između pojedinih dijelova. Koristeći primjer stranice Fakulteta informacijskih tehnologija Univerziteta „Džemal Bijedić“ u Mostaru, na **Error! Reference source not found.** je prikazan primjer slijeda događaja u okviru stranice www.fit.ba.



Ilustracija 55 - Dizajn interakcije stranice www.fit.ba

Razvoj savremenih aplikacija bi se teško mogao zamisliti bez korištenja predložaka ili templejta dizajna (engl. Design Patterns) koji predstavljaju efikasan način ili opšteprihvaćeni pristup za rješavanje nekog od čestih izazova ili problema. Mada su, idejno, proizašli iz područja urbanističkog planiranja, i ne odnose se direktno na dijelove softvera s kojima korisnik ostvaruje direktnu interakciju, predlošci su neizostavna komponenta svih softverskih proizvoda. Naime, urbanističko porijeklo je uzrokovano željom da se spriječi pojava u kojoj bi, u različitim dijelovima svijeta, inženjeri konstantno radili na rješavanju istih problema. U tom kontekstu je usvojen pristup prema kome bi se najbolje prakse i rješenja (npr. način izgradnje specifičnih kupola, mostova i drugih objekata), u formi predložaka, učinile dostupnim svima i na taj način svaki put radilo na unapređenju postojećih, a nikako na ponovnom iznalaženju stih ili sličnih rješenja.

Predlošci dizajna softvera kao što su singleton, factory i mnogi drugi, zahvaljujući univerzalnosti, predstavljaju aksiome za uspješnu i efikasnu implementaciju projekta. Uzimajući u obzir spomenuto, područje dizajniranja interakcije nije ostalo imuno na potrebu za definisanjem predložaka ili univerzalnih rješenja. Naravno, u kontekstu interakcije, oni se odnose na rješenja koja se tiču samog dizajna i poteškoća u interakciji. Većina savremenih aplikacija posjeduje veliki broj sličnih korisničkih zahtjeva ili funkcionalnosti, pa postojanje provjerenih rješenja predstavlja značajnu olakšicu i relativnu garanciju uspjeha. Spomenuto je posebno karakteristično za slučajeve u kojima se određena aplikacija koristi u različitim okruženjima npr. *web* aplikacija koju koriste desktop i mobilni uređaji.

Prilikom prvog korištenja aplikacije, većina korisnika će rezervisati samo mali dio vremena za snalaženje u novom okruženju i pronalazak željenih funkcionalnosti. Upravo zbog toga, neadekvatna navigacija predstavlja jedan od glavnih razloga zbog kojih korisnici odustaju od korištenja aplikacije. Prevencija odustajanja korisnika i stvaranje osjećaja ugodnosti od prvih momenata korištenja je ukazala na potrebu za definisanjem navigacijskih predložaka kojim bi se unificirala navigacijska logika kroz aplikaciju tj. prikaz dostupnih funkcionalnosti ili meni (engl. menu). Adekvatno označavanje menija, odnosno korištenje prepoznatljivih ikona je potrebno unificirati, bez obzira da li se aplikaciji pristupa s desktop računara koji posjeduje 32" monitor ili mobilnog uređaja sa 5" ekranom. Moglo bi se reći da je ikonografija vezana za označavanje menija danas poprilično standardizirana, na što ukazuje i jednostavna pretraga pojmova „menu icon“ u nekom od pretraživača koja bi trebala rezultirati sličnim prikazom kao na ilustracija 56.

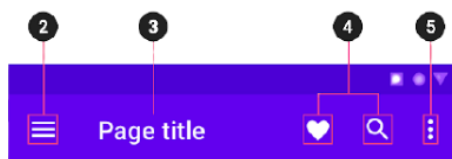


Ilustracija 56 Ikone za označavanje menija - dostupnih opcija

Korištenjem neke od ikona s Ilustracija 56 bi zasigurno trebala zadovoljiti karakteristike opažanja i predvidljivost, jer korisnici već posjeduju uspostavljenu asocijaciju između navedenih ikona i prikaza dostupnih funkcionalnosti. Pored menija, svakako ne treba zanemariti ni ostale ikone koje su korisnicima dobro poznate, a neke od njih su prikazane u narednoj tabeli.

Ikona	Opis
	Pretraga
	Poruke
	Notifikacije
	Lokacija
	Opcije, podešavanja

Glavni akteri u području savremenih tehnologija, kao što je na primjer kompanija Google, definišu standarde koji se odnose kako na segment dizajna aplikacija tako i kompletne interakcije s korisnicima. U tom kontekstu treba pomenuti Material Design kao set preporuka koji je doživio veliku popularnost još od 2014. godine kada je predstavljen javnosti. Na Ilustracija 57 je prikazan primjer Material Design aplikacijske trake na kojoj je s oznakom 2 označen meni opisan na Ilustracija 56, s oznakom 4 dobro poznate ikone za pretragu i dodavanje posmatranog entiteta u listu omiljenih, te sa oznakom 5 je označen akcijski meni (najčešće označen sa engl. overflow menu).



Ilustracija 57 Elementi Material Design aplikacijske trake²⁴

Pored unificiranja ikonizacije, praksa je pokazala da je korisnicima od izuzetne važnosti da u svakom momentu znaju u kojem dijelu aplikacije se nalaze. Za tu svrhu se, kao što je prikazano na Ilustracija 58, najčešće koriste navigacijske putanje (engl. navigation breadcrumbs) koje omogućavaju jednostavnije prebacivanje na neki od ranije posjećenih dijelova aplikacije ili samo za potrebe orijentacije u odnosu na kompletan proces koji pokušavaju realizovati. Na primjer, ako se pokušava izvršiti neka narudžba ili kompleksnija registracija koja se realizuje kroz nekoliko koraka, onda je korisniku jako značajan podatak o tome u kojoj fazi kompletnog procesa se trenutno nalazi.

Modul za studente

Početna stranica > Profil > Uspjeh > Prva godina

Ilustracija 58 Primjer navigacijske putanje

Interakcija s korisnikom u većini slučajeva zahtijeva unos podataka u različitom formatu, bez obzira da li se radi o registraciji korisnika, popunjavanju određenog zahtjeva (npr. narudžbe, izdavanje računa, poreske prijave, vakciniranje protiv COVID-19 virusa i dr.), definisanju vrijednosti filtera pretrage i sl. Kontrole namijenjene unosu podataka koji posjeduju različite formate (npr. datum, vrijednost s oznakom valute, specijalni znakovi i dr.) svakako bi trebali pružati određeni stepen fleksibilnosti jer njihov unos, posebno ako on nije jasno naglašen i preciziran, obično predstavlja dodatni napor. Naime, u spomenutim slučajevima, korisnik se primorava da više puta unosi iste vrijednosti kako bi se prilagodio očekivanjima koja nisu pretjerano jasno definisana.

Na Ilustracija 599 je prikazan primjer u kome se od korisnika zahtijeva unos datuma rođenja koji može biti u različitim formatima. U prvom slučaju korisnik ne dobija nikakvu informaciju o tome kakav format datuma treba unijeti, a mogućih opcija je poprilično mnogo npr. 11.05.2000, 11/05/2000, 11-5-2000, 5/11/2000, 11/5/2000,

²⁴ <https://material.io/components/app-bars-top#anatomy>

2000/5/11 i td. U druga dva slučaja se, na različite načine, korisniku sugeriše koji je očekivani format unosa datuma rođenja, što će značajnije ubrzati interakciju s korisnikom.

Datum rođenja:	
Datum rođenja (dd.mm.gggg):	
Datum rođenja:	dd.mm.gggg

Ilustracija 59 Definisanje formata podataka

Posebna pažnja se mora posvetiti interakciji u kojoj se zahtijeva unos ličnih podataka, a koje su korisnici sve manje voljni dijeliti. Trajni gubitak povjerenja zasigurno će nastupiti u slučajevima kada se od korisnika zahtijevaju osjetljivi podaci nakon čijeg unosa ne bude moguće kompletirati proces npr. kreiranja narudžbe ili naručeni artikla, odnosno servis, nikada ne bude ustupljen krajnjem korisniku. Jedan od preporučenih pristupa podrazumijeva odgođenu registraciju u kontekstu da se od korisnika odmah ne zahtijevaju osjetljivi podaci, već samo osnovni, a da se za dostupnost nekih naprednijih funkcionalnosti zahtijeva kompletnija registracija. Kao primjer se mogu navesti *web* stranice za edukaciju (npr. *Pluralsight*) ili pregled video sadržaja (npr. *Netflix*) koji omogućavaju pregled kratkih sadržaja ili besplatan period korištenja.

Pored spomenutih predložaka dizajna postoji i grupa označena kao *anti predlošci* (engl. anti-patterns), a sadrži opis pristupa koji u praksi i nisu ostvarili najbolje rezultate, odnosno korisniku otežavaju interakciju s aplikacijom. Mada nisu nepoznanica, obično se pojavljuju u nedostatku vremena potrebnog za dizajniranje i testiranje interakcijskih modela.

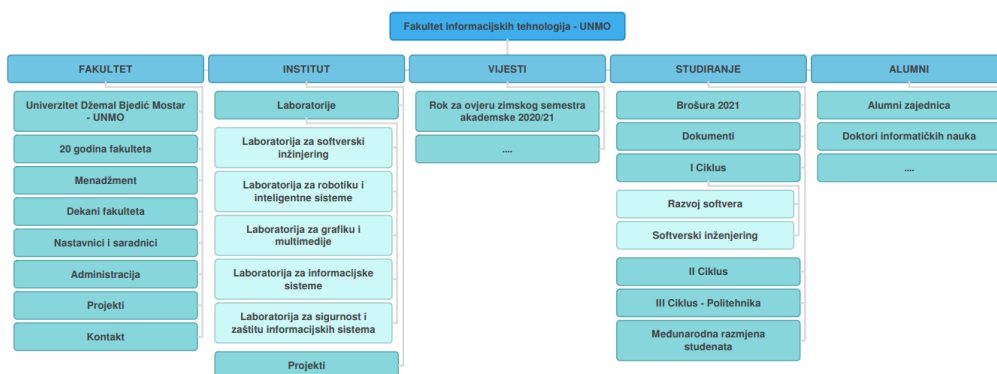
Navigacijski anti-predložak bi se mogao odnositi na neadekvatno isticanje elemenata koji služe za navigacijsku interakciju. Spomenuto će od korisnika zahtijevati dodatni napor kako bi identificirao elemente u okviru sadržaja koji će osigurati početak ili nastavak procesa koji se realizuje. Kao primjer bi se mogli navesti slučajevi u kojima se navigacijski meni nalazi u okviru akcijskog menija (obrnuta lokacija opcija dostupni poda oznakama 2 i 5) predstavljenog na Ilustracija 57.

Pored navigacijskog anti-predložka, particioniranje sadržaja koji bi, u zavisnosti od korištenog uređaja za prikaz, mogao uzrokovati da određeni, nerijetko bitan, sadržaj (npr. napomene vezane za dodatne troškove, uslove isporuke, zatezne kamate,

prijavu ispita, i sl.) ostane nevidljiv za krajnjeg korisnika. Particioniranje podrazumijeva postojanje prostora između dijelova sadržaja što uzrokuje slučajeve u kojima korisnik nije svjestan postojanja dodatnog sadržaja, te bezbrižno nastavlja finaliziranje započetog procesa. U cilju eliminisanja spomenutih slučajeva potrebno je koristiti adekvatan indikator koji ukazuje na dodatni sadržaj ili kontrole koje uzrokuju nastavak procesa postaviti u dijelovima koji će zahtijevati pregled kompletnog sadržaja (npr. na dnu stranice ili sadržaja).

Zasigurno često susretani anti-predložak, koji je i dalje itekako prisutan, podrazumijeva obavještanje korisnika da određeni uneseni podaci nisu validni bez jasne identifikacije tih podataka, što je u direktnoj vezi sa slučajevima predstavljenim na Ilustracija 599. Ne treba posebno naglašavati negativne aspekte slučaja u kome neuspjela validacija uzrokuje gubitak svih unesenih podataka, što će zasigurno na strani korisnika uzrokovati manjak strpljenja i želje za okončanjem procesa.

Nerijetko, čak i napredniji korisnici povremeno osjećaju određeni stepen napora prilikom korištenja poprilično jednostavnih aplikacija, što će, kako je i ranije naglašeno, kod korisnika, okarakterisanih kao početnici, uzrokovati pojavu želje za odustajanjem. Jedan od mogućih razloga za spomenuto svakako može biti kompleksnost u navigaciji i dostupnosti pojedinih funkcionalnosti, a sve zbog neadekvatnog planiranja. Kao jedan od mogućih načina za eliminisanje kompleksnosti svakako je i kreiranje mape dijelova aplikacija koja će jasno ukazati na jednostavnost i realnost ruta prilikom pristupa pojedinim funkcionalnostima, te potencijalne nedoumice koje se mogu pojaviti na strani korisnika. Mapa aplikacije predstavlja listu dostupnih funkcionalnosti poredanih u određenoj hijerarhijskoj formi. Primjer mape *web* stranice Fakulteta informacijskih tehnologija Univerziteta „Džemal Bijedić“ je prikazan na Ilustracija 600.



Ilustracija 60 Mapa web stranice Fakulteta informacijskih tehnologija

Mapa *web* stranice jasno predstavlja logičku organizaciju pojedinih cjelina, te putanje kojima korisnik može doći do željenog dijela stranice ili funkcionalnosti. U navigacijskom kontekstu, stranica može biti kreirana da prati strukturalni pristup koji podrazumijeva da se do podataka o nekoj od laboratorija (npr. Laboratoriji za grafiku i multimediju) prvenstveno mora posjetiti glavna stranica Fakulteta, nakon toga glavna stranica sekcije Institut, glavna stranica sekcije Laboratorije, pa tek nakon toga stranica željenog laboratorija.

Pored strukturalne, može se govoriti i o asocijativnoj navigaciji koja omogućava da se, prilikom posjete određenom dijelu stranice ili nekoj obavijesti koja posjeduje zajednički sadržaj (npr. ključne riječi, tagove i sl.), direktno pristupi drugim dijelovima *web* stranice bez obzira što oni trenutno nisu na posmatranoj hijerarhijskoj ruti.

Navigacijsku strukturu je poprilično jednostavno organizovati u slučaju da se odnosi na aplikaciju sa manjim brojem funkcionalnosti. Međutim, kompleksnije aplikacije zahtijevaju različite nivoe navigacijske infrastrukture, od kojih je jedna globalna i najčešće implementirana u okviru navigacijskog menija lociranog u gornjem dijelu aplikacije. Kao što je navedeno na ilustracija 61, globalni navigacijski dio obično sadrži i logo kompanije kao i linkove na glavne funkcionalnosti. Odabirom nekog od linkova iz globalnog navigacijskog nivoa korisnik se usmjerava na drugi dio aplikacije koji će sadržavati opcije karakteristične za specifičnu funkcionalnost. Uzimajući u obzir činjenicu da se korisnici najugodnije osjećaju u poznatom okruženju, kompleksnije aplikacije uvijek zadržavaju globalni navigacijski bar dostupan krajnjem korisniku, a unutar vertikalnih/bočnih dijelova prikazuju funkcionalnosti i navigaciju specifičnu za taj dio aplikacije. Spomenuti linkovi pripadaju lokalnom navigacijskom sistemu.

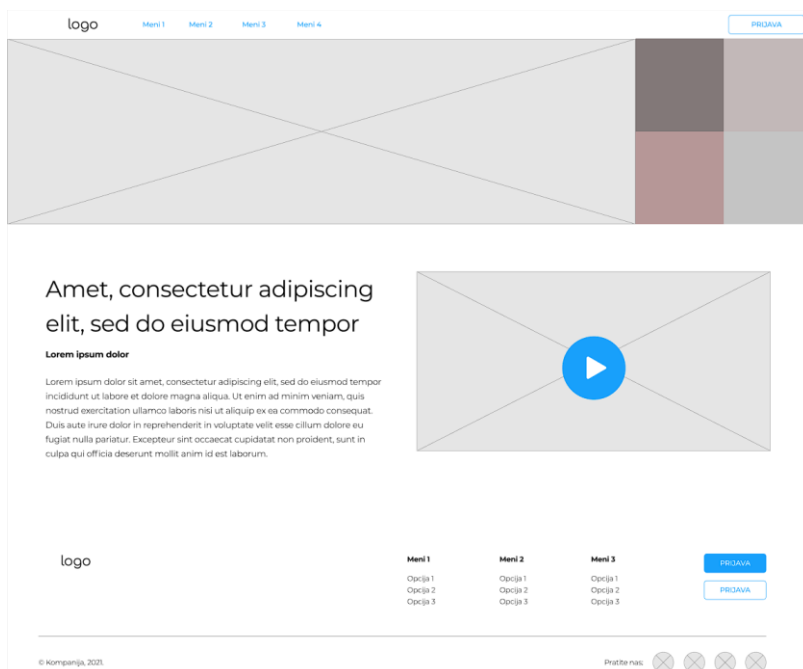
Posljednjih godina, popularnost jednostraničnih ili SPA (*Single Page Application*) aplikacija je izuzetno porasla. Pored velikog broja prednosti, kao što je eliminisanje potrebe za kompleksnijim navigacijskim nivoima, jedna od glavnih nedostataka svakako je činjenica da nisu pogodne za razvoj kompleksnije aplikacije. Također, jednostranične aplikacije kao i druge aplikacije koje prezentuju veliku količinu sadržaja, korisnicima trebaju osigurati jednostavan način pronalaska i pregleda sadržaja, a jedan od načina za to je korištenje sidra (engl. *anchor*) koji omogućavaju jednostavno fokusiranje željenog dijela sadržaja.

Interakcija korisnika s aplikacijama se od samih početaka ostvaruje korištenjem tastature i miša. Mada je, od njihove pojave, tehnologija itekako uznapredovala, oni su i dalje gotovo nezamjenjive komponente. Mada, zbog implementacijskih ograničenja, još uvijek nemaju pretjerano široku primjenu, glasovne komande predstavljaju jedan od načina interakcije od koga se očekuje da preuzme primat u skorijoj budućnosti. Trenuto se kvalitetnom podrškom interpretaciji glasovnih komandi mogu pohvaliti prvenstveno vodeće svjetske kompanije koje su virtualne asistente (npr. *Siri*, *Google Assistant*, *Cortana*) implementirali unutar vlastitih operativnih sistema namijenjenih mobilnim i desktop uređajima. Aplikacije koje posjeduju podršku za govorne komande u značajnoj mjeri ubrzavaju interakciju i korisniku stvaraju osjećaj sigurnosti jer je planirane akcije u stanju realizovati ponekad i na neočekivan način. Pomenute aplikacije obično posjeduju adekvatan VUI (*Voice User Interface*) koji korisniku signalizira da je interakcija moguća i korištenjem glasovnih komandi. U takvim slučajevima se vrši i tekstualna interpretacija kompletne interakcije što korisniku omogućava da se uvjeri kako je aplikacija, odnosno asistent razumio njegovu poruku. Glasovna interakcija s aplikacijom je od neizmjernog značaja za osobe koji imaju slabiji vid, te bi se na taj način u listu korisnika aplikacije mogao uključiti i taj dio populacije koji po brojnosti, ali i svim drugima karakteristikama, nikako ne treba zanemariti.

5.2 Vizualni dizajn

Prilikom kreiranja vizualnog dizajna jedna od glavnih principa o kojima treba voditi računa je naglašavanje najznačajnijih komponenti na način da one svojom pozicijom, dimenzijama ili bojom privuku pažnju korisnika. Korištenje nekih od ranije opisanih predložaka dizajna, koji sadrži malo više detalja nego što je u inicijalnoj verziji potrebno, ali inkorporira preporuke vezane za vizualni dizajn, opisane u nastavku, i dizajn interakcije, prikazan je na Ilustracija 61.

Za kreiranje predložaka dizajna moguće je koristiti neki od postojećih alata koji omogućavaju crtanje različitih oblika (npr. neki od *Microsoft Office* alata). Spomenuti alati ne posjeduju predefinisane sadržaje niti pružaju preporuke vezane za pozitivne prakse u oblasti dizajna ili interakcije, te je u takvim slučajevima potrebno koristiti neki od profesionalnih grafičkih alata, a neki, ne pretjerano popularni se, u ograničenim funkcionalnostima, mogu koristiti besplatno online.



Ilustracija 61 Izgled predložka dizajna web stranice www.fit.ba

Sasvim je realno očekivati da će veći elementi privlačiti veću pažnju korisnika, dok će manji elementi odavati dojam podređenosti. U kontekstu podređenosti, udaljenost između elemenata, odnosno njihovo grupisanje, također mogu stvarati osjećaj pripadnosti. U cilju demnostracije, kako se dimenzijama i bojom može naglasiti određeni element, na Ilustracija 62 je prikazan jednostavan primjer u kome se dugmić *Prijava* ni na koji način ne razlikuje od drugih dugmića (slika lijevo) u odnosu na slučaj kada je on jasno naglašen (slika desno).

Ilustracija 62 Naglašavanje elemenata forme

Pored navedenog, prijavom na modul za studente, najveći stepen naglašenosti bi trebale posjedovati bitne obavijesti u okviru kojih se koristi podebljani naslov i/ili ikone jarkih boja, ispitni termini koji se uskoro organizuju, novi nastavni materijali i dr. Ukoliko bi se posmatrala aplikacija za online prodaju, naglasak bi bio na slikama proizvoda koje bi imale zadatak da privuku korisnika. Tek nakon iskazivanja interesa, korisnik bi imao mogućnost da pristupi detaljima vezanim za opis proizvoda. Identičan pristup se koristi i prilikom naglašavanja određenih popusta ili najznačajnijih karakteristika koji posmatrani proizvod diferencira od ostalih. Nakon određenog vremena korisnici kreiraju percepciju o načinu korištenja pojedinih elemenata, a prvenstveno prema sličnosti. Na primjeru Ilustracija 62 korisnici će plavi dugmić prihvatiti kao element kojim se označava pokretanje akcije ili procesa.

Značajni efekti se mogu ostvariti korištenjem animacija kojima se elementi, obično noviteti ili oni s kojima se mora ostvariti interakcija u cilju nastavka procesa, žele naglasiti. Animacije u posljednjem periodu preuzimaju primat u kontekstu interaktivne pomoći u korištenju aplikacije, a posebno prilikom njenog prvog pokretanja ili pristupa.

Elementi unutar aplikacije koji se odnose na isti entitet ili komponentu (npr. obavijest) se obično organizuju prema nekoj hijerarhiji, te se na taj način pojedinim elementima daje veći značaj. Alati za formiranje hijerarhije su najčešće veličina i boja elemenata, te bi, na primjeru obavijesti, naslov trebao imati najveće dimenzije, nakon toga kratki sadržaj, te na kraju elementi koji zauzimaju najniži rang u hijerarhiji, a to su datum kreiranja i ime kreatora obavijesti (**Error! Reference source not found.** Ilustracija 63).

WRD :: Rezultati ispita :: 25.09.2021

Web razvoj i dizajn | Elmir Babović | 29.09.2021 15:38

Ispitu iz predmeta Web razvoj i dizajn održanom 25.09.2021. godine pristupilo je 180 studenata pri čemu 73 studenta nisu položila eliminacijski test. U narednoj tabeli je prikazan osvojeni broj bodova...

Ilustracija 63 Hijerarhija na primjeru obavijesti

Prateći hijerarhijski pristup, redoslijed elemenata također može naglasiti značaj. Kao primjer se može navesti procedura registracije u okviru koje će se veoma rijetko prvo unositi datum rođenja, spol ili broj telefona.

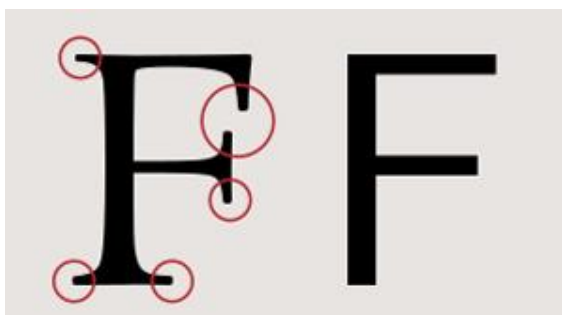
Unificiranost u dizajnu je izuzetno poželjna osobina svake aplikacije, ali svakako treba voditi računa o balansu kako bi se korisnici, u ključnim trenucima, mogli fokusirati na bitnije sadržaje. Na Ilustracija 64 je prikazan primjer naglašavanja sadržaja koji je bitan za korisnika, a zahvaljujući ikonografiji i bojama obavijesti ovog formata ne bi trebale ostati neprimjećene. Naime, boje su gotovo uvijek asocirane s određenim osjećajima, te crvena uvijek koristi označavanje hitnosti.



Ilustracija 64 Obavijest korisniku o bitnim događajima

Prilikom dizajniranja uopšte, veoma često se koristi pravilo 60-30-10 prema kome 60% prostora treba zauzimati glavna boja (npr. ako se govori o životnom prostoru onda bi on mogao pripadati zidovima), 30% pomoćna boja (npr. dijelovi namještaja, prozora i sl.), a 10% boja koja ima zadatak kreirati naglasak (npr. jastuci, slike i sl.). Gotovo identičan raspored bi se mogao preslikati i u vizualni dizajn aplikacija što se jednim dijelom može primijetiti u prethodnim ilustracijama.

Značajan aspekt vizualnog dizajna zasigurno pripada odabiru fonta koji će najadekvatnije predstaviti određeni brend, uslugu i sl. Kada je riječ o fontovima, danas postoji veliki broj različitih fontova (Calibri, Gorgia, Helvetica, Times New Roman i mnogi drugi), ali od posebnog značaja za vizualni dizajn je poznavanje njihove podjele u dvije kategorije, a to su Serif i Sans Serif. Glavna razlika između pomenutih kategorija je u činjenici da, za razliku od Sense Serif, fontovi iz kategorije Serif posjeduju dekorativne nastavke na krajevima znakova kao što je vidljivo na Ilustracija 65.



Ilustracija 65 Serif i Sense Serif kategorije fontova ²⁵

U najpoznatije članove Serif kategorije fontova se ubrajaju: Times New Roman, Georgia i Courier New, dok su: Arial, Helvetica i Calibri neki od najreprezentativnijih članova Sense Serif fontova. Poznavanje pripadnosti pojedinim kategorijama u značajnoj mjeri determinira i odabir fonta za aplikaciju i vizualni identitet jer bi npr. kompanije koje kupcima i posjetiocima žele ostaviti dojam tradicije i povjerenja trebali birati neki od fontova iz Serif kategorije. S druge strane, Sens Serif je namijenjen više jednostavnim, kreativnim, modernističkim idejama i pristupima, prvenstveno targetirajući mlađu populaciju i trendove. Posmatrajući primjere na Ilustracija 63, Ilustracija 64 i Ilustracija65 može se zaključiti o vrsti publike koju targetira dizajnirana aplikacija.

Veliki broj kompanija posjeduje vlastiti font koji koriste u svim dokumentima i javnim istupima, te ih čak dostavljaju poslovnim partnerima kako se prilikom isticanja njihovih proizvoda ili usluga ne bi narušio vizualni identitet.

U ovom poglavlju su predstavljeni samo osnovni koncepti koji trebaju biti uzeti u obzir prilikom kreiranja dizajna aplikacije. Bez obzira da li se radi o interakciji ili vizualnom dizajnu, postoji još mnogo detalja kojima se u ovom segmentu treba ovladati. Ipak, ne treba zaboraviti efekte koje kvalitetan dizajn sa sobom donosi, te da će uvijek predstavljati magnet za korisnike.

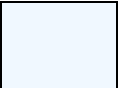
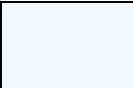
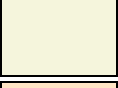
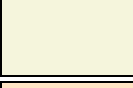
²⁵ <https://www.fonts.com/content/learning/fontology/level-1/type-anatomy/serif-vs-sans-for-text-in-print>

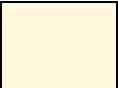
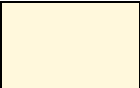
Reference

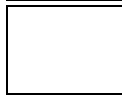
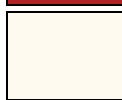
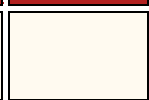
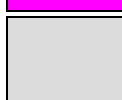
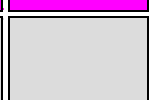
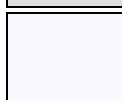
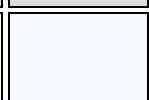
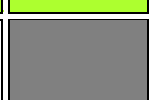
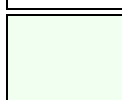
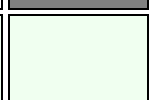
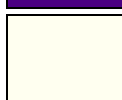
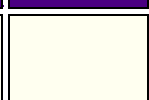
- [1.] Ember.js in Action; Joachim Haagen Skeie; Manning Publications; ISBN: 9781617291456, 2014
- [2.] Building Web Apps with Ember.js; Jesse Cravens, Thomas Q Brady; O'Reilly Media; ISBN: 978-1-449-37092-3; 2014
- [3.] React: Up & Running: Building Web Applications, Stoyan Stefanov, ISBN: 978-1491931820, O'Reilly Media; 1st edition, 2016
- [4.] Fullstack React: The Complete Guide to ReactJS and Friends, Anthony Accomazzo, Nate Murray, Ari Lerner, ISBN: 978-0991344628, Fullstack.io, 2017
- [5.] Fullstack Vue: The Complete Guide to Vue.js, Hassan Djirdeh, Nate Murray, Ari Lerner, ISBN: 978-1987595291, CreateSpace Independent Publishing Platform, 2018

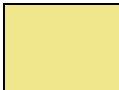
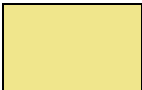
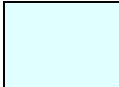
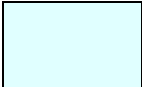
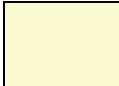
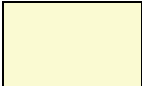
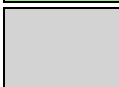
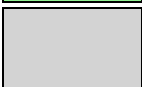
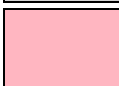
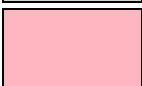
Prilozi

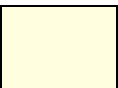
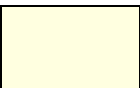
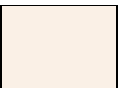
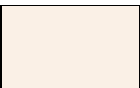
CSS paleta boja s nazivima i pripadajućim vrijednostima

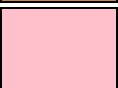
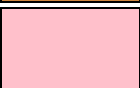
Name d	Numeri c	Color name	Hex rgb	Decimal
		<i>aliceblue</i>	#F0F8FF	240,248,255
		<i>antiquewhite</i>	#FAEBD7	250,235,215
		<i>aqua</i>	#00FFFF	0,255,255
		<i>aquamarine</i>	#7FFFD4	127,255,212
		<i>azure</i>	#F0FFFF	240,255,255
		<i>beige</i>	#F5F5DC	245,245,220
		<i>bisque</i>	#FFE4C4	255,228,196
		<i>black</i>	#000000	0,0,0
		<i>blanchedalmond</i>	#FFEBCD	255,235,205
		<i>blue</i>	#0000FF	0,0,255
		<i>blueviolet</i>	#8A2BE2	138,43,226
		<i>brown</i>	#A52A2A	165,42,42
		<i>burlywood</i>	#DEB887	222,184,135
		<i>cadetblue</i>	#5F9EA0	95,158,160
		<i>chartreuse</i>	#7FFF00	127,255,0
		<i>chocolate</i>	#D2691E	210,105,30
		<i>coral</i>	#FF7F50	255,127,80
		<i>cornflowerblue</i>	#6495ED	100,149,237

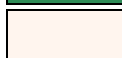
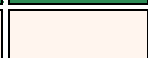
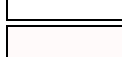
		<i>cornsilk</i>	#FFF8DC	255,248,220
		<i>crimson</i>	#DC143C	220,20,60
		<i>cyan</i>	#00FFFF	0,255,255
		<i>darkblue</i>	#00008B	0,0,139
		<i>darkcyan</i>	#008B8B	0,139,139
		<i>darkgoldenrod</i>	#B8860B	184,134,11
		<i>darkgray</i>	#A9A9A9	169,169,169
		<i>darkgreen</i>	#006400	0,100,0
		<i>darkgrey</i>	#A9A9A9	169,169,169
		<i>darkkhaki</i>	#BDB76B	189,183,107
		<i>darkmagenta</i>	#8B008B	139,0,139
		<i>darkolivegreen</i>	#556B2F	85,107,47
		<i>darkorange</i>	#FF8C00	255,140,0
		<i>darkorchid</i>	#9932CC	153,50,204
		<i>darkred</i>	#8B0000	139,0,0
		<i>darksalmon</i>	#E9967A	233,150,122
		<i>darkseagreen</i>	#8FBC8F	143,188,143
		<i>darkslateblue</i>	#483D8B	72,61,139
		<i>darkslategray</i>	#2F4F4F	47,79,79
		<i>darkslategrey</i>	#2F4F4F	47,79,79
		<i>darkturquoise</i>	#00CED1	0,206,209
		<i>darkviolet</i>	#9400D3	148,0,211
		<i>deeppink</i>	#FF1493	255,20,147
		<i>deepskyblue</i>	#00BFFF	0,191,255

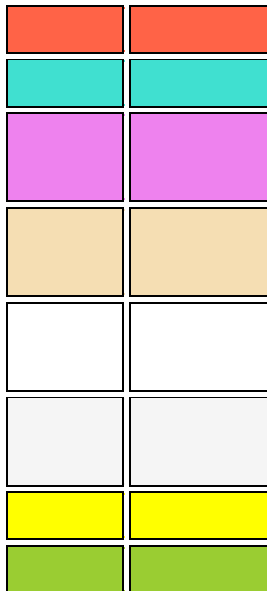
		<i>dimgray</i>	#696969	105,105,105
		<i>dimgrey</i>	#696969	105,105,105
		<i>dodgerblue</i>	#1E90FF	30,144,255
		<i>firebrick</i>	#B22222	178,34,34
		<i>floralwhite</i>	#FFFAF0	255,250,240
		<i>forestgreen</i>	#228B22	34,139,34
		<i>fuchsia</i>	#FF00FF	255,0,255
		<i>gainsboro</i>	#DCDCDC	220,220,220
		<i>ghostwhite</i>	#F8F8FF	248,248,255
		<i>gold</i>	#FFD700	255,215,0
		<i>goldenrod</i>	#DAA520	218,165,32
		<i>gray</i>	#808080	128,128,128
		<i>green</i>	#008000	0,128,0
		<i>greenyellow</i>	#ADFF2F	173,255,47
		<i>grey</i>	#808080	128,128,128
		<i>honeydew</i>	#F0FFF0	240,255,240
		<i>hotpink</i>	#FF69B4	255,105,180
		<i>indianred</i>	#CD5C5C	205,92,92
		<i>indigo</i>	#4B0082	75,0,130
		<i>ivory</i>	#FFFFF0	255,255,240

		<i>khaki</i>	#F0E68C	240,230,140
		<i>lavender</i>	#E6E6FA	230,230,250
		<i>lavenderblush</i>	#FFF0F5	255,240,245
		<i>lawngreen</i>	#7CFC00	124,252,0
		<i>lemonchiffon</i>	#FFFACD	255,250,205
		<i>lightblue</i>	#ADD8E6	173,216,230
		<i>lightcoral</i>	#F08080	240,128,128
		<i>lightcyan</i>	#E0FFFF	224,255,255
		<i>lightgoldenrodyellow</i>	#FAFAD2	250,250,210
		<i>lightgray</i>	#D3D3D3	211,211,211
		<i>lightgreen</i>	#90EE90	144,238,144
		<i>lightgrey</i>	#D3D3D3	211,211,211
		<i>lightpink</i>	#FFB6C1	255,182,193
		<i>lightsalmon</i>	#FFA07A	255,160,122
		<i>lightseagreen</i>	#20B2AA	32,178,170
		<i>lightskyblue</i>	#87CEFA	135,206,250
		<i>lightslategray</i>	#778899	119,136,153

		<i>lightslategrey</i>	#778899	119,136,153
		<i>lightsteelblue</i>	#B0C4DE	176,196,222
		<i>lightyellow</i>	#FFFFE0	255,255,224
		<i>lime</i>	#00FF00	0,255,0
		<i>limegreen</i>	#32CD32	50,205,50
		<i>linen</i>	#FAF0E6	250,240,230
		<i>magenta</i>	#FF00FF	255,0,255
		<i>maroon</i>	#800000	128,0,0
		<i>mediumaquamarine</i>	#66CDAA	102,205,170
		<i>mediumblue</i>	#0000CD	0,0,205
		<i>mediumorchid</i>	#BA55D3	186,85,211
		<i>mediumpurple</i>	#9370DB	147,112,219
		<i>mediumseagreen</i>	#3CB371	60,179,113
		<i>mediumslateblue</i>	#7B68EE	123,104,238
		<i>mediumspringgreen</i>	#00FA9A	0,250,154
		<i>mediumturquoise</i>	#48D1CC	72,209,204
		<i>mediumvioletred</i>	#C71585	199,21,133
		<i>midnightblue</i>	#191970	25,25,112
		<i>mintcream</i>	#F5FFFA	245,255,250
		<i>mistyrose</i>	#FFE4E1	255,228,225

		<i>moccasin</i>	#FFE4B5	255,228,181
		<i>navajowhite</i>	#FFDEAD	255,222,173
		<i>navy</i>	#000080	0,0,128
		<i>oldlace</i>	#FDF5E6	253,245,230
		<i>olive</i>	#808000	128,128,0
		<i>olivedrab</i>	#6B8E23	107,142,35
		<i>orange</i>	#FFA500	255,165,0
		<i>orangered</i>	#FF4500	255,69,0
		<i>orchid</i>	#DA70D6	218,112,214
		<i>palegoldenrod</i>	#EEE8AA	238,232,170
		<i>palegreen</i>	#98FB98	152,251,152
		<i>paleturquoise</i>	#AFEEEE	175,238,238
		<i>palevioletred</i>	#DB7093	219,112,147
		<i>papayawhip</i>	#FFEFD5	255,239,213
		<i>peachpuff</i>	#FFDAB9	255,218,185
		<i>peru</i>	#CD853F	205,133,63
		<i>pink</i>	#FFC0CB	255,192,203
		<i>plum</i>	#DDA0DD	221,160,221
		<i>powderblue</i>	#B0E0E6	176,224,230

		<i>purple</i>	#800080	128,0,128
		<i>red</i>	#FF0000	255,0,0
		<i>rosybrown</i>	#BC8F8F	188,143,143
		<i>royalblue</i>	#4169E1	65,105,225
		<i>saddlebrown</i>	#8B4513	139,69,19
		<i>salmon</i>	#FA8072	250,128,114
		<i>sandybrown</i>	#F4A460	244,164,96
		<i>seagreen</i>	#2E8B57	46,139,87
		<i>seashell</i>	#FFF5EE	255,245,238
		<i>sienna</i>	#A0522D	160,82,45
		<i>silver</i>	#C0C0C0	192,192,192
		<i>skyblue</i>	#87CEEB	135,206,235
		<i>slateblue</i>	#6A5ACD	106,90,205
		<i>slategray</i>	#708090	112,128,144
		<i>slategrey</i>	#708090	112,128,144
		<i>snow</i>	#FFFAFA	255,250,250
		<i>springgreen</i>	#00FF7F	0,255,127
		<i>steelblue</i>	#4682B4	70,130,180
		<i>tan</i>	#D2B48C	210,180,140
		<i>teal</i>	#008080	0,128,128
		<i>thistle</i>	#D8BFD8	216,191,216



tomato
turquoise
violet
wheat
white
whitesmoke
yellow
yellowgreen

#FF6347	255,99,71
#40E0D0	64,224,208
#EE82EE	238,130,238
#F5DEB3	245,222,179
#FFFFFF	255,255,255
#F5F5F5	245,245,245
#FFFF00	255,255,0
#9ACD32	154,205,50

